

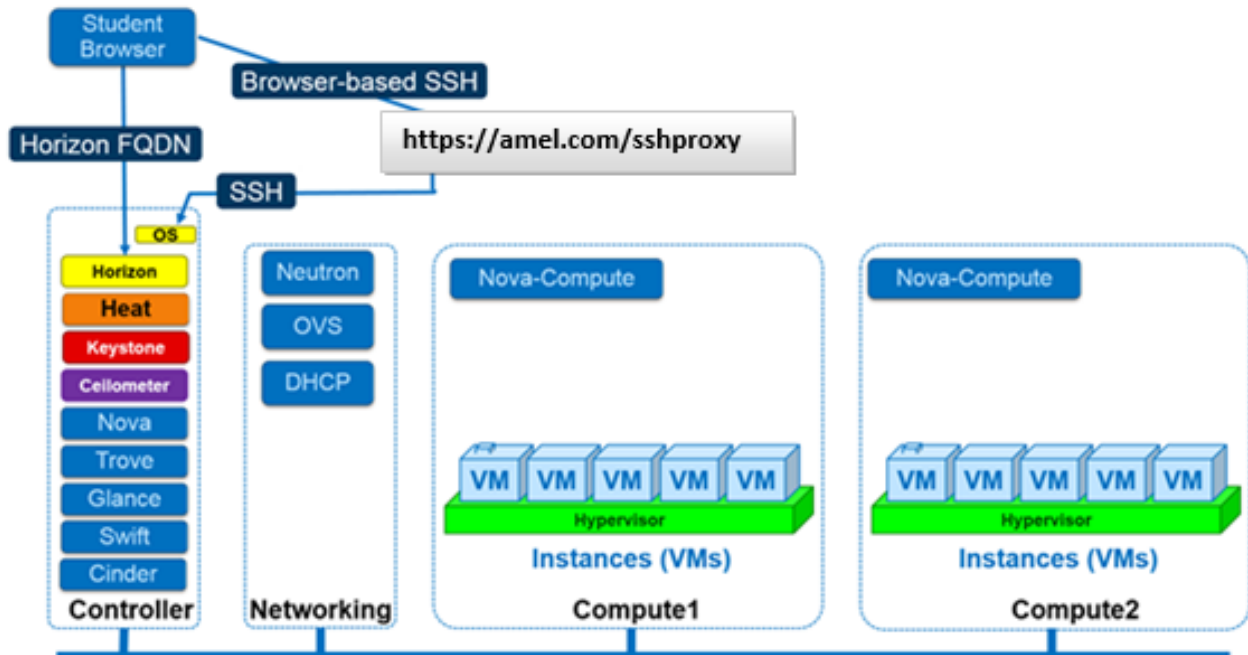
OpenStack Environment

Requirement:

VMware

Install Ubuntu/ CentOS 7.2 Operating System

OpenStack environment:



2. While holding down the control key, click the link with your name to connect to Horizon. If your name is not here contact us ASAP.

Your Horizon FQDN :

<http://awsl.amel.training>

<http://stuar.amel.training>

<http://agus.amel.training>

<http://awen.amel.training>

<http://told.amel.training>

<http://twlf.amel.training>

3. While holding down the control key, click: <https://amel.com/sshproxy> to connect to your SSH proxy.

4. Answer the following questions to confirm that you understand the lab environment:

- How many browser tabs will you have open to do these labs?
- When you are asked to connect via command line, will everyone use the same URL? If so, what is it?
- When you connect to OpenStack Horizon, what URL will you browse? (Hint, find your name on the table above)
- How many nodes are in your OpenStack lab environment?

- e) When connecting your your lab environment, which node will you initially connect?
- f) How will you SSH to the other nodes?
- g) Are the Networking services (Neutron) on the same node as your controller?
- h) Where will VMs run?

01 Introduction to administering an OpenStack cloud.

Although you install each OpenStack service separately, the OpenStack services work together to meet your cloud needs: Identity, Compute, Image service, Block Storage, Networking (neutron), Object Storage, Orchestration, and Telemetry. These services may be distributed or local to one another, but in all instances, after you have authenticated yourself through Identity (keystone), you use OpenStack APIs to create and admin OpenStack cloud resources.

1. Servers and Nova Services

Using some basic CLI tools can help give you a sense of the shape of your OpenStack cloud; number of servers, the health of each server (Up / Down), and what services are running where.

2026-03-18
2026-03-18
2026-03-18
2026-03-18
2026-03-18
2026-03-18

1. SSH to your controller and log in as root

```
[root@controller]# nova-manage service list | sort
```

Binary	Host	Zone	Status	State	Updated At
nova-cert	controller.localdomain	internal	enabled	:-)	2026-03-18 19:24:49
nova-compute	compute1.localdomain	nova	enabled	:-)	2026-03-18 19:24:57
nova-compute	compute2.localdomain	nova	enabled	:-)	2026-03-18 19:24:54
nova-conductor	controller.localdomain	internal	enabled	:-)	2026-03-18 19:24:49
nova-consoleauth	controller.localdomain	internal	enabled	:-)	2026-03-18 19:24:49
nova-scheduler	controller.localdomain	internal	enabled	:-)	2026-03-18 19:24:50

2. Look at the output and ask yourself the following questions... (record your answers for now, we'll discuss them at the conclusion of the lab).
 - o How many compute servers are running?
 - o Where are the compute servers running?
 - o How many OpenStack cloud controllers are running?
 - o What is the current state of all of your services?

A State of :-) means the service is up and running, whereas XXX means the service is no longer available.

2. Checking OpenStack Version

1. Source the keystone_admin file

```
[root@controller]# source keystone_admin
```

This command will be explained in lab coming up, for right now, just know it has to do with setting proper permissions within the bash shell; a shortcut that tells the OpenStack cloud that we are the user admin. Your CLI prompt will also change

2. Display the service catalog for the OpenStack admin

```
[root@controller ~(keystone_admin)]# keystone catalog | less
```

For students who do not have much experience using the less command:

Press CTRL + F to go forward one window, and CTRL + B to go back one window
Press CTRL + D to go forward one half window, and CTRL + U to go back one half window
Press q to quit Less utility and return to the CLI
If you have never used the less utility, you might find the following cheat sheet useful.

(<http://sheet.shiar.nl/less>)

- o Answer the following questions:
- o What is the adminURL of the following services?
- o What IP address are these services being served from?
- o Whose IP address is this?
- o How are they different?
- o How many services are there?

3. List the nova service endpoints, does this output look familiar?

```
[root@controller ~(keystone_admin)]# nova service-list
```

4. Collect the various version releases of your OpenStack services by issuing the following commands. What will be displayed is the release date of each service.

```
[root@controller ~(keystone_admin)]# keystone-manage --version  
[root@controller ~(keystone_admin)]# nova-manage --version  
[root@controller ~(keystone_admin)]# glance-manage --version
```

5. Let's correlate the release date to our version. Navigate in browser to: [OpenStack Releases](#)
6. Look over the chart and notice that the version of Kilo is the same version that was being displayed each time you issued the `[$SERVICE]-manage --version` command.

- OpenStack is a project with ambitious release dates. Take a minute to read over the notes concerning how the project is maintained.
7. That's it for this lab! Hang tight until the other students finish up.

02 OpenStack cloud with the Horizon Dashboard as both an admin and user.

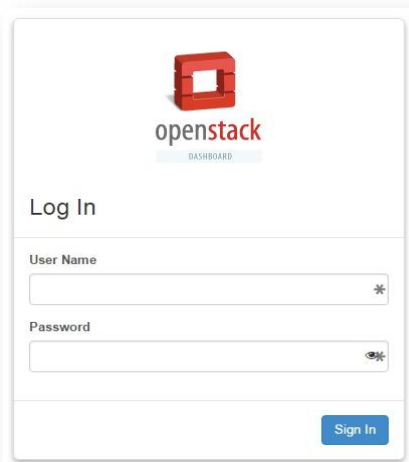
Let's role play a bit. Your new OpenStack deployment has gone live and you are ready to take on your first customers. These may be paying customers or perhaps a particular department within your organization.

For the purposes of this lab, we'll take two orders. The first order we'll provision via the OpenStack Horizon GUI, the second order, we'll complete with the CLI.

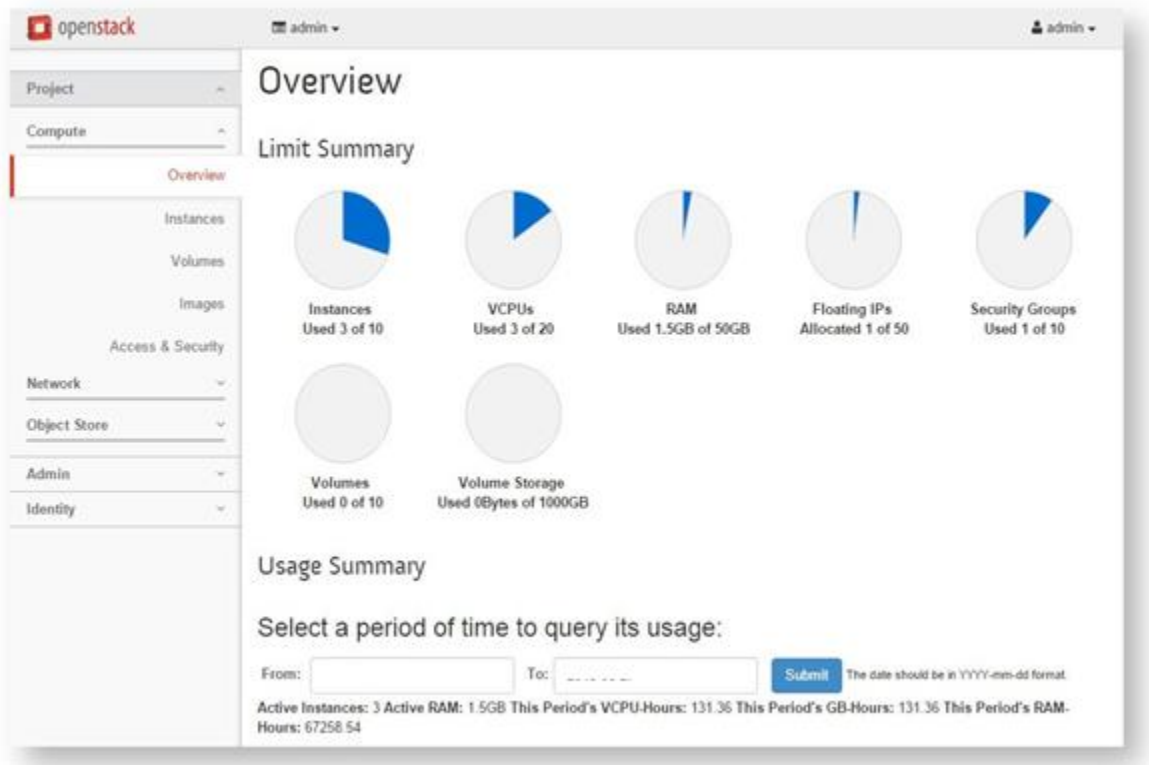
- Order 1: Acme Inc; they would like two users, Amel VerdySon, and Irene Magda to be able to admin a small cloud for their company.
- Order 2: VaultTek Enterprises; they would like Chester Copperpot to be able to admin a small cloud for their company.

1. Navigating Horizon Admin Interfaces

1. To get started, if you are not already, **sign into the OpenStack web GUI** as the user admin (<http://<userid>.amel.training>)



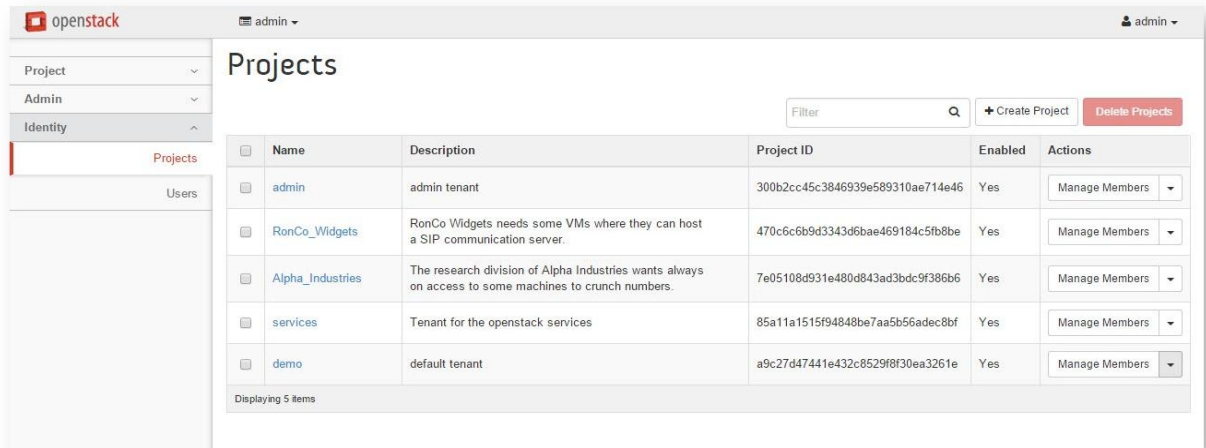
- Before we get started, **click on Project > Compute > Overview**



One more thing we can take note of: in the upper right part of the screen is the user you are currently logged in as. In the upper left side of the screen is the project you are currently working within. Look at the screenshot below. The project drop down menu and user drop down menu have both been clearly defined with large red arrows.

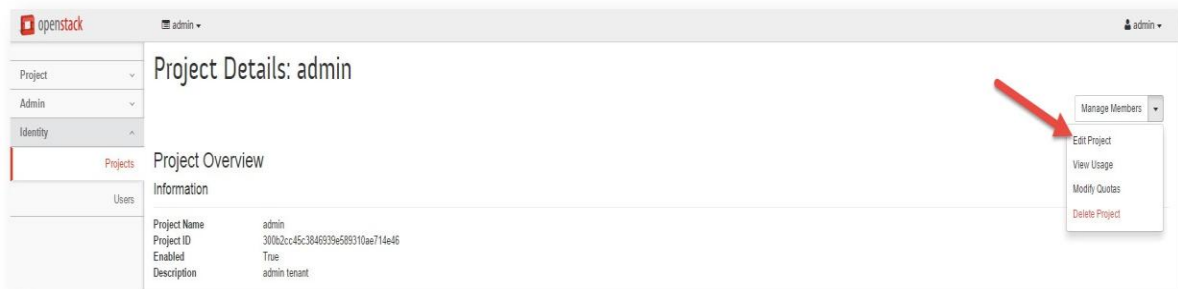


- On the left hand side screen **click on Identity > Projects** You won't have the same projects on your machine, and that's fine, but below is a projects page that an admin might see.



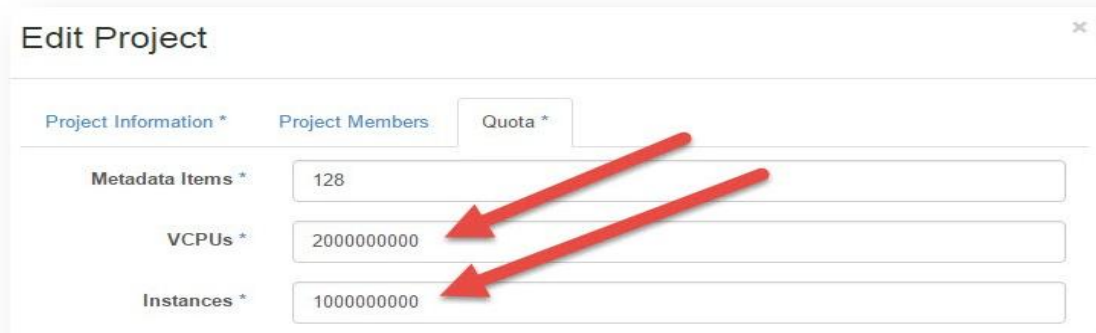
Note: In OpenStack, users belong to Projects. Projects and tenants are interchangeable terms, used to refer to a group of users, and the total amount of cloud resources those users may consume (for, say, switching on instances). When OpenStack was new, compute had its own authentication mechanism and used to term 'project'. Authorization is now handled by the Keystone service, where it uses the word 'tenant'. The admin tenet has their own default project. This can be useful location to host virtualized services that are integrated into the OpenStack deployment.

- To start, let's edit the project we are already a part of, On the Projects screen, in the Name column **click on the blue admin link**
- You should now be on the 'Project Details: admin' page. In the drop down menu in the upper right, **select 'Edit Project'**.



- In the Edit Project box that pops up, click on the **Quota tab**
- Change VCPUs from its current value**, to something absurd. After all, we're just trying to drive home a point. How about 2 billion? Change the value of VCPU to read, VCPU: 2000000000

8. Change Instances from its current value, to something absurd. **Change the value of instances to read, Instances: 1000000000**



The screenshot shows the 'Edit Project' dialog box with the 'Quota' tab selected. The 'Instances' field is set to 1000000000 and the 'VCPU's' field is set to 2000000000. Two red arrows point to these fields.

Field	Value
Metadata Items *	128
VCPU's *	2000000000
Instances *	1000000000

9. After you've made those two changes, **click the 'Save' button at the bottom of the screen.**
10. **Navigate to Project > Compute > Overview**

Examine the pie charts for Instances, and VCPUs. They've changed radically, but only to reflect the permissions our project has been given. Hopefully it is clear that we couldn't possibly have just increased the resource capacity of our system by a factor of 100 million by simply changing the value of a variable.

11. To view actual available system resources, **click on Admin > System > Hypervisors**. You're welcome to go there now, but we'll devote another lab to exploring this section later. So for now, don't get too caught up in it.

What we want to take away from this section is that projects have associated maximum thresholds that are defined by the project's quotas. Quotas may exceed the actual resource capacity of the total cloud system, therefore the size of a project may be limited by the quota, or resource capacity of the system itself.

2. Adding a Project with Horizon

Consider that we are currently signed into the system as the admin for the OpenStack deployment. You would never want to give this account out to your customers, so let's create a new project and apply some users.

Users may belong to many projects, but they always must belong to at least one. If a user belongs to a project, and that project is deleted, then that user will not be able to log into the Horizon dashboard until they are once again assigned (at least) one project.

Before we create new users, let's first create a few projects (tenants).

1. Log in as admin to the Horizon interface, and **navigate to Identity > Projects**.
2. In the upper right hand corner, **click the '+Create Project' button**. The following screenshot has a giant red arrow on the button you looking for.



3. On the Project Information tab, **input the following Project Information** (do *NOT* click Create Project yet!)

Name: acme_inc
Description: Acme Inc. is the world's leader in the production of widgets
Enabled: (checked)

The screenshot shows the 'Create Project' dialog box. It has three tabs: 'Project Information *', 'Project Members', and 'Quota *'. The 'Project Information *' tab is selected. It contains three fields: 'Name *' with the value 'acme_inc', 'Description' with the text 'Acme Inc. is the world's leader in the production of widgets', and 'Enabled' with a checked checkbox. At the bottom right, there are 'Cancel' and 'Create Project' buttons.

4. **Click on the 'Project Members' tab**. We don't yet have any users from the Acme Inc organization, so we won't apply any yet.
5. Now **click on 'Quota'**

This page controls the operational limits of a project could place on available resources within our infrastructure. For example, we certainly don't want our customer, Acme Inc, to max out our cloud's storage space with widget designs.

6. Let's set an appropriate quota:

Metadata Items: 128

When booting an instance, you can also add arbitrary meta data so that you can more easily identity it among other running instances. This value limits the length of the string where 128 is the default.

VCPUs: 2

VCPUs are the number of available virtualized cores

Instances: 2

The number of instances, or VMs, that maybe created

Injected Files: 5

Injected files are arbitrary local files that can be placed into the instance file system at the time of creation.

Injected File Content (Bytes): 10240

Limits total size (in bytes) of allowable injected files.

Volumes: 2

Volume-Snapshots: 2

Total-Size-of-Volumes-and-Snapshots-(GB): 200

Total accrued size of all stored data.

RAM-(MB): 51200

Security-Groups: 5

The total available security groups this project may create

Security-Group-Rules: 20

The total available security group rules this project may create

Floating-IPs: 4

Total number of floating IP addresses the project may utilize. Floating IPs are applied to instances.

Networks: 3

Number of virtualized networks that may be created

Ports: 50

Routers: 3

Subnets: 3

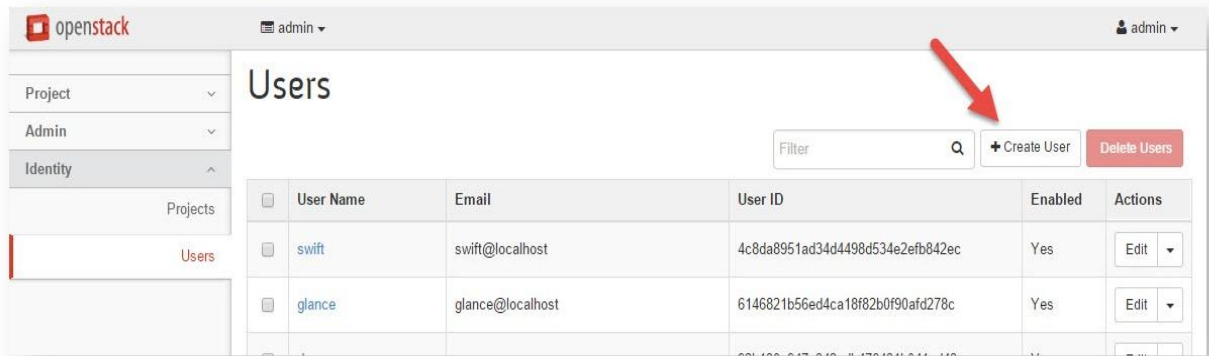
Maximum number of subnets that may be created by a project.

7. You are now permitted to **click the Create Project button**

3. Adding Users with Horizon

Acme Inc wanted two users to be available to admin their systems, Alice Anderson and Bob Barker, so lets add them now that we have the project space created.

1. Log in to Horizon as admin. On the left hand side of the OpenStack Horizon GUI, **click on Identity > Users**
2. In the upper right hand corner, **click the '+Create User' button.**



3. Fill out the 'Create User' screen as follows and **click Create User**

White spaces and capitals may be used, but they make the login more complex, so please don't use them.

Password: fa5tpa55w0rd
 ConfirmPassword: fa5tpa55w0rd
 UserName: amelirene
 Email: <blank>
 PrimaryProject: acme_inc
 Role: _member_

Create User

Password *

fa5tpa55w0rd

Confirm Password *

fa5tpa55w0rd

User Name *

aliceanderson

Email

Primary Project *

acme_inc

Role *

member

Description:

Create a new user and set related properties including the Primary Project and Role.

Cancel

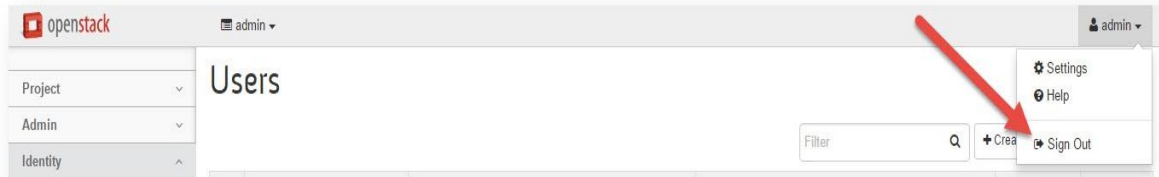
Create User

4. Create a second user, this will be the Bob Barker profile.

Fill out the second 'Create User' screen and **click Create User**

```
Password: fa5tpa55w0rd
ConfirmPassword: fa5tpa55w0rd
UserName: bobbarker
Email: <blank>
PrimaryProject: acme_inc
Role: _member_
```

5. Log out the admin account within the Horizon GUI by clicking in the upper right hand corner, and **click 'Sign Out'**.



6. Log in as ameliorene:fa5tpa55w0rd
7. **Click on Project > Compute > Overview**

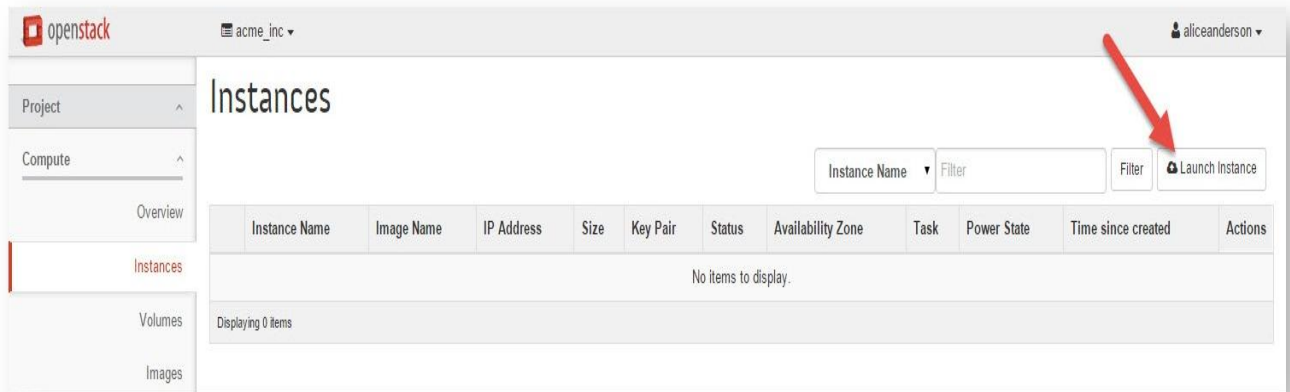
This screen should look quite a bit different than the overview screen you saw when you were logged in as admin. Notice that Alice Anderson may only create a total of 2 instance, and utilize a total of 2 VCPUs. These settings reflect what was provisioned in the Project's Quota tab.

If you forget what the admin quota screen looked like, sign out, and sign back in as admin. It is important that you understand that Alice Anderson is a user within a project (tenant), therefore she may only utilize the resources that have been made permissible to the project for which she belongs (acme_inc).

Make sure you clearly understand this partitioning of available system resources into projects (tenants) via quotas before continuing the lab. We're not racing here. Take all the time you need.

4. Launching Instances as a User (not admin)

1. Double check that you are signed in as ameliorene:fa5tpa55word
Notice that the project in the upper left change has changed. You should be signed in as the user ameliorene, within the project acme_inc.
2. **Click on Project > Compute > Instances**
3. In the upper right corner, **click 'Launch Instance'**



4. The Launch Instance dialog box should pop up, on the Details tab, set the settings as follows:

```
AvailabilityZone: nova
InstanceName: acme_vm1
Flavor: m1.xlarge
InstanceCount: 1
InstanceBootSource: Boot from image
ImageName: cirros
```

5. **Click 'Launch'**, and the instance will fail.

That's OK, just driving home the point that the project acme_inc does not have enough resources provisioned by their quota to boot the m1.xlarge flavor. Afterall, the m1.xlarge flavor wants to run 4 VCPUs, and we only have 2 available to our project.

6. Let's try again, this time, configure the instance details tab as follows:

```
AvailabilityZone: nova
InstanceName: acme_vm1
Flavor: m1.tiny
InstanceCount: 1
InstanceBootSource: Boot from image
ImageName: cirros
```

7. This instance still is not ready for us to launch. **Click on the 'Networking' tab**
8. **Add the network 'private' and 'public'** to the instance, by clicking the + sign beside those networks.

Launch Instance

Details * Access & Security **Networking *** Post-Creation Advanced Options

Selected networks

NIC-1 private (20ca30a2-e3fb-4e62-aa78-08dc471e93ed) [x]

NIC-2 public (31250e3e-fe20-421e-98f0-41a368627507) [x]

Available networks

Choose network from Available networks to Selected networks by push button or drag and drop, you may change NIC order by drag and drop as well.

Cancel **Launch**

- Once complete, **click the 'Launch' button**

Immediately upon launching the instance (virtual machine), it will begin spawning. This may take a few minutes, and that's OK. How long the instance takes to spawn is completely dependent on how many resources the instance has been created with, and how the the underlying OpenStack architecture has been designed. Spawning m1.tiny should not take longer than 3 or 5 minutes.

Instances

Instance Name Filter Filter Launch Instance Terminate Instances More Actions

Instance Name	Image Name	IP Address	Size	Key Pair	Status	Availability Zone	Task	Power State	Time since created	Actions
acme_vm1	cirros		m1.tiny	-	Build	nova	Spawning	No State	0 minutes	Associate Floating IP

Displaying 1 item

- Once instance changes from 'Spawning' to 'Running', **sign out of the amelirene account**
- Log in as** bobbarker:fa5tpa55w0rd
- Click on **Project > Compute > Overview**

You should see the same total available resources that the account amelirene saw. Remember, both Alice and Bob are part of the same project (acme_inc)

13. Click on Project > Compute > Instances

You should still see the same instance that the account amelirene saw. Again, both Alice and Bob are part of the same project (acme_inc)

14. Logged in as bobbarker, click on the 'Launch Instance' button, then create the following instance:

```
AvailabilityZone: nova
InstanceName: acme_vm2
Flavor: m1.tiny
InstanceCount: 1
InstanceBootSource: Boot from image
ImageName: cirros
```

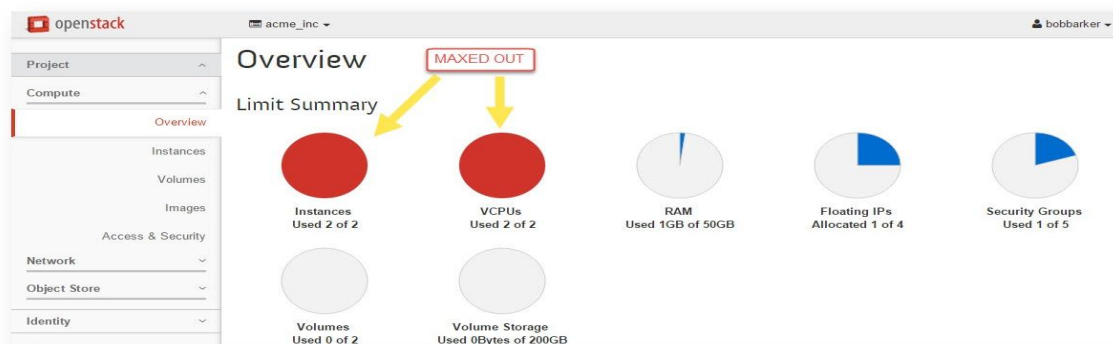
15. Click on the 'Networking' tab

16. Add the network 'private' and 'public' to the instance, but clicking the + sign beside those networks.

17. Once complete, click the 'Launch' button, and wait for the acme_vm2 instance to finish spawning. Again, this may take a few moments.

18. Once acme_vm2 is running, click on Project > Compute > Overview

Spend a few minutes checking out this screen. Notice that no more instances or VCPUs are available for this project. It should look similar to the screen shot below:



19. Navigate to Project > Compute > Instances > Instance Name: acme_vm2 > Log Tab

What is the login and password for this vm? Hint look in bottom of the log entries.

5. Taking control with admin

1. Log out of bobbarker, and log back in as admin.
2. **Click on Admin > System > Instances**
3. Check all the tick boxes beside the running instances. **Click 'Terminate Instances'**. All instances should begin shutting down.
4. Once more log out of admin, and log back in as amelirene or bobbarker.

The acme_inc project should once again have no instances running. Check the Project > Compute > Overview dashboard and the Project > Compute > Instances dashboard to be sure.

Terminating an instance will kill the instance (forever)

In this case, the *admin* user terminated a machine created by *bobbarker*. You may be wondering why bobbarker did not terminate the machine that he created, and indeed that is normally how it is done. In this case, we just wanted to show you that if the admin has access to the tenant, then the admin can also terminate the tenant's machines.

 **03 Create a project and user at the CLI (same as we did in Horizon), and introduce the concept of controlling OpenStack user permissions with 'roles'.**

Created projects and users in the previous lab, but we used the Horizon dashboard. In production, it is highly unlikely that system admins will spend much time within Horizon.

1. Project, Users, and Roles definitions (reference)

1. An OpenStack admin will manage projects, users, and roles.
 - **Projects** are organizational units in the cloud to which you can assign users. Projects are also known as tenants or accounts.
 - **Users** can be members of one or more projects.

- **Roles** define which actions users can perform. You assign roles to user-project pairs.

2. Creating a Project and User at the CLI

Back to our story. Remember, we still promised our customer Vault Tek Enterprise that they could have access to a project space with a single user, Chester Copperpot. Let's try setting this user up at the CLI.

1. SSH to your controller as root, you might use:
 - While holding the control key, click: (<https://amel.com/sshproxy>)
2. Apply the admin access variables to the bash environment. It prevents you from issuing needlessly long CLI commands. This will be covered in another lesson if it has not been covered already.

```
[root@controller]# source keystone_admin
```

3. Keystone's job is management of users and credentials. Issue this command to the keystone service to create a new tenant.

```
[root@controller ~(keystone_admin)]# keystone tenant-create --name vault_vtech
```

Notice the deprecation warning message that came up? If you didn't, let's check it out now.

```
/usr/lib/python2.7/site-packages/keystoneclient/shell.py:65:
DeprecationWarning: The keystone CLI is deprecated in favor of python-
openstackclient. For a Pyth
, continue using python-keystoneclient.
'python-keystoneclient.', DeprecationWarning)
```

The openstack cli has been rewritten (python openstack wrapper). These commands will still work, at least for now, but support has moved to the new cli client. You'll have an opportunity to access the python-openstackclient soon.

4. Create the user chestercopperpot, within the tenant (project) vault_tek, the password fa5tpa55w0rd, and the email address chester@vault_vtech.example

```
[root@controller ~(keystone_admin)]# keystone user-create --name
chestercopperpot --tenant vault_tek --pass fa5tpa55w0rd --email
chester@vault_vtech.example
```

If this command is successfully run, you should see output similar to below:

Property	Value
email	chester@vault_vtech.example
enabled	True

id	c9b7d3be7fe843c78412ff510457ddd3
name	chestercopperpot
tenantId	a411280ca390483ea2161209371d133a
username	chestercopperpot

- Display the tenants (projects) currently available.

```
[root@controller ~(keystone_admin)]# keystone tenant-list
```

The results look like this and are analogous to the Identity > Projects page in the Horizon dashboard.

id	name	enabled
67f9b771e7f24bcc9109a01005f96c3a	acme_incAcme Inc. is the worlds leader in production of widgets	True
300b2cc45c3846939e589310ae714e46	admin	True
a9c27d47441e432c8529f8f30ea3261e	demo	True
85a11a1515f94848be7aa5b56adec8bf	services	True
31c02925465b4da5b8f18cfaa6fba382	temp_tenant	True
39ad6e677744194bf6baf036762e7f6	the_shire	True
8fd4710d774466f9685af6fa69aaa3	vault_tek	True

- Display the users that have been created in the system.

```
[root@controller ~(keystone_admin)]# keystone user-list
```

The results look like this:

id	name	enabled	email
e9483918caa940a48fe882ba092c7e0b	admin	True	root@localhost
ba62896bc6c0474bb552946e097ed71f	amelirene	True	
489a6706a097475099e5d2d4aa024c11	bobbarker	True	
9b1f9b36d7354b1ea121ccfbae339d8b	ceilometer	True	ceilometer@localhost
abafc32941b6465885378aa6d5c74a1e	chestercopperpot	True	chester@vault_vtech.example
e96c13b017b341a79808ebc9f04afa98	cinder	True	cinder@localhost
92b130a947e342adb476431b041ed43c	demo	True	
a6f37fe1d7e84b68a6846c9987a79409	gandalf	True	gandalf@theshire.example
6146821b56ed4ca18f82b0f90afd278c	glance	True	glance@localhost
ff5f50ecf064e65bd45d2d0b91bbdab	neutron	True	neutron@localhost
d3125ab7d2eb4ba0a23d480eee988419	nova	True	nova@localhost
4c8da8951ad34d4498d534e2efb842ec	swift	True	swift@localhost

- Log in to the Horizon interface as chestercopperpot // fa5tpa55w0rd

- If you have forgotten how to get to Horizon, simply navigate to your FQDN in a web browser

- Once logged in, you should be part of the new project called vault_tek.

- How can you quickly identify the project that you are a part of in Horizon?

HINT: Look in the upper left corner of your screen.

- We logged into Horizon in the previous lab, but if you are new to OpenStack, click around, and make sure everything appears as you would expect it to within the OpenStack Horizon dashboard.

9. That's it! Good job. Leave everything the way it is. We'll tweak the `acme_inc` and `vault_tek` tenants (projects) in a subsequent lab. If you are done early, feel free to perform the subsequent section on using the `python-openstackclient`.

3. Exploring roles at the CLI

OpenStack uses a Role Based Access Control (RBAC) mechanism to manage accesses to resources. Users' roles granted on each project and domain are stored into Keystone, and can be updated through Keystone's API. However, policy enforcement (actually allowing or not the access to resources according to a user's roles) is performed independently in each service, based on the rules defined in each `policy.json` file.

You assign roles to user-project pairs. Define actions for OpenStack service roles in the `/etc/project/policy.json` files. For example, define actions for Compute service roles in the `/etc/nova/policy.json` file.

1. First let us find all of the `policy.json` files on the system.

```
[root@controller ~(keystone_admin)]# find / -name 'policy.json'
```

Note that there is a policy for nova, keystone, cinder, glance, neutron, and ceilometer

```
/etc/nova/policy.json
/etc/keystone/policy.json
/etc/cinder/policy.json
/etc/glance/policy.json
/etc/neutron/policy.json
/etc/ceilometer/policy.json
```

2. Great! Now let's take a look at what one of these files contains.

```
[root@controller ~(keystone_admin)]# cat /etc/cinder/policy.json | less
```

For students who do not have much experience using the `less` command: Press CTRL + F to go forward one window, and CTRL + B to go back one window Press CTRL + D to go forward one half window, and CTRL + U to go back one half window Press q to quit Less utility and return to the CLI

If you have never used the `less` utility, you might find the following cheat sheet useful (<http://sheet.shiar.nl/less>) The rules contained inside of these files may be customized. Customizing these rules controls who may call on a particular service. For example, maybe you want to create a user that may ONLY create a virtual router with neutron, or ONLY create cinder volumes. This is all made possible with roles.

Each policy is defined by a one-line statement in the form `"" : ""`.

Examples:

```
"volume:get": "", Means anyone can do a "volume:get": "",
```

"volume:get": "rule:admin_api", Means that ONLY admin_api can "volume:get_volume_admin_metadata"

"volume:get": "!", Means NO ONE EVER can "volume:get_volume_admin_metadata"

```
{
  "context_is_admin": "role:admin",
  "admin_or_owner": "is_admin:True or project_id:!(project_id)s",
  "default": "rule:admin_or_owner",
  "admin_api": "is_admin:True",
  "volume:create": "",
  "volume:delete": "",
  "volume:get": "",
  "volume:get_all": "",
  "volume:get_volume_metadata": "",
  "volume:get_volume_admin_metadata": "rule:admin_api",
  "volume:delete_volume_admin_metadata": "rule:admin_api",
  "volume:update_volume_admin_metadata": "rule:admin_api",
  "volume:get_snapshot": "",
  "volume:get_all_snapshots": "",
  "volume:extend": "",
  "volume:update_readonly_flag": "",
  "volume:retype": "",
  "volume_extension:types_manage": "rule:admin_api",
  "volume_extension:types_extra_specs": "rule:admin_api",
  "volume_extension:volume_type_access": "",
  "volume_extension:volume_type_access:addProjectAccess": "rule:admin_api",
  "volume_extension:volume_type_access:removeProjectAccess": "rule:admin_api",
  "volume_extension:volume_type_encryption": "rule:admin_api",
  "volume_extension:volume_encryption_metadata": "rule:admin_or_owner",
  "volume_extension:extended_snapshot_attributes": "",
  "volume_extension:volume_image_metadata": "",
  "volume_extension:quotas:show": "",
  "volume_extension:quotas:update": "rule:admin_api",
  "volume_extension:quota_classes": "",
  "volume_extension:volume_admin_actions:reset_status": "rule:admin_api",
  "volume_extension:snapshot_admin_actions:reset_status": "rule:admin_api",
  "volume_extension:backup_admin_actions:reset_status": "rule:admin_api",
  "volume_extension:volume_admin_actions:force_delete": "rule:admin_api",
  "volume_extension:volume_admin_actions:force_detach": "rule:admin_api",
  "volume_extension:snapshot_admin_actions:force_delete": "rule:admin_api",
  "volume_extension:volume_admin_actions:migrate_volume": "rule:admin_api",
  "volume_extension:volume_admin_actions:migrate_volume_completion": "rule:admin_api",
  "volume_extension:volume_host_attribute": "rule:admin_api",
  "volume_extension:volume_tenant_attribute": "rule:admin_or_owner",
  "volume_extension:volume_mig_status_attribute": "rule:admin_api",
  "volume_extension:hosts": "rule:admin_api",
  "volume_extension:services": "rule:admin_api",
  "volume_extension:volume_manage": "rule:admin_api",
  "volume_extension:volume_unmanage": "rule:admin_api",
  "volume:services": "rule:admin_api",
  "volume:create_transfer": "",
  "volume:accept_transfer": "",
  "volume:delete_transfer": "",
  "volume:get_all_transfers": "",
  "volume_extension:replication:promote": "rule:admin_api",
  "volume_extension:replication:reenable": "rule:admin_api",
  "backup:create": "",
  "backup:delete": "",
  "backup:get": "",
  "backup:get_all": "",
  "backup:restore": "",
  "backup:backup-import": "rule:admin_api",
  "backup:backup-export": "rule:admin_api",
  "snapshot_extension:snapshot_actions:update_snapshot_status": "",
  "consistencygroup:create": "group:nobody",
  "consistencygroup:delete": "group:nobody",
  "consistencygroup:update": "group:nobody",
  "consistencygroup:get": "group:nobody",
  "consistencygroup:get_all": "group:nobody",
  "consistencygroup:create_cgsnapshot": "group:nobody",
}
```

```
"consistencygroup:delete_cgsnapshot": "group:nobody",
"consistencygroup:get_cgsnapshot": "group:nobody",
"consistencygroup:get_all_cgsnapshots": "group:nobody",
"scheduler_extension:scheduler_stats:get_pools" : "rule:admin_api"
}
```

3. To view the current roles available, type the following:

```
[root@controller ~(keystone_admin)]# keystone role-list
```

There are two roles you should know about:

The `_member_` role, when assigned to a user, allows a user to manage all of the resources within their project (instances, volumes, and so on). If no role is defined, then this default role will be applied.

The `admin` role, when assigned to a user, allows the user total control over the entire OpenStack platform.

id	name
73ac60d79e5d48049781908adbfe45b	ResellerAdmin
f2db5327b06140d8aeb3beb9ab09ba73	SwiftOperator
9fe2ff9ee4384b1894a90878d3e92bab	_member_
6d840affa1c9401d9e883c9c8496c750	admin
10e8b93ef4b1494ebbc82d1195210252	ultra_admin

4. Create a new role called `ultra_admin` by typing the following:

```
[root@controller ~(keystone_admin)]# keystone role-create --name ultra_admin
```

Roles are applied when the user is created, and may also be applied after the user is created.

Users may occupy more than one role

Property	Value
id	a9e79b4e382e4eeca7a13adeadf0a9f2
name	ultra_admin

5. Let's give alice the role `ultra_admin`, so start by obtaining `acme_inc`'s ID

```
[root@controller ~(keystone_admin)]# keystone tenant-list
```

id	name	enabled
57d57fce06ea44d79d2abd6307216600	acme_inc	True
300b2cc45c3846939e589310ae714e46	admin	True
a9c27d47441e432c8529f8f30ea3261e	demo	True
85a11a1515f94848be7aa5b56adec8bf	services	True
c4a7843ebc974911aa039fe6191866dc	vault_tek	True

<- Highlight YOUR id (**NOT** this one) for acme_inc

6. Now assign alice the role of ultra-admin of tenant `acme_inc` [root@controller ~(keystone_admin)]# keystone user-role-add --user ameliorene --role ultra_admin --tenant <INSERT acme_inc id here>

There will be no special reward if you get this right, just something like this:
/usr/lib/python2.7/site-packages/keystoneclient/shell.py:65: DeprecationWarning:
The keystone CLI is deprecated in favor of python-openstackclient. For a Python
library, continue using python-keystoneclient.

7. Confirm that amelirene is now a 'ultra_admin'

```
[root@controller ~(keystone_admin)]# keystone user-role-list --user amelirene  
--tenant <right_click_to_paste_the_tenant_ID_you_highlighted>
```

id	name	user_id	tenant_id
9fe2ff9ee4384b1894a90878d3e92bab	_member_	ba62896bc6c0474bb552946e097ed71f	67f9b771e7f24bcc9109a01005f96c3a
10e8b93ef4b1494ebbc82d1195210252	ultra_admin	ba62896bc6c0474bb552946e097ed71f	67f9b771e7f24bcc9109a01005f96c3a

8. Of course, amelirene being an ultra_admin doesn't really do anything, until we edit the policy.json file so that the role ultra_admin performs the way we want it to, but hopefully roles are clear.
- Now is a great time to ask the instructor to explain roles if you are still a bit unclear.
 - Manipulating these policy.json files is tricky, but the following link is a great resource with a cool working example if you're :
<http://www.florentflament.com/blog/customizing-openstack-rbac-policies.html>

3. OPTIONAL - Creating a Project at the CLI (using python-openstackclient)

Earlier in this lab, there was mention of the python-openstackclient. The python-openstackclient is trying to unite all of the OpenStack services on a single platform with a common set of commands. Some admins are using it, and some are not. At the end of the day, there is nothing 'new' you can do within this platform.

The following is a list of the steps you would take to utilize the python-openstackclient in order to create the project 'the_shire' and the user 'gandalf'. You already did this so there is no need to do it a second time. However, if you are a rocket scientist and finished this lab early, feel free to stare & compare and play around with the python-openstackclient environment.

- SSH into your controller as root (if you are not already)
- If you are not sourced as admin already, issue the following command:

```
[root@controller]# source keystone_admin
```

3. Start the python-openstackclient

```
[root@controller ~(keystone_admin)]# openstack
```

Within the python-openstackclient, you'll be issuing commands as admin since the admin permissions were set when you executed openstack

4. Get a sense of the commands you can issue:

```
(openstack) help
compute service list  object save          token issue
|
compute service set   object show          token revoke
|
console log show      project create       usage list
|
console url show      project delete       usage show
|
container create      project list         user create
|
container delete      project set          user delete
|
container list        project show         user list
|
container save        project usage list   user role list
|
container show        quota set           user set
|
ec2 credentials create quota show          user show
|
ec2 credentials delete role add            volume create
|
ec2 credentials list  role create        volume delete
|
ec2 credentials show  role delete        volume list
|
endpoint create       role list          volume set
|
endpoint delete       role remove        volume show
|
endpoint list         role show          volume type create
|
endpoint show         security group create volume type delete
|
extension list        security group delete volume type list
|
flavor create         security group list  volume type set
|
flavor delete         security group rule create volume type unset
|
flavor list           security group rule delete volume unset
|
flavor set            security group rule list
|
flavor show           security group set
|
flavor unset          security group show
|
help                  server add security group
|
host list             server add volume
|
host show             server create
|
hypervisor list       server delete
|
hypervisor show       server image create
|
hypervisor stats show server list
|
image create          server lock
|
image delete          server migrate
|
image list            server pause
|
image save            server reboot
|
image set             server rebuild
|
```

```
image show          server remove security group
|
ip fixed add        server remove volume
```

5. Issue the following commands to create the project, and then create the user gandalf with the password noneshallpass

```
(openstack) project create the_shire
(openstack) user create gandalf --project the_shire --password noneshallpass -
-email gandalf@theshire.example
(openstack) exit
```
6. Log in to the Horizon interface as gandalf // noneshallpass
 - If you have forgotten how to get to Horizon, simply navigate to your FQDN in a web browser
 - The login should work! If it did, good job! If other students are still working, feel free to play around in the python-openstackclient a bit more.

04 OpenStack Identity service (Keystone).

Having appropriate credentials is required if you wish to use any CLI tools to query on the OpenStack services.

To use an OpenStack cloud, you need to authenticate against the Identity service Keystone (which returns a Token and the Service Catalog). To request an authentication token, you must supply a payload of credentials in the authentication request. Credentials are usually a combination of your user name and password, and optionally, the name or ID of the tenant in which your cloud runs. When you send API requests, you include the token in an HTTP header called the "X-Auth-Token" header.

If you access multiple OpenStack services, you must get a token for each service. A token is valid for a limited time before it expires. A token can also become invalid for other reasons. For example, if the roles for a user change, existing tokens for that user are invalid. Credentials allow OpenStack services to determine where your service endpoints and your authentication information are located.

1. Understanding permissions

1. SSH to your controller as root, you might use:
 - While holding the control key, click: (<https://amel.com/sshproxy>)
2. Once logged into the controller, type the following command (it won't work!):

```
[root@controller ~]# nova flavor-list
```

```
ERROR (CommandError): You must provide a username or user id via --os-username, --os-user-id, env[OS_USERNAME] or env[OS_USER_ID]
```

This command results in an error because Nova doesn't know who you are.

- Does nova know that you have permissions to issue flavor-list?
- Consider how Horizon works. Does ALWAYS Horizon know who you are?
- HINT: To use Horizon, you have to login as a user (admin, amelirene, bobbarker, etc.)

3. Let's try another command that won't work!

```
[root@controller ~]# keystone tenant-list
```

Another error!? That's alright... Keystone doesn't know who you are. Do you have permissions to view the current tenants?

4. We focusing on the permissions required to issue a command successfully without an error. We only chose flavor-list as a example.

```
[root@controller ~]# nova --os-auth-url=http://192.168.0.10:5000/v2.0 --os-tenant-name=admin --os-user-name=admin --os-password=amel flavor-list
```

ID	Name	Memory_MB	Disk	Ephemeral	Swap	VCPUs	RXTX_Factor	Is_Public
1	m1.tiny	512	1	0		1	1.0	True
2	m1.small	2048	20	0		1	1.0	True
3	m1.medium	4096	40	0		2	1.0	True
4	m1.large	8192	80	0		4	1.0	True
5	m1.xlarge	16384	160	0		8	1.0	True

Now the flavors are displayed. This is because you issued a command that included values of environmental variables that disclose your permissions. Nova can use this information to make an API call to the Keystone service and determine if you (admin) have sufficient permissions to execute the command (flavor-list)

5. So it worked, but we had to include a long list of credentials. If you'd like, trying using just Nova flavor-list again (without the --flags).

```
[root@controller ~]# nova flavor-list
```

Once again, it didn't work. Clearly the credentials are required every time you issue commands to manipulate your OpenStack cloud.

6. Use the nova help commands to better understand the credentials:

"nova -h" will display help for the nova command "grep" means "display only these lines that contain the following search string" Therefore, "nova -h | grep os-auth-url" means display help, and only display the lines that contain os-auth-url


```
[root@controller ~]# nova -h
[root@controller ~]# nova -h | grep os-auth-url
[root@controller ~]# nova -h | grep os-tenant-name
[root@controller ~]# nova -h | grep os-user-name
[root@controller ~]# nova -h | grep os-password
```

The purpose of issuing these commands is so that you know what the flags are that you are setting

- Let's go a step further and issue the same command, but this time include a debug flag and see if we can deep dive into what is going on!

```
[root@controller ~]#
nova --os-auth-url=http://192.168.0.10:5000/v2.0 --os-tenant-name=admin --os-user-name=admin --os-password=amel --debug flavor-list
```

Debug attempts to display the processes that are going on in the background to make this possible. We'll spend some more time checking out some of the returned information in the next lab, for now, we just wanted you to be aware that it was possible.

```
DEBUG (v2:76) Making authentication request to http://192.168.0.10:5000/v2.0/tokens
|
INFO (connectionpool:238) Resetting dropped connection: 192.168.0.10
|
DEBUG (connectionpool:383) "POST /v2.0/tokens HTTP/1.1" 200 3974
|
DEBUG (iso8601:184) Parsed 2015-10-21T04:27:26Z into {'tz_sign': None, 'second_fraction': None, 'hour':
u'04', 'daydash': u'21', 'tz_hour': None, 'month': Non|
e, 'timezone': u'Z', 'second': u'26', 'tz_minute': None, 'year': u'2015', 'separator': u'T', 'monthdash':
u'10', 'day': None, 'minute': u'27'} with default ti|
mezone <iso8601.iso8601.Utc object at 0x22229d0>
|
DEBUG (iso8601:140) Got u'2015' for 'year' with default None
|
DEBUG (iso8601:140) Got u'10' for 'monthdash' with default 1
|
DEBUG (iso8601:140) Got 10 for 'month' with default 10
|
DEBUG (iso8601:140) Got u'21' for 'daydash' with default 1
|
DEBUG (iso8601:140) Got 21 for 'day' with default 21
|
DEBUG (iso8601:140) Got u'04' for 'hour' with default None
|
DEBUG (iso8601:140) Got u'27' for 'minute' with default None
|
DEBUG (iso8601:140) Got u'26' for 'second' with default None
|
DEBUG (session:195) REQ: curl -g -i -X GET
http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46/flavors/detail -H "User-Agent: python-
novaclient" -H "|
Accept: application/json" -H "X-Auth-Token: {SHA1}914b01c69b7491be6cd35d89a3f7c55e90603dfe"
|
INFO (connectionpool:203) Starting new HTTP connection (1): 192.168.0.10
|
DEBUG (connectionpool:383) "GET /v2/300b2cc45c3846939e589310ae714e46/flavors/detail HTTP/1.1" 200 2089
|
DEBUG (session:224) RESP: [200] date: Wed, 21 Oct 2015 03:27:27 GMT connection: keep-alive content-type:
application/json content-length: 2089 x-compute-reqe|
st-id: req-980927ce-3964-441f-bb9a-58437a05b130
|
RESP BODY: {"flavors": [{"name": "m1.tiny", "links": [{"href":
"http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46/flavors/1", "rel": "self"}, {"href|
f": "http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46/flavors/1", "rel": "bookmark"}]}, "ram": 512,
"OS-FLV-DISABLED:disabled": false, "vcpus": 1, "sw|
ap": "", "os-flavor-access:is_public": true, "rxtx_factor": 1.0, "OS-FLV-EXT-DATA:ephemeral": 0, "disk": 1,
"id": "1"}, {"name": "m1.small", "links": [{"href"|
": "http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46/flavors/2", "rel": "self"}, {"href":
"http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e4|
```

```
6/flavors/2", "rel": "bookmark"}], "ram": 2048, "OS-FLV-DISABLED:disabled": false, "vcpus": 1, "swap": "",
"os-flavor-access:is_public": true, "rxtx_factor": 1.0, "OS-FLV-EXT-DATA:ephemeral": 0, "disk": 20, "id": "2"}, {"name": "m1.medium", "links": [{"href":
"http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46/flavors/3", "rel": "self"}], {"href":
"http://192.168.0.10:8774/300b2cc45c3846939e589310ae714e46/flavors/3", "rel": "bookmark"}], "ram": 4096, "OS-
FLV-DI
SABLED:disabled": false, "vcpus": 2, "swap": "", "os-flavor-access:is_public": true, "rxtx_factor": 1.0, "OS-
FLV-EXT-DATA:ephemeral": 0, "disk": 40, "id": "3"}, {"name": "m1.large", "links": [{"href":
"http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46/flavors/4", "rel": "self"}], {"href":
"http://192.168.0.10:8774/300b2cc45c3846939e589310ae714e46/flavors/4", "rel": "bookmark"}], "ram": 8192, "OS-FLV-
DISABLED:disabled": false, "vcpus": 4, "swap": "", "os-flavor-
-access:is_public": true, "rxtx_factor": 1.0, "OS-FLV-EXT-DATA:ephemeral": 0, "disk": 80, "id": "4"}, {"name": "m1.xlarge", "links": [{"href": "http://192.168.
0.10:8774/v2/300b2cc45c3846939e589310ae714e46/flavors/5", "rel": "self"}], {"href":
"http://192.168.0.10:8774/300b2cc45c3846939e589310ae714e46/flavors/5", "rel": "bookmark"}], "ram": 16384, "OS-FLV-DISABLED:disabled": false, "vcpus": 8, "swap": "", "os-flavor-
access:is_public": true, "rxtx_factor": 1.0, "OS-FLV-EXT-
DATA:ephemeral": 0, "disk": 160, "id": "5"}]}
```

ID	Name	Memory_MB	Disk	Ephemeral	Swap	VCPUs	RXTX_Factor	Is_Public
1	m1.tiny	512	1	0		1	1.0	True
2	m1.small	2048	20	0		1	1.0	True
3	m1.medium	4096	40	0		2	1.0	True
4	m1.large	8192	80	0		4	1.0	True
5	m1.xlarge	16384	160	0		8	1.0	True

2. Getting User Credentials (source keystone_admin)

So what we've all figured out by this point, is that setting flag on every CLI command involves far too much typing. Let's make our commands easier to issue by setting some environmental variables for our current bash session. NOTE: If environmental vs shell variables, and parent vs child sessions are new terms for you, that is OK. There is a small section at the conclusion of this lab that will make it more clear.

1. Display the contents of the keystone_admin file

```
[root@controller ~]# cat keystone_admin
```

Comments have been added to the output, your output will lack the comments (lines that begin with #).

```
# This is the user name
export OS_USERNAME=admin
# This is the tenant, or project
export OS_TENANT_NAME=admin
# Password for the above user
export OS_PASSWORD=amel
# The Identity API served through Keystone
export OS_AUTH_URL=http://192.168.0.10:5000/v2.0/
# The OpenStack Region
export OS_REGION_NAME=RegionOne
# Appends the CLI prompt with keystone_admin
export PS1='[\u@\h \W(keystone_admin)]\$ '
```

The export commands within this file will mark each VAR for automatic export to the environment of subsequently executed commands (i.e. make the local shell

variable VAR global). Essentially, we can set all of our permissions in this file, source this file, and then not have to ever type our permissions again as they'll always be available to the bash environment.

NOTE: The name of this file may change slightly depending on your environment. RDO deployments use the formatting structure `keystonerc_$USER`

For more info on regions: http://docs.openstack.org/openstack-ops/content/scaling.html#segregate_cloud

2. Display a list of the environmental variables currently set

```
[root@controller ~]# printenv
```

Look over the displayed entries. Do you see any mention of the aforementioned credentials, such as `OS_TENANT_NAME` or `OS_USERNAME`? (Feel free to use `grep` if you'd like.)

```
XDG_SESSION_ID=16
HOSTNAME=controller.localdomain
SELINUX_ROLE_REQUESTED=
TERM=screen
SHELL=/bin/bash
HISTSIZE=1000
SSH_CLIENT=71.251.147.235 52298 22
SELINUX_USE_CURRENT_RANGE=
SSH_TTY=/dev/pts/0
USER=root
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:
mi=01;05;37;41:su=37;41:sg=30;43:ca=30;41:tw=30;42:ow=34
;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lz4=01
;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*.txz=01;31:*.t
zo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.Z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lz
o=01;31:*.xz=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.
tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01;31:*.ear=01;31:*.sar=01;31:*.rar=01
;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7
z=01;31:*.rz=01;31:*.cab=01;31:*.jpg=01;35:*.jpeg=01;35:*.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;3
5:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=
01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;3
5:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.og
m=01;35:*.mp4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;3
5:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi=01;35:*.fli=
01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.
axv=01;35:*.anx=01;35:*.ogv=01;35:*.ogx=01;35:*.aac=01;3
6:*.au=01;36:*.flac=01;36:*.mid=01;36:*.midi=01;36:*.mka=01;36:*.mp3=01;36:*.mpc=01;36:*.ogg=01;36:*.r
a=01;36:*.wav=01;36:*.axa=01;36:*.oga=01;36:*.spx=01;36:
*.xspf=01;36:
MAIL=/var/spool/mail/root
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/root/bin
PWD=/root
LANG=en_US.UTF-8
SELINUX_LEVEL_REQUESTED=
HISTCONTROL=ignoredups
SHLVL=1
HOME=/root
LOGNAME=root
SSH_CONNECTION=71.251.147.235 52298 192.168.0.10 22
LESSOPEN=||/usr/bin/lesspipe.sh %s
XDG_RUNTIME_DIR=/run/user/0
_=/usr/bin/printenv
```

3. Set the environmental variables found in **keystonerc_admin**

```
[root@controller ~]# source keystonerc_admin
```

This is a bash shell built-in command that executes the content of the file passed as a argument in the current shell. It is synonymous with ' . ' (period). After executing the above command, the CLI line will change. This is just a reminder that you are now passing commands to the OpenStack services as some user (in this case admin).

Notice that your CLI prompt has changed to indicate that you are now 'sourced' with admin permissions

```
[root@controller ~(keystone_admin)]#
```

4. Once again, display a list of the environmental variables currently set.

```
[root@controller ~(keystone_admin)]# printenv
```

Look over the displayed entries. Now you will see a listing for the variables contained within keystone_admin, such as OS_TENANT_NAME and OS_USERNAME. Just for your understanding, these entries are only set for this current bash session. If you type exit, then reconnect as root, these variables will unset themselves. If you want to remove them manually, use the unset command.

```
XDG_SESSION_ID=16
HOSTNAME=controller.localdomain
SELINUX_ROLE_REQUESTED=
TERM=screen
SHELL=/bin/bash
HISTSIZE=1000
SSH_CLIENT=71.251.147.235 52298 22
OS_REGION_NAME=RegionOne <-----
SELINUX_USE_CURRENT_RANGE=
SSH_TTY=/dev/pts/0
USER=root
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:
mi=01;05;37;41:su=37;41:sg=30;43:ca=30;41:tw=30;42:ow=34
;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lzh=01
;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*.txz=01;31:*.t
zo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.Z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lz
o=01;31:*.xz=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.
tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01;31:*.ear=01;31:*.sar=01;31:*.rar=01
;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7
z=01;31:*.rz=01;31:*.cab=01;31:*.jpg=01;35:*.jpeg=01;35:*.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;3
5:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=
01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;3
5:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.og
m=01;35:*.mp4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;3
5:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi=01;35:*.fli=
01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.
axv=01;35:*.anx=01;35:*.ogv=01;35:*.ogx=01;35:*.aac=01;3
6:*.au=01;36:*.flac=01;36:*.mid=01;36:*.midi=01;36:*.mka=01;36:*.mp3=01;36:*.mpc=01;36:*.ogg=01;36:*.r
a=01;36:*.wav=01;36:*.axa=01;36:*.oga=01;36:*.spx=01;36:
*.xspf=01;36:
MAIL=/var/spool/mail/root
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/root/bin
PWD=/root
OS_PASSWORD=amel <-----
LANG=en_US.UTF-8
PS1=[\u@\h \W(keystone_admin)]\$
SELINUX_LEVEL_REQUESTED=
OS_AUTH_URL=http://192.168.0.10:5000/v2.0/
HISTCONTROL=ignoredups
OS_USERNAME=admin <-----
SHLVL=1
HOME=/root
OS_TENANT_NAME=admin <-----
LOGNAME=root
SSH_CONNECTION=71.251.147.235 52298 192.168.0.10 22
LESSOPEN=||/usr/bin/lesspipe.sh %s
```

```
XDG_RUNTIME_DIR=/run/user/0
_=/usr/bin/printenv
```

5. Flex your new admin environmental-variable-given powers

```
[root@controller ~(keystone_admin)]# nova flavor-list
```

This time it should just work without having to type the extra permissions. Try executing a few more commands to the other OpenStack services as well.

ID	Name	Memory_MB	Disk	Ephemeral	Swap	VCPUs	RXTX_Factor	Is_Public
1	m1.tiny	512	1	0		1	1.0	True
2	m1.small	2048	20	0		1	1.0	True
3	m1.medium	4096	40	0		2	1.0	True
4	m1.large	8192	80	0		4	1.0	True
5	m1.xlarge	16384	160	0		8	1.0	True

6. Issue a few more commands.

```
[root@controller ~(keystone_admin)]# keystone tenant-create --name temp_tenant
[root@controller ~(keystone_admin)]# keystone user-list
```

7. This should launch the **python-openstackclient**.

OpenStack is attempting to 'unite' all of the OpenStack commands inside of a common python 'wrapper', or a platform that allows you to issue a common set of commands for every OpenStack service. This is why some CLI commands are receiving depreciation warnings. Some people are using it, some are not, but we think it is important to know it exists.

```
[root@controller ~(keystone_admin)]# openstack
(openstack) help
(openstack) exit
```

The permissions still apply! If you have sourced the keystone_admin file, then launch the **python-openstackclient**, then you are issuing commands to OpenStack as admin.

3. Creating a keystone_admin_chestercopperpot file for Chester Copperpot of Vault Tek

Remember our buddy Chester Copperpot of Vault Tek? You should see him when you issued the previous command keystone user-list.

Let's make it easy for us to execute OpenStack commands as Chester Copperpot who is part of the project vault_tek at the CLI.

1. Set the environmental variables found in keystone_admin if this is not already done

```
[root@controller ~(keystone_admin)]# source keystone_admin
```

2. List the keystone users

```
[root@controller ~(keystone_admin)]# keystone user-list
```

3. Look at the user chestercopperpot, and note his tenantID

```
[root@controller ~(keystone_admin)]# keystone tenant-list
```

4. Search for chestercopperpot's tenantID value and note the name of the tenant he belongs to (should be vault_tek)

5. Make a copy of the keystone_admin file, then open it in nano so we can edit it.

```
(keystone_admin)# cp keystone_admin keystone_admin_chestercopperpot  
(keystone_admin)# nano keystone_admin_chestercopperpot
```

6. Edit **keystone_admin_chestercopperpot** so it looks like the following:

```
export OS_USERNAME=chestercopperpot  
export OS_TENANT_NAME=vault_tek  
export OS_PASSWORD=fa5tpa55w0rd  
export OS_AUTH_URL=http://192.168.0.10:5000/v2.0/  
export OS_REGION_NAME=RegionOne  
export PS1='\u@\h \W(keystone_admin_chestercopperpot)]\$ '
```

- When you're done editing changes, do the following:
- press CTRL + O (to write out aka save)
- press ENTER (to confirm save)
- press CTRL + X (to exit)

7. Display the contents of **keystone_admin_chestercopperpot** and verify that they are correct

```
[root@controller ~(keystone_admin)]# cat keystone_admin_chestercopperpot  
export OS_USERNAME=chestercopperpot  
export OS_TENANT_NAME=vault_tek  
export OS_PASSWORD=fa5tpa55w0rd  
export OS_AUTH_URL=http://192.168.0.10:5000/v2.0/  
export OS_REGION_NAME=RegionOne  
export PS1='\u@\h \W(keystone_admin_chestercopperpot)]\$ '
```

8. Set global variables in the current shell to correspond with the user chestercopperpot

```
[root@controller ~(keystone_admin)]# source keystone_admin_chestercopperpot
```

9. Note the CLI changed. Run commands and see how the results differ from chester's perspective.

```
[root@controller ~(keystone_admin_chestercopperpot)]# nova flavor-list
```

OK, this worked!

```
[root@controller ~(keystone_chestercopperpot)# keystone tenant-list
```

This should result in an HTTP 403 failure, as the user chesetercopperpot does not have sufficient permissions to issue this keystone command.

10. Let's repeat the process for the user amelirene

```
[root@controller ~(keystone_chestercopperpot)# cp kestonerc_admin  
kestonerc_amelirene
```

```
[root@controller ~(keystone_chestercopperpot)# nano kestonerc_amelirene
```

11. Edit **kestonerc_amelirene** so it looks like the following (or copy and paste is fine):

```
export OS_USERNAME=amelirene  
export OS_TENANT_NAME=acme_inc  
export OS_PASSWORD=fa5tpa55w0rd  
export OS_AUTH_URL=http://192.168.0.10:5000/v2.0/  
export OS_REGION_NAME=RegionOne  
export PS1='[\u@\h \w(keystone_amelirene)]\$ '
```

- When you're done editing changes, do the following:
- press CTRL + O (to write out aka save)
- press ENTER (to confirm save)
- press CTRL + X (to exit)

12. Display the contents of **kestonerc_amelirene** and verify that they are correct

```
[root@controller ~(keystone_admin)# cat kestonerc_amelirene
```

13. Finally, lets repeat the process again for user bobbarker

```
[root@controller ~(keystone_chestercopperpot)# cp kestonerc_admin  
kestonerc_bobbarker
```

```
[root@controller ~(keystone_chestercopperpot)# nano kestonerc_bobbarker
```

14. Edit **kestonerc_bobbarker** so it looks like the following (copy and paste is fine):

```
export OS_USERNAME=bobbarker  
export OS_TENANT_NAME=acme_inc  
export OS_PASSWORD=fa5tpa55w0rd  
export OS_AUTH_URL=http://192.168.0.10:5000/v2.0/  
export OS_REGION_NAME=RegionOne  
export PS1='[\u@\h \w(keystone_bobbarker)]\$ '
```

- When you're done editing changes, do the following:
- press CTRL + O (to write out aka save)
- press ENTER (to confirm save)

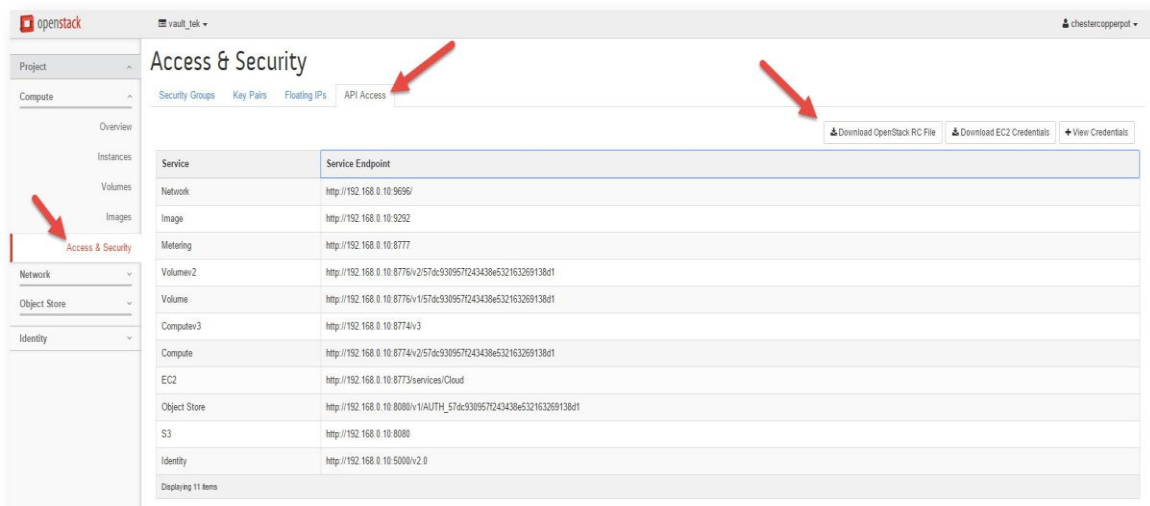
- press CTRL + X (to exit)

15. Display the contents of **keystonerc_bobbarker** and verify that they are correct

```
[root@controller ~(keystone_chestercopperpot)# cat keystonerc_bobbarker
```

4. Using the OpenStack Horizon Dashboard to assist in creating a keystonerc file for new user Doris Day of Vault Tek

1. Log in to the OpenStack Horizon Dashboard as chestercopperpot // fa5tpa55w0rd
2. Click on **Project > Compute > Access & Security**
3. Click on the tab **API Access**
4. In the upper right corner, click on the button **Download OpenStack RC File**



5. This file will download locally. Open it in your favorite text editor. The file you open will look similar to the one shown below. If your "favorite" text editor is Microsoft Notepad or Microsoft Word, then it is recommended you take 2 minutes and go [download Notepad++](#)

```
bash
#!/bin/bash
# To use an OpenStack cloud you need to authenticate against the Identity
# service named keystone, which returns a **Token** and **Service Catalog**.
# The catalog contains the endpoints for all services the user/tenant has
# access to - such as Compute, Image Service, Identity, Object Storage, Block
# Storage, and Networking (code-named nova, glance, keystone, swift,
# cinder, and neutron).
#
#
#
# *NOTE*: Using the 2.0 *Identity API* does not necessarily mean any other
```



```

# OpenStack API is version 2.0. For example, your cloud provider may implement
# Image API v1.1, Block Storage API v2, and Compute API v2.0. OS_AUTH_URL is
# only for the Identity API served through keystone.
export OS_AUTH_URL=http://192.168.0.10:5000/v2.0
#
#
# With the addition of Keystone we have standardized on the term **tenant**
# as the entity that owns the resources.
export OS_TENANT_ID=57dc930957f243438e532163269138d1
export OS_TENANT_NAME="vault_tek"
export OS_PROJECT_NAME="vault_tek"
#
#
# In addition to the owning entity (tenant), OpenStack stores the entity
# performing the action as the **user**.
export OS_USERNAME="chestercopperpot"
#
# # # # LOOK AT THIS # # # #
# With Keystone you pass the keystone password.
echo "Please enter your OpenStack Password: "
read -sr OS_PASSWORD_INPUT
export OS_PASSWORD=$OS_PASSWORD_INPUT
# # # # END LOOK AT THIS # # # #
#
# If your configuration has multiple regions, we set that information here.
# OS_REGION_NAME is optional and only valid in certain environments.
export OS_REGION_NAME="RegionOne"
#
#
# Don't leave a blank variable, unset it if it was empty
if [ -z "$OS_REGION_NAME" ]; then unset OS_REGION_NAME; fi

```

This file is a bit fancier than the previous one we studied, but it is well commented. There are two key differences between this file and ours:

- i. The first is that this file will require the user to enter their password instead of storing it locally as plain-text. In production, this is desirable, as asking users to enter their password will prevent admins from having to manage security and permissions on user files because they contain plain-text passwords.
- ii. The second difference is that this file also sets some additional environmental variables, which could be problematic, as our other `keystonerc_*` files do NOT set these variables. If we source back to another user without first unsetting these variables, we might get ourselves into trouble. So instead of using this file, let's just borrow the piece of code that asks the user for the password. It is shown below for reference:

```

# With Keystone you pass the keystone password.
echo "Please enter your OpenStack Password: "
read -sr OS_PASSWORD_INPUT
export OS_PASSWORD=$OS_PASSWORD_INPUT

```

6. We're done in Horizon for now. Return to your SSH session with the controller and source `keystone_admin`

```
[root@controller ~]# source keystone_admin
```

- Let's create a new user (Doris Day) within the vault_tek vault_tek project (tenant) and then create a source file that prompts for a password.

```
[root@controller ~(keystone_admin)]# keystone user-create --name dorisday --tenant vault_tek --pass fa5tpa55w0rd --email dorisday@vault_tek.example
```

Property	Value
email	dorisday@vault_tek.example
enabled	True
id	8086288736dc4147bf38ffaf07acd9b5
name	dorisday
tenantId	8f6d4710d774466f96858af6fa69aaa3
username	dorisday

- Create a file we can source for Doris Day of Vault Tek.

```
[root@controller ~(keystone_admin)]# nano keystone_admin_dorisday
```

- Edit keystone_admin_dorisday so it looks like the following:

```
export OS_USERNAME=dorisday
export OS_TENANT_NAME=vault_tek
echo "Please enter your OpenStack Password: "
read -sr OS_PASSWORD_INPUT
export OS_PASSWORD=$OS_PASSWORD_INPUT
export OS_AUTH_URL=http://192.168.0.10:5000/v2.0/
export OS_REGION_NAME=RegionOne
export PS1='[\u@\h \W(keystone_admin_dorisday)]\$ '
```

- When you're done editing changes, do the following:
- press CTRL + O (to write out aka save)
- press ENTER (to confirm save)
- press CTRL + X (to exit)

- Confirm that the file was saved correctly, by displaying the output with cat, and then source keystone_admin_dorisday

```
[root@controller ~(keystone_admin)]# source keystone_admin_dorisday
Please enter your OpenStack Password: fa5tpa55w0rd
```

- QUESTION: Is this mechanism of prompting for a password more secure? Why or why not?

- Make sure the file works. Test by issuing a CLI command requiring authentication.

```
[root@controller ~(keystone_admin_dorisday)]# nova flavor-list
If output like this is displayed, then it worked!
```

ID	Name	Memory_MB	Disk	Ephemeral	Swap	VCPUs	RXTX_Factor	Is_Public
1	m1.tiny	512	1	0		1	1.0	True
2	m1.small	2048	20	0		1	1.0	True
3	m1.medium	4096	40	0		2	1.0	True
4	m1.large	8192	80	0		4	1.0	True
5	m1.xlarge	16384	160	0		8	1.0	True

Great job! You are all done with this lab. If you are done early, go onto the next section. It will teach you a little bit more about what is going on when you run the source command.

5. OPTIONAL: Linux variables

In OpenStack, we learned that we could type out the `--os-tenant-name`, `--os-user-name`, `--os-password`, and `--os-auth-url` along with every command, or we could set these values so they persist. If you are not a master of the CLI, you might be curious about how this worked! Understanding how it works might also help you in troubleshooting. After all, what if you ran a source file (set environmental variables), and wish you could un-run that source file (unset environmental variables).

Variables can be of two basic types: shell variables, or environmental variables. Shell variables are only available within the current session, whereas environmental variables are available to the current session, as well as any spawned sessions. What is a session? You're in one right now. The Bash shell that you are sitting at is a 'session'. If you were to type `bash`, you'd launch another session that would be considered a 'child' session, where your current session would be the 'parent'.

If a child session is spawned by a parent session, environmental variables will be made available to the child sessions, whereas a shell variable will not. To better understand the difference, perform this quick procedure; define a shell variable, and then an environmental variable within parent and child bash shells.

1. Create a shell variable within the current session. Convention says it should be all caps. Because the string we've defined includes a bang, we are using single quotes to surround 'Hello OpenStack!'.

```
[root@controller ~]# LEARNING_VAR='Hello OpenStack!'
```

2. Great! We can confirm that this is a shell variable that will not be passed down to child processes by grepping within output from `set`

```
[root@controller ~]# set | grep LEARNING_VAR
```

3. `LEARNING_VAR='Hello OpenStack!'` should be displayed. Now confirm it is not an environmental variable by performing the same procedure with `printenv`.

```
[root@controller ~]# printenv | grep LEARNING_VAR
```

No output should be returned, as printenv will only print environmental variables.

- Let's take a moment to demonstrate how to access any shell or environmental variable. When the shell encounters a \$ sign, it takes it to mean that it should substitute the value of the variable when it comes across it. Type the following command.

```
[root@controller ~]# echo $LEARNING_VAR
```

The string Hello OpenStack! should be displayed.

- If you type 'bash', you'll spawn a child shell. Think of this as accessing a 'shell within a shell'. Within this child session our variable, \$LEARNING_VAR, should not be available.

```
[root@controller ~]# bash
```

```
[root@controller ~]# echo $LEARNING_VAR
```

No output should be displayed. Hopefully, why no output was display is clear. This is a child session, and was not passed the previous shell variables. Remember, shell variables are only good for the current shell. Type the following to test this assertion.

- Type exit to close the child session and return to the original session.

```
[root@controller ~]# exit
```

```
[root@controller ~]# echo $LEARNING_VAR
```

Once again, the string Hello OpenStack! should be displayed, as we have returned to the parent shell (the shell we started in).

- Now let's make LEARNING_VAR an environmental variable by using the export command.

```
[root@controller ~]# export LEARNING_VAR='OpenStack Hello!'
```

```
[root@controller ~]# printenv | grep LEARNING_VAR
```

- Now, run the same experiment with a minor change, export LEARNING_VAR='Hello OpenStack!' then **Hello Openstack** rather than *Openstack Hello* should be displayed. Once again, spawn a child shell.

```
[root@controller ~]# bash
```

```
[root@controller ~]# echo $LEARNING_VAR
```

Hello OpenStack! should be displayed.

- Confirm that it is still set as an environmental variable.

```
[root@controller ~]# printenv | grep LEARNING_VAR
```

- Now let's set an environmental variable from within our current child shell.

```
[root@controller ~]# export NEW_LEARNING_VAR='Bonjour OpenStack!'
```

11. Confirm that it was indeed set as an environmental variable. Then exit the child session and return to the parent shell.

```
[root@controller ~]# printenv | grep NEW_LEARNING_VAR  
[root@controller ~]# exit
```

12. Now see if the environmental variable NEW_LEARNING_VAR is still available.

```
[root@controller ~]# printenv | grep NEW_LEARNING_VAR
```

No output is returned. This is because NEW_LEARNING_VAR was an environmental variable set in the child shell. Emphasis on the word 'was', because when we exited our child shell, that environmental variable was destroyed.

13. The original variable, LEARNING_VAR can be changed back into a shell variable (demoted), by using export and the -n flag.

```
[root@controller ~]# export -n LEARNING_VAR
```

14. Test that it is no longer a global variable, but still a shell variable

```
[root@controller ~]# printenv | grep LEARNING_VAR
```

No output will be shown as LEARNING_VAR is no longer an environmental variable.

```
[root@controller ~]# echo $LEARNING_VAR
```

Hello OpenStack! should be displayed, as LEARNING_VAR is now a shell variable.

```
[root@controller ~]# set | grep LEARNING_VAR
```

LEARNING_VAR='Hello OpenStack!' will be displayed, confirming LEARNING_VAR exists as a shell variable.

15. A shell and environmental variable can be completely removed by issuing the unset command.

```
[root@controller ~]# unset LEARNING_VAR  
[root@controller ~]# echo $LEARNING_VAR
```

Nothing will be displayed, as LEARNING_VAR has been unset

```
[root@controller ~]# set | grep LEARNING_VAR
```

Nothing will be displayed.

05 To take a look at what is going on under the hood when you authenticate to Keystone, in addition to studying the nature of RESTful messages.

This task will be completed with assistance by the cURL command. Before we get started, remember that endpoints are URLs that point to OpenStack services. When you authenticate to Keystone you get back a token which has a service catalog in it. The service catalog is basically a list of the OpenStack services that you have access to and the URLs you can use to get to them; their endpoints.

NOTE: The OpenStack API documentation is available here: <http://developer.openstack.org/api-ref.html>.

1. Endpoint field definitions (reference)

1. The schema definition for an endpoint is in endpoints.xsd under keystone/content/common/xsd in the Keystone code repo. The fields are:
 - **id** - A unique ID for the endpoint.
 - **type** - The OpenStack-registered type (ex. 'compute', 'object-store', 'image service') This can also be extended using the OpenStack Extension mechanism to support non-core services. Extended services will be in the form extension:type (e.g. dnsextension:dns)
 - **name** - This can be anything that the operator of OpenStack chooses. It could be a brand or marketing name (ex. Rackspace Cloud Servers).
 - **region** - This is a string that identifies the region where this endpoint exists. Examples are 'North America', 'Europe', 'Asia'. Or 'North' and 'South'. Or 'Data Center 1', 'Data Center 2'. The list of regions and what a region means is decided by the operator. The spec treats them as opaque strings.
 - **publicURL** - This is the URL to use to access that endpoint over the internet.
 - **internalURL** - This is the URL to use to communicate between services. This is generally a way to communicate between services over a high bandwidth, low latency, unmetered (free, no bandwidth charges) network. An example would be if you want to access a swift cluster from inside your

Nova VMs and want to make sure the communication stays local and does not go over a public network and rack up your bandwidth charges.

- **adminURL** - This is the URL to use to administer the service. In Keystone, this URL is only shown to users with the appropriate rights.
- **tenantId** - If an endpoint is specific to a tenant, the tenantId field identifies the tenant that URL applies to. Some operators include the tenant in the URLs for a service, while others may provide one endpoint and use some other mechanism to identify the tenant. This field is therefore optional. Having this field also means you do not have to parse the URL to identify a tenant if the operator includes it in the URL.
- **versionId** - This identifies the version of the API contract that endpoint supports. While many APIs include the version in the URL (ex: <https://compute.host/v1>), this field allows you to identify the version without parsing the URL. It therefore also allows operators and service developers to publish endpoints that do not have versions embedded in the URL.

2. Issue a cURL command to the OpenStack Identity Service (Keystone)

1. SSH to your controller as root, you might use:

- While holding the control key, click: (<https://amel.com/sshproxy>)

2. Cut and paste the following to cURL the keystone endpoint.

```
curl -s -X POST http://192.168.0.10:5000/v2.0/tokens -H "Content-Type: application/json" -d '{"auth":{"tenantName":"admin","passwordCredentials":{"username":"admin","password":"amel"}}}' | python -m json.tool | less
```

-s Silent or quiet mode (don't show error or progress messages)
-X Forces the method that follows (this HTTP message must use a POST in this case)

-H Include this header

-d Send this data in the body of the POST

python -m json.tool Formats the output in a 'pretty' manner

less Launches the less utility so we can easily scroll up and down

For students who do not have much experience using the less command:

Press CTRL + F to go forward one window, and CTRL + B to go back one window
Press CTRL + D to go forward one half window, and CTRL + U to go back one half window
Press q to quit Less utility and return to the CLI
If you have never used the less utility, you might find the following cheat sheet useful (<http://sheet.shiar.nl/less>)

3. Here is a snippet of what will be returned, but we're going to take some time to see if we can understand it all.

```
{
  "endpoints": [
    {
      "adminURL":
"http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46",
      "id": "5aed2a95d76c42728b89804900f4ab85",
      "internalURL":
"http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46",
      "publicURL":
"http://192.168.0.10:8774/v2/300b2cc45c3846939e589310ae714e46",
      "region": "RegionOne"
    }
  ],
  "endpoints_links": [],
  "name": "nova",
  "type": "compute"
}
```

4. Based on the output returned to you, answer the following questions:
 - What services are available to this user?
 - What user's service catalog was returned?
 - Was this user given a token?
 - How long is this token good for? (what amount of time?)
 - What is Nova's Admin URL?
 - To what region are these services associated?
5. The information we just studied was in the body of an HTTP response. Let's tweak our curl command, and take a look at the HTTP responses that come back.

```
curl -v -X POST http://192.168.0.10:5000/v2.0/tokens -H "Content-Type:
application/json" -d
'{"auth":{"tenantName":"admin","passwordCredentials":{"username":"admin","pa
ssword":"ame1"}}}' | less
```

- The json message body is no longer displayed in a pretty manner, but that's alright. What is the type of HTTP response that was returned?

Hint: This HTTP response indicates success.

6. Just for fun, let's issue another cURL command, but this time change the password we are passing so that it is incorrect.

```
curl -v -X POST http://192.168.0.10:5000/v2.0/tokens -H "Content-Type: application/json" -d '{"auth":{"tenantName":"admin","passwordCredentials":{"username":"admin","password":"ame14567"}}}' | python -m json.tool | less
```

- What is the type of HTTP response that was returned? Was any json returned in the message body? Why not?
7. That's it for this lab! While other students are finishing up, feel free to play around with other permutations of the cURL command.

Some content of this lab was inspired and adapted from <https://github.com/mseknibilel/OpenStack-Install-and-Understand-Guide>

More on Git & GitHub later...

06 Launch an instance from the CLI (utilizing the Nova service).

When an instance is launched, it is assigned a flavor, which is a set of resources. Flavors can be of the type public, or private, so we'll study both.

1. Creating new (public) flavors at the CLI

1. SSH to your controller as root, you might use:
 - While holding the control key, click: (<https://amel.com/sshproxy>)
2. Run help on the nova service, and review the nova usage. For now, pay special attention to the flavor commands. You can run help by typing the following:

```
[root@controller ~]# source keystonerc_admin
[root@controller ~(keystone_admin)]# nova help | less
```

- Using the less command:
- Press CTRL + F to go forward one window, and CTRL + B to go back one window

- Press CTRL + D to go forward one half window, and CTRL + U to go back one half window
 - Press q to quit Less utility and return to the CLI
 - If you have never used the less utility, you might find the following cheat sheet useful (<http://sheet.shiar.nl/less>)
3. Use nova flavor-list to see what default flavors are installed with OpenStack. When you see these, the concept of 'resources' should become more clear.

```
[root@controller ~(keystone_admin)] nova flavor-list
```

ID	Name	Memory_MB	Disk	Ephemeral	Swap	VCPUs	RXTX_Factor	Is_Public
1	m1.tiny	512	1	0		1	1.0	True
2	m1.small	2048	20	0		1	1.0	True
3	m1.medium	4096	40	0		2	1.0	True
4	m1.large	8192	80	0		4	1.0	True
5	m1.xlarge	16384	160	0		8	1.0	True

RXTX Factor - Aggregate outbound bandwidth, in megabits per second, across all attached network interfaces (PublicNet, ServiceNet, and Cloud Networks). Outbound public Internet bandwidth can be up to 40% of the aggregate limit. Host networking is redundant, and bandwidth is delivered over two separate bonded interfaces, each able to carry 50% of the aggregate limit. We recommend using multiple Layer 4 connections to maximize throughput. Inbound traffic is not limited.

Picking an example: first row 512MB standard instance, RXTX factor 80.00/mbs is the maximum speed across all the network interfaces (public, private, cloud network) that data can leave the server

4. Answer the following questions:
- How many more VCPUs does m1.xlarge have over m1.tiny?
 - How much more memory does m1.xlarge have over m1.tiny?
 - What do you think Is_Public means?
 - Hint: Is it possible a specific tenant might require access to a 'special' flavor?
5. Maybe all of our customers want access to a specific flavor; one with 512 Mb of RAM, 2 volumes, 1 VCPU, named **"custom.shrimpy"**. We can do that with the following command:

```
[root@controller ~(keystone_admin)]# nova flavor-create custom.shrimpy 6 512 2 1
```

- Question: Look at the command you just copy & pasted. The values 512 (Mb of RAM), and 2 (volumes), and 1 (VCPU) should be clear. So why was the value of 6 included?

- Hint: Run the command "nova flavor-list", and examine the table once more. 1... 2... 3... 4... 5...
 - If the command is still unclear, run 'nova help flavor-create'
6. Look for **custom.shrimpy** on the flavor list as multiple users. What we are illustrating, is that this is a publicly available flavor.

```
[root@controller ~(keystone_admin)]# nova flavor-list
```

ID	Name	Memory_MB	Disk	Ephemeral	Swap	VCPUs	RXTX_Factor	Is_Public
1	m1.tiny	512	1	0		1	1.0	True
2	m1.small	2048	20	0		1	1.0	True
3	m1.medium	4096	40	0		2	1.0	True
4	m1.large	8192	80	0		4	1.0	True
5	m1.xlarge	16384	160	0		8	1.0	True
6	custom.shrimpy	512	2	0		1	1.0	True

<---- !!!

7. Switch to the user Alice Anderson, and confirm she can see the flavor. Type the following:

```
[root@controller ~(keystone_admin)]# source keystone_rc_amelirene
[root@controller ~(keystone_amelirene)]# nova flavor-list
```

8. Switch to the user Chester Copperpot, and confirm he can see the flavor. Type the following:

```
[root@controller ~(keystone_alicenaderson)]# source
keystone_rc_chestercopperpot
[root@controller ~(keystone_chestercopperpot)]# nova flavor-list
```

2. Creating new (private) flavors at the CLI

Alice Anderson and her team from tenant acme_inc might need a custom flavor that is uniquely tuned for a project she is working on. For example, Alice's team needs to occasionally render video on a 'monster' instance with 128 GB of memory. We want Alice and her team to have access to this instance, but not our other customers. If we create a new flavor as described above, the user would have access to the custom flavor, but so would all other tenants in the cloud. Giving everyone access to a 'monster' flavor might not be desirable. Customers not requiring (or paying) for the 'monster' flavor might drive our cloud into full capacity rather quickly. To prevent this, let's restrict access to the custom flavor.

1. If you haven't done so before, issue the nova help flavor-create command.

```
[root@controller ~(keystone_chestercopperpot)]# nova help flavor-create
```

- Review the output, then answer the following questions:

- Is RAM described in GB or MB?
 - Is Disk size described GB or MB?
 - What is the default value for --is-public?
2. Notice that a flavor may have an optional argument --is-public to be set as true or false, where the **default is true**. Let's toggle it to false and see what happens.

```
[root@controller ~(keystone_chestercopperpot)]# source keystone_admin
[root@controller ~(keystone_admin)]# nova flavor-create --is-public false
secret.monster 8 125000 8 200
[root@controller ~(keystone_admin)]# nova flavor-list
```

- Why is the number 8 included after secret.monster?
 - Hint: If you've forgotten, scroll up a little bit and look at the output from the nova help flavor-create command
 - Do you see the new secret.monster flavor listed? Think for a moment. Why or why not?
 - Hint: Did we say which tenants have access to this secret flavor yet?
3. Read nova help screen as a review to create We need to tell OpenStack who should gain access to this project. Let's choose admin to start. Type the following:

```
[root@controller ~(keystone_admin)]# nova help | grep flavor
flavor-access-add      Add flavor access for the given tenant.
flavor-access-list     Print access information about the given flavor.
flavor-access-remove   Remove flavor access for the given tenant.
flavor-create          Create a new flavor
flavor-delete          Delete a specific flavor
flavor-key             Set or unset extra_spec for a flavor.
flavor-list            Print a list of available 'flavors' (sizes of
flavor-show            Show details about the given flavor.
[root@controller ~(keystone_admin)]# nova help flavor-access-add
Add flavor access for the given tenant.
```

Positional arguments:

```
<flavor>      Flavor name or ID to add access for the given tenant.
<tenant_id>   Tenant ID to add flavor access for.
```

4. Discover the admin's username id and tenant ID

```
[root@controller ~(keystone_admin)]# keystone user-get admin
```

Property	Value	
email	root@localhost	
enabled	True	
id	e9483918caa940a48fe882ba092c7e0b	<-- admin username id
name	admin	
tenantId	300b2cc45c3846939e589310ae714e46	<-- admin tenant id (aaaa below)
username	admin	

5. Copy the **admin tenantId**

```
[root@controller ~(keystone_admin)]# nova flavor-access-add 8 aaaa <-- replace
aaaa from above
[root@controller ~(keystone_admin)]# nova flavor-list
```

You have just allowed the tenant admin access to flavor #8, secret-monster. This is why admin can now see the flavor

6. Give amelirene's tenant access to the secret.monster flavor as well.
The commands you will use are all here, hold the control key and click:
<https://amel.com/labs/openstack/keystone-commands/>

7. Discover amelirene's username id and tenantId [root@controller ~(keystone_admin)]# keystone user-get amelirene <-- Discover amelirene's **tenantId**

Property	Value	
email		
enabled	True	
id	070c514afa1a463882008c36a269ff87	<--Alice's username ID
name	amelirene	
tenantId	37d530d789ca43ddae4ad18b395218ae	<- Alice's tenantID (xxxx below)
username	amelirene	

We now know amelirene's tenantId is a really big number, (37d530d789ca43ddae4ad18b395218ae), but unless we have a mind that can remember every customerID in our database, we probably do not remember that **37d530d789ca43ddae4ad18b395218ae** is "**acme_inc**". Interestingly, since working on OpenStack, I constantly tell my wife that her new name is "**070c514afa1a463882008c36a269ff87**", but she NEVER responds. All she would have to do is a keystone user-get 070c514afa1a463882008c36a269ff87 and she would clearly see the association, but she refuses! I cannot figure out why...

8. Find out who amelirene's really big tenantID number really is.

```
[root@controller ~(keystone_admin)]# keystone tenant-get xxxx <-- Replace xxxx
with alice's tenantId from above
```

Property	Value	
description	Acme Inc. is the world's leader in the production of widgets	
enabled	True	
id	37d530d789ca43ddae4ad18b395218ae	<-- acme_inc's tenant (zzzz below)
name	acme_inc	

9. Give acme_inc access to flavor secret.monster

```
[root@controller ~(keystone_admin)]# nova flavor-access-add 8 zzzz where zzzz
= the tenant id from above
```

10. Check out if you were successful!

```
[root@controller ~(keystone_admin)]# source keystonerc_amelirene
[root@controller ~(keystone_amelirene)]# nova flavor-list
```

Alice Anderson should see the presence of secret.monster displayed, as the tenant acme_inc was given access to the project

11. Find out if Chester Copperpot (a tenant of VaultTek) can see the project.

```
[root@controller ~(keystone_amelirene)]# source keystonerc_chestercopperpot
[root@controller ~(keystone_chestercopperpot)]# nova flavor-list
```

Failed? Good! The user chestercopperpot is a member of VaultTek and VaultTek cannot see the flavor secret.monster because VaultTek has not been given access to the flavor secret.monster.

3. Launching instances from the CLI

1. Let's try launching some instances from the CLI. SSH to your controller and log in as root, source yourself as the keystone_chestercopperpot user permission set.
2. See what flavor, images, and networks are available for use. Type the following commands:

```
[root@controller ~(keystone_chestercopperpot)]# nova flavor-list
[root@controller ~(keystone_chestercopperpot)]# nova image-list
[root@controller ~(keystone_chestercopperpot)]# neutron net-list
```

id	name	subnets
31250e3e-feb0-421a-95f2-41a36f627507	public	ba02f29e-de66-4f21-84c4-777ce7e863c4 172.24.4.224/28
20ca30a2-e3fb-4e62-bd78-08dc471e93ed	private	86e6e695-7775-46ef-827f-64c4306bb69f 10.0.0.0/24 <-- private

3. There are a few things we have not studied yet that we are working with here. We have not studied networking yet. Normally, we would first create a network for a tenant, and then launch that instance within that tenant's private network. We can study that later, for now, highlight the id of the private network
- Remember, highlighting will auto-copy
4. Boot an instance called vt1 with the flavor custom.shrimpy, from the image cirros, and place it on the private network (more on networking is coming up)

NOTE: Instances typically will be placed on their own tenant network, and include a 'security group' (a set of IP rules); things we haven't yet studied (we'll get there

in subsequent labs). Launching an instance without defining a security group, will result in using the 'default' security group, which denies all incoming traffic.

```
[root@controller ~(keystone_chestercopperpot)]# nova boot --flavor custom.shrimpy --image cirros --nic net-id=<xxxxxxxxxxxx> vt1
```

where <xxxxxxxx> = the internal private_network_id that you cut and paste from the above command `neutron net-list`

Property	Value
OS-DCF:diskConfig	MANUAL
OS-EXT-AZ:availability_zone	nova
OS-EXT-STS:power_state	0
OS-EXT-STS:task_state	scheduling
OS-EXT-STS:vm_state	building
OS-SRV-USG:launched_at	-
OS-SRV-USG:terminated_at	-
accessIPv4	
accessIPv6	
adminPass	7XbbjiX9Pbcn
config_drive	
created	2015-10-04T22:18:35Z
flavor	custom.shrimpy (7)
hostId	
id	bee8e2d9-ac2e-4190-a251-139216bb0a8b
image	cirros (8a724f78-2673-4ea1-b607-373626a15afb)
key_name	-
metadata	{}
name	vt1
os-extended-volumes:volumes_attached	[]
progress	0
security_groups	default
status	BUILD
tenant_id	ad1d9eeaf2884c7e8eac33ec0f3ef6e5
updated	2015-10-04T22:18:36Z
user_id	85787ab96a6f442a8ab51bac131175b4

- Of course your output will be unique from what is listed above. Use your output to answer the following questions:
- What is the task state?
- What ID was this instance assigned?
- What tenant does this instance belong to?
- What name was this instance given?
- What image was used to boot this instance?

5. Display a list of the running instances within the current tenant

```
[root@controller ~(keystone_chestercopperpot)]# nova list
```

- What is the status?
- What is the power state?
- Hint: Issue the command again in a few minutes if they do not say "ACTIVE" and "Running" respectively.

6. Display information only about the running instance vt1 that was just created:

```
[root@controller ~(keystone_chestercopperpot)]# nova show vt1
```

7. Log into the Horizon dashboard interface as chestercopperpot // fa5tpa55w0rd then navigate to:

- Navigate to: Project > Compute > Instances
 - You've had some experience clicking around in Horizon instances before. Do it again! If nothing else, check out the log and console tabs for the instance.
 - This found by clicking on the instance name (vt1), then clicking on the log tab (or console tab).
8. That's it for this lab! We'll continue to explore manipulating instances, but feel free to exploring launching instances from the CLI or Horizon, or checking out other Nova commands while you wait.

07 administration of instances within the OpenStack cloud.

Instances all have different 'states', and it is important to understand what those might be, and when it may be appropriate for an admin to force a customer instance into a different state. This lab has you select an instance from a project (which admin may or may not be a part of), and manipulate it as the OpenStack admin. If you finish this lab early, back through it again with an instance from a different project (tenant), or practice any admin skills involving instances (diagnostic details / start / stop / suspend / pause / reboot / rescue).

1. Getting at an instance with admin

1. SSH to your controller as root, you might use:
 - While holding the control key, click: (<https://amel.com/sshproxy>)
2. Once logged into the controller, issue the following command:

```
[root@controller ~]# source keystone_admin
```

3. First look at all of the instances in the on the OpenStack cloud. Note that the command we are about to issue will not tell us any details about the instances that are running (what compute node is hosting the instance, the flavor it is running, the image being utilized, and so on). Type the following command:

```
[root@controller ~(keystone_admin)]# nova list --all-tenants
```

ID	Name	Tenant ID	Status	Task State	Power State	Networks
----	------	-----------	--------	------------	-------------	----------


```
| a06d7547-d380-44e6-9be8-650240faada0 | vt1 | ad1d9eeaf2884c7e8eac33ec0f3ef6e5 | ACTIVE | - | Running | public=172.24.4.231; private=10.0.0.7 | <-- Our instance from the last lab!
```

- Your output might differ, after all this table is a reflection of what instances you have created. In the last lab, we all created an instance (vt1), so we should all see it ACTIVE and Running.
 - If the instance is not ACTIVE and Running, let the instructor know
4. Highlight the id for the instance vt1 (NOT the Tenant ID), then issue the following command to get a list of details about the instance. Throughout this lab, we'll continually reference this highlighted ID, so we could keep highlighting it, or set a shell variable that we can reference.
- We'll adhere to the convention of keeping all caps for the variable name (VAR_UUID), and set it to a simple string.

```
[root@controller ~(keystone_admin)]#
VAR_UUID=replace_with_the_id_for_vt1_you_highlighted
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

5. Your input should look like this. Answer the questions below:

```
+-----+-----+-----+-----+-----+-----+ Property | Value | +-----+  
| OS-DCF:diskConfig | MANUAL | | OS-EXT-AZ:availability_zone | nova | | OS-EXT-SRV-ATTR:host | compute2.localdomain  
-<- D || OS-EXT-SRV-ATTR:hypervisor_hostname | compute2.localdomain <-- D || OS-EXT-SRV-ATTR:instance_name | instance-00000008 || OS-EXT-  
STS:power_state | 1 || OS-EXT-STS:task_state | - | | OS-EXT-STS:vm_state | active <-- C || OS-SRV-USG:launched_at | 2015-11-03T21:53:31.000000 || OS-SRV-  
USG:terminated_at | - | | accessIPv4 | | | accessIPv6 | | | config_drive | | | created | 2015-11-03T21:52:08Z <- A | | flavor | custom.shrimpy (6) <- B | | hostId |  
9fe51a49b6f300c621228a6c4de6b662d9b14a0bf2e12771b32e27528 || id | ae7ee3ae-15fc-4da5-907f-16d924db6b920 || image | cirros (Ba724f78-2673-4a41-b607-  
373626a15afb) || key_name | - | | metadata | {} || name | vt1 || os-extended-volumes:volumes_attached || private network | 10.0.0.11 <--- E || progress | 0 ||  
security_groups | [default] || status | ACTIVE <--- C || tenant_id | 0fe62574bea74125ac00a0a2fbcca18 <- F || updated | 2015-11-03T21:53:32Z || user_id |  
9ba3000acc3c4c3d98a29912dad40d740 | +-----+-----+-----+-----+-----+-----+
```

- A. What time was this instance created?
- B. What flavor is being utilized?
- C. Is the machine powered on, or powered off?
- D. Which compute node is hosting this instance?
- E. What is the IP address of this instance?
- F. To what tenant does this instance belong?
- Hint: Having trouble figuring out the tenant to which this instance belongs? Here's what to do, locate the value for 'tenant_id', it is a long UUID value. Issue a request to the identity service (keystone) for a list of all of the tenant IDs.

```
[root@controller ~(keystone_admin)]# keystone tenant-list
```

6. To reveal the name of the tenant, match the value of 'tenant_id' with the displayed ids. Sometimes administrators need to troubleshoot issues with an instance. An admin can retrieve diagnostic information about an instance with the following command (once again, highlight the id of the instance)

```
[root@controller ~(keystone_admin)]# nova diagnostics $VAR_UUID
```

7. Answer the following questions about the output from nova diagnostics:

- (CPU details) What is the CPU time (in nano seconds)?
- (Memory details) What is the max amount of memory (in MB)?
- (Memory details) What is the actual amount of memory currently being used?
- (Network details) How many octets have been received?
- (Network details) How many octets have been dropped?
- (Network details) How many packets have been transmitted?
- (Disk details) How many disk reads have their been (in bytes)?
- (Disk details) How many disk write requests have their been?

8. Now let's set some metadata about our instance. Here I am using mtag1, but you can use anything. Try starting with the following:

```
[root@controller ~(keystone_admin)]# nova meta $VAR_UUID set mtag1='vault_tek box'
```

9. Look to see that your first metadata tag has been set.

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

10. Now let's set a second metadata tag concerning our instance.

```
[root@controller ~(keystone_admin)]# nova meta $VAR_UUID set mtag2='uh oh I set this tag incorrectly'
```

11. Look to see that your second metadata tag has been set.

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

12. Now remove the second metadata tag.

```
[root@controller ~(keystone_admin)]# nova meta $VAR_UUID delete mtag2
```

13. Confirm the second metadata tag has been deleted.

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

2. Starting and Stopping instances with admin

In this section, we'll start and stop an instance. If an instance is already stopped, it can't hurt to start it. Just like if it's started, it won't hurt to start the instance 'again' (in both cases a 409 HTTP error is returned).

1. Stop the instance (vt1)

```
[root@controller ~(keystone_admin)]# nova stop $VAR_UUID
```

2. Check to see if the instance (vt1) is really stopped

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

3. Start the instance (vt1). Note that admin may or may not be part of this project, and is still able to manipulate this instance.

```
[root@controller ~(keystone_admin)]# nova start $VAR_UUID
```

4. Is the instance running? Find out by using the nova show command

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

3. Pausing and unpausing instances with admin

In this section we'll pause and unpauses a running instance. Pausing an instance stores the instance in memory (RAM), so in laptop terms, think of it as putting the instance into "sleep" mode.

1. Try pausing the instance.

```
[root@controller ~(keystone_admin)]# nova pause $VAR_UUID
```

2. Use the CLI to confirm that the instance is paused

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

3. Use the browser to confirm that the instance is paused. Log into the Horizon OpenStack dashboard as `chestercopperpot // fa5tpa55w0rd` and confirm that the instance is indeed paused.

4. Try unpausing the instance

```
[root@controller ~(keystone_admin)]# nova unpause $VAR_UUID
```

5. Use the CLI to confirm that the instance is ACTIVE

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

6. Once again, use the Horizon OpenStack dashboard as `chestercopperpot // fa5tpa55w0rd` to confirm that the instance is Running.

4. Suspending and resuming instances with admin

In this section we'll suspend and resume a running instance. Suspending an instance stores the instance on the disk, so in laptop terms, think of it as putting the instance into "hibernation" mode.

1. Try suspending the instance

```
[root@controller ~(keystone_admin)]# nova suspend $VAR_UUID
```

2. Use the CLI to confirm that the instance is suspended

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

3. Use the browser to confirm that the instance is suspended. Log into the Horizon OpenStack dashboard as `chestercopperpot // fa5tpa55w0rd` and confirm that the instance is indeed suspended.

4. Try resuming the instance

```
[root@controller ~(keystone_admin)]# nova resume $VAR_UUID
```

5. Use the CLI to confirm that the instance is resumed.

```
[root@controller ~(keystone_admin)]# nova show $VAR_UUID
```

6. Once again, use the Horizon OpenStack dashboard as `chestercopperpot // fa5tpa55w0rd` to confirm that the instance is running.

5. Restarting an instance with admin

In this section we'll perform a reset of a customer instance (vt1) from the admin account.

1. Issue the following commands to reboot the instance and then view the status (this is a graceful shutdown, to perform a forceful shutdown, include `--hard` after the reboot command)

```
[root@controller ~(keystone_admin)]# nova reboot $VAR_UUID
```

HURRY! You have less than 1 second to issue the next command... Just try again if you do not see the REBOOT status (No one ever gets this on the first try!) Most people issue the next command and it is already too late, so the status shows ACTIVE, therefore, use the up arrow key to reissue the `nova reboot`

\$VAR_UUID and the nova show \$VAR_UUID back to back. The second try usually will give the results shown below. Of course you could always try issuing both commands on the same line: nova reboot \$VAR_UUID && nova show \$VAR_UUID but where is the challenge in that? :)

[root@controller ~(keystone_admin)]# nova show \$VAR_UUID

Property	Value
OS-DCF:diskConfig	MANUAL
OS-EXT-AZ:availability_zone	nova
OS-EXT-SRV-ATTR:host	compute2.localdomain
OS-EXT-SRV-ATTR:hypervisor_hostname	compute2.localdomain
OS-EXT-SRV-ATTR:instance_name	instance-00000008
OS-EXT-STS:power_state	1
OS-EXT-STS:task_state	reboot_started
OS-EXT-STS:vm_state	active
OS-SRV-USG:launched_at	2016-01-12T20:12:30.000000
OS-SRV-USG:terminated_at	-
accessIPv4	
accessIPv6	
config_drive	
created	2016-01-12T20:11:20Z
flavor	custom.shrimpy (6)
hostId	c9c3bf6f7af051fdafebba0f66a46f73d5d4532cf3274dfa782584
id	1baa79d5-9c70-470b-ada7-9ca800e271dc
image	cirros (8a724f78-2673-4ea1-b607-373626a15afb)
key_name	-
metadata	{"mtag1": "vault_tek box"}
name	vt1
os-extended-volumes:volumes_attached	[]
private_network	10.0.0.11
security_groups	default
status	REBOOT <---- This is what you are looking for!
tenant_id	f5db9e5a50ef40b593b05e6e9b3f0db5
updated	2016-01-12T21:08:01Z
user_id	2767b86c1dbe4e029c84af1d7b444aa0

You will have a second to issue the nova show \$VAR_UUID command or the above screen will show

- Continue to issue the nova show command until the instance is shown to be in a ACTIVE / running once again

6. Starting an instance with rescue

Sometimes a user/customer will lock themselves out of their machine, or perhaps they corrupt their file system. The result is that they are unable to access or start their instance. The solution, is to start the machine with nova rescue.

- nova rescue will first gracefully shutdown the instance (after 60 seconds, it will perform a poweroff, to increase this value, edit nova.conf)
- This command attaches a secondary bootdisk, so that the instance's volumes can be accessed through a Linux image (new login and password is expressed as Property and Value)
- Think of this process as removing a HDD from a computer that doesn't work, jamming it in a rescue machine so that you can try to fix it (or maybe reset the password)

1. To put a instance in rescue mode, issue the following:

```
[root@controller ~(keystone_admin)]# nova rescue $VAR_UUID
```

```
+-----+-----+
| Property | Value |
+-----+-----+
| adminPass | BpZh5d475JZ6 |
+-----+-----+
```

When SSH-ing into the rescued image:

Property means login

Value is the password

2. To take the instance out of rescue mode, issue the following (you can do this, or not, it is up to you):

```
[root@controller ~(keystone_admin)]# nova unrescue $VAR_UUID
```

Way to go! That's it for this lab. Hang tight until the other students are caught up.

08 understand how security groups work.

A common issue with OpenStack is failing to set appropriate security group(s) when launching an instance. As a result, the user is unable to contact the instance on the network. The security groups are sets of IP filter rules that are applied to the instance's networking.

Security Groups are also project specific, and project members can edit the default rules for their group and add new rules sets. All projects have a "default" security group, which is applied to instances that have no other security group defined. Unless changed, this security group denies all incoming traffic. If you've tried to connect to an instance in the past, this is likely one of the reasons you were unsuccessful.

1. Creating Security Groups at the CLI

1. SSH to your controller as root, then source the keystone_chestercopperpot permissions file.

```
[root@controller ] # source keystone_chestercopperpot
[root@controller ~(keystone_chestercopperpot)]#
```

2. Examine the current security group rules available to the vault-tek project (the project to which chestercopperpot belongs)

```
[root@controller ~(keystone_chestercopperpot)]# nova secgroup-list
```

Id	Name	Description
ccaf984f-f819-4aa6-943c-360d2f41ee97	default	Default security group

- Note that only a single security group will exist, which is named default.
- We need to create a few basic security group rules which we can apply to our instances. The group that is about to be created is called "http-ssh"

```
[root@controller ~(keystone_chestercopperpot)]# nova secgroup-create http-ssh "Allowing http and ssh traffic"
```

Id	Name	Description
0b79a2b0-ef2e-44de-a215-0c74759bbf0c	http-ssh	Allowing http and ssh traffic

- Create an exception for TCP traffic coming to and from port 80 via any IP address

```
[root@controller ~(keystone_chestercopperpot)]# nova secgroup-add-rule http-ssh tcp 80 80 0.0.0.0/0
```

IP Protocol	From Port	To Port	IP Range	Source Group
tcp	80	80	0.0.0.0/0	

- Create an exception for TCP traffic coming to and from port 22 via any IP address

```
[root@controller ~(keystone_chestercopperpot)]# nova secgroup-add-rule http-ssh tcp 22 22 0.0.0.0/0
```

IP Protocol	From Port	To Port	IP Range	Source Group
tcp	22	22	0.0.0.0/0	

- Review the rules that we just created for http-ssh security group

```
[root@controller ~(keystone_chestercopperpot)]# nova secgroup-list-rules http-ssh
```

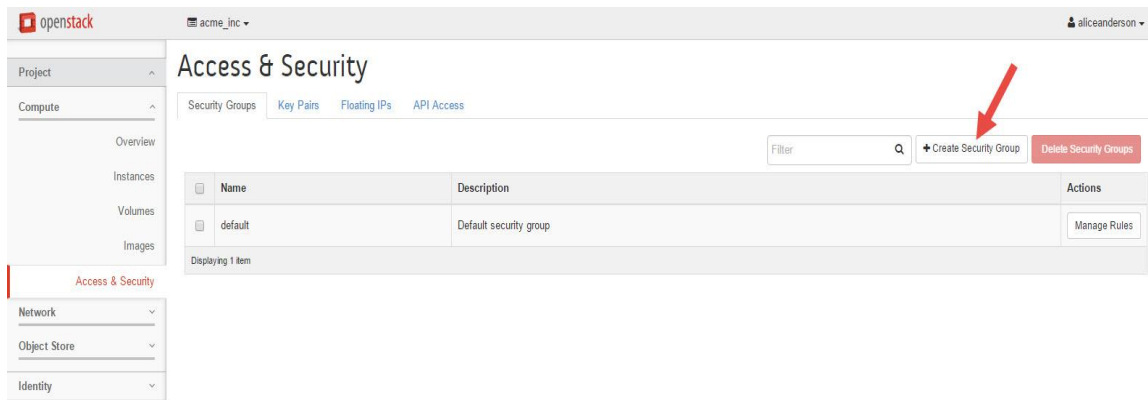
IP Protocol	From Port	To Port	IP Range	Source Group
tcp	80	80	0.0.0.0/0	
tcp	22	22	0.0.0.0/0	

- Good job! Now the vault-tek project could apply the http-ssh rule set to a launched instance (we'll do this soon), but for now, continue on...

7. Log into Horizon as chestercopperpot / fa5tpa55w0rd, and navigate to the following location:
 - Navigate to: Project > Compute > Access & Security
 - Can chestercopperpot see the new http-ssh rule? Why or why not?
8. Log into Horizon as admin, and navigate to the following location:
 - Navigate to: Project > Compute > Access & Security
 - Can admin see the new http-ssh rule? Why or why not?
9. Log into Horizon as amelirene / fa5tpa55w0rd, and navigate to the following location:
 - Navigate to: Project > Compute > Access & Security
 - Can amelirene see the new http-ssh rule? Why or why not?

2. Creating Security Groups at Horizon

1. While still logged in as amelirene, let's use Horizon to create a new security group rule for the tenant (project) to which amelirene belongs.
2. Within Horizon, navigate to: **Project > Compute > Access & Security**
3. In the Security Group tab, find the button +Create Security Group
 - If you are unsure of where to click, see the screenshot below.



4. A window should pop-up titled, "Create Security Group", fill this screen out as follows:

Name: **http-ssh**

Description: **Allows http and ssh access**

After filling out the pop-up window, click on the **Create Security Group** button. If you are unsure of where to click, see the screenshot below.

Create Security Group [X]

Name *
http-ssh

Description
Allows http and ssh access

Description:
Security groups are sets of IP filter rules that are applied to the network settings for the VM. After the security group is created, you can add rules to the security group.

Cancel Create Security Group

5. You should be looking at a page that includes the newly created rule, click on the "Manage Rules" button.

- If you are unsure of where to click, see the screenshot below.

openstack acme_inc aliceanderson

Access & Security

Security Groups Key Pairs Floating IPs API Access

Filter [Q] + Create Security Group Delete Security Groups

	Name	Description	Actions
☐	http-ssh		Manage Rules
☐	default	Default security group	Manage Rules

Displaying 2 items

6. You're now looking at the list of IP permissions currently applied. Click the "+Add Rule" button.

- If you are unsure of where to click, see the screenshot below.

Project

Compute

Overview

Instances

Volumes

Images

Access & Security

Network

Object Store

Identity

Manage Security Group Rules: http-ssh (2b34338a-54a9-4249-9f68-fa715d8b1bed)

+ Add Rule Delete Rules

Direction	Ether Type	IP Protocol	Port Range	Remote IP Prefix	Remote Security Group	Actions
Egress	IPv6	Any	Any	::0	-	Delete Rule
Egress	IPv4	Any	Any	0.0.0.0/0	-	Delete Rule

Displaying 2 items

7. Our first rule we create will be for SSH access. Fill out the windows as follows:

Rule: SSH (choose from dropdown menu)
 Remote: CIDR
 CIDR: 0.0.0.0/0

- Click on "Add" button when complete
- When finished, your rule should look like the one before

Add Rule

Rule *

SSH

Remote *

CIDR

CIDR

0.0.0.0/0

Add

Description:

Rules define which traffic is allowed to instances assigned to the security group. A security group rule consists of three main parts:

Rule: You can specify the desired rule template or use custom rules, the options are Custom TCP Rule, Custom UDP Rule, or Custom ICMP Rule.

Open Port/Port Range: For TCP and UDP rules you may choose to open either a single port or a range of ports. Selecting the "Port Range" option will provide you with space to provide both the starting and ending ports for the range. For ICMP rules you instead specify an ICMP type and code in the spaces provided.

Remote: You must specify the source of the traffic to be allowed via this rule. You may do so either in the form of an IP address block (CIDR) or via a source group (Security Group). Selecting a security group as the source will allow any other instance in that security group access to any other instance via this rule.

8. Repeat step 6, once again, click on the "+Add Rule" button

9. Our second rule we create will be for HTTP access. Fill out the windows as follows:

10. Rule: HTTP (choose from dropdown menu)

11. Remote: CIDR
12. CIDR: 0.0.0.0/0

- Click on "Add" button when complete
- When finished, your rule should look like the one before

Add Rule

Rule *
HTTP

Remote *
CIDR

CIDR
0.0.0.0/0

Description:
Rules define which traffic is allowed to instances assigned to the security group. A security group rule consists of three main parts:

Rule: You can specify the desired rule template or use custom rules, the options are Custom TCP Rule, Custom UDP Rule, or Custom ICMP Rule.

Open Port/Port Range: For TCP and UDP rules you may choose to open either a single port or a range of ports. Selecting the "Port Range" option will provide you with space to provide both the starting and ending ports for the range. For ICMP rules you instead specify an ICMP type and code in the spaces provided.

Remote: You must specify the source of the traffic to be allowed via this rule. You may do so either in the form of an IP address block (CIDR) or via a source group (Security Group). Selecting a security group as the source will allow any other instance in that security group access to any other instance via this rule.

Add

10. After pressing the "Add button", your screen should look like the following:

openstack acme_inc aliceanderson

Manage Security Group Rules: http-ssh (2b34338a-54a9-4249-9f68-fa715d8b1bed)

+ Add Rule Delete Rules

☐	Direction	Ether Type	IP Protocol	Port Range	Remote IP Prefix	Remote Security Group	Actions
☐	Egress	IPv6	Any	Any	::/0	-	Delete Rule
☐	Egress	IPv4	Any	Any	0.0.0.0/0	-	Delete Rule
☐	Ingress	IPv4	TCP	22 (SSH)	0.0.0.0/0	-	Delete Rule
☐	Ingress	IPv4	TCP	80 (HTTP)	0.0.0.0/0	-	Delete Rule

Displaying 4 Items

11. Click on the "Access & Security" link on the left.

The screenshot shows the OpenStack Horizon dashboard for the 'acme_inc' project. The left sidebar has a menu with 'Project', 'Compute', 'Access & Security' (highlighted), and 'Network'. The main content area is titled 'Access & Security' and has tabs for 'Security Groups', 'Key Pairs', 'Floating IPs', and 'API Access'. The 'Security Groups' tab is active, showing a table with two security groups: 'http-ssh' and 'default'. The 'http-ssh' group has a description 'Allows http and ssh access' and a 'Manage Rules' button. The 'default' group has a description 'Default security group' and a 'Manage Rules' button. Above the table are a search filter, a '+ Create Security Group' button, and a 'Delete Security Groups' button. The footer of the table says 'Displaying 2 items'.

12. If you see the new security group listed here, then that's it for this lab! We'll use this Security Group next time we launch and instance. Hang tight while the other students wrap up.

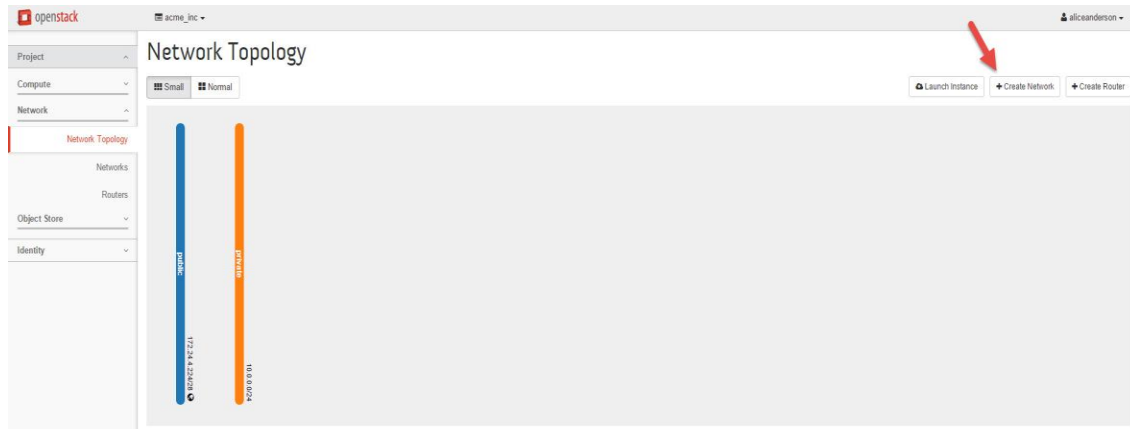
09 controlled from the OpenStack Horizon dashboard.

However, visually minded individuals may find it helpful to use the Horizon dashboard to to examine current network configuration as it does produce very clear illustrations of current network configuration. At the conclusion of this lab, students should feel comfortable creating and modifying network settings with Neutron at the CLI.

1. Creating a virtual network and a router with OpenStack Horizon dashboard

In this section, we'll create a network for a tenant (acme_inc), so that you can compare and contrast the differences between working with the OpenStack Horizon dashboard, and the CLI commands.

1. Log into the OpenStack Horizon dashboard on the controller as `amelirene // fa5tpa55w0rd`
2. Navigate to **Project > Network > Network Topology** and click "+ create network"



3. Fill out the pop up box with the following information:

```
NetworkName: acme-inc-network  
AdminState: UP
```

A screenshot of the 'Create Network' pop-up form. The form has three tabs: 'Network', 'Subnet', and 'Subnet Details'. The 'Network' tab is active. It contains a 'Network Name' text input field with the value 'acme-inc-network', and an 'Admin State' dropdown menu with the value 'UP'. A red arrow points to the 'Network Name' field. At the bottom right, there are three buttons: 'Cancel', '« Back', and 'Next »'. A red arrow points to the 'Next »' button.

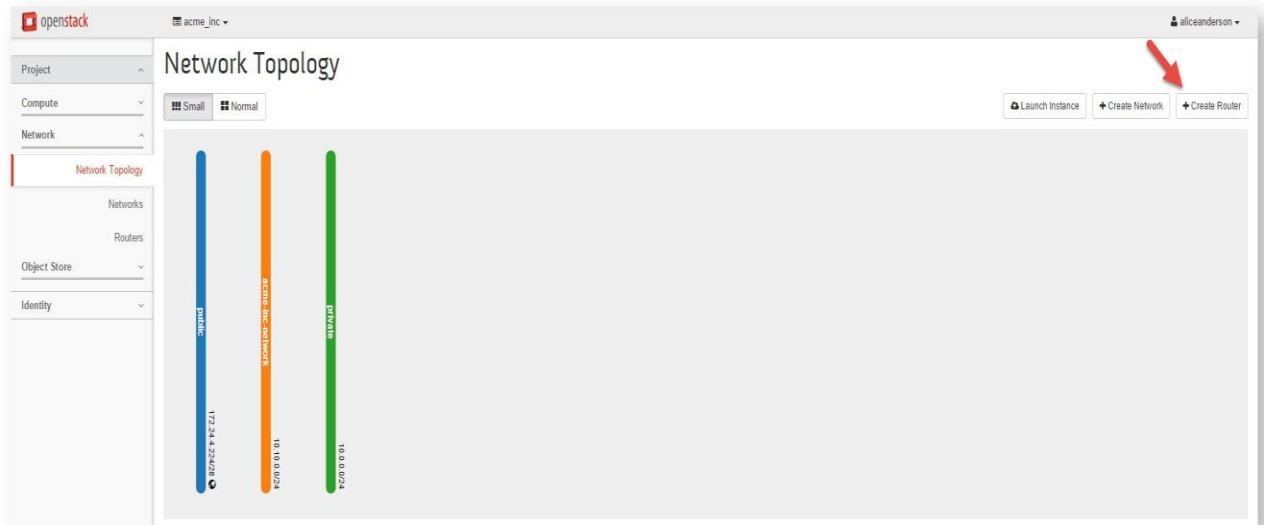
4. Fill out the next screen with the following information:

```
CreateSubnet: (checked)  
SubnetName: acme-inc-network-subnet-10  
NetworkAddress: 10.10.0.0/24  
IPVersion: IPv4  
GatewayIP: 10.10.0.1
```

The screenshot shows a 'Create Network' dialog box with three tabs: 'Network', 'Subnet', and 'Subnet Details'. The 'Subnet' tab is selected. A red arrow points to the 'Subnet' tab. Below the tabs, there is a checkbox labeled 'Create Subnet' which is checked. A red arrow points to this checkbox. To the right of the checkbox, there is a text block: 'Create a subnet associated with the new network, in which case "Network Address" must be specified. If you wish to create a network without a subnet, uncheck the "Create Subnet" checkbox.' Below this, there are four input fields: 'Subnet Name' with the value 'acme-inc-network-subnet-10', 'Network Address' with the value '10.10.0.0/24', 'IP Version' with a dropdown menu showing 'IPv4', and 'Gateway IP' with the value '10.10.0.1'. Red arrows point to each of these four fields. At the bottom right, there are three buttons: 'Cancel', '< Back', and 'Next >'. A red arrow points to the 'Next >' button. At the bottom left, there is a checkbox labeled 'Disable Gateway' which is unchecked.

- Hit ENTER or click NEXT.
 - The last screen is "Subnet Details". All the defaults here are fine (Enable DHCP should be checked, Allocation Pools should be empty, DNS Name server should be empty, and Host Routes should be empty). Just press the "Create" button to create your new network!
5. Now click the **"Create Router" button**

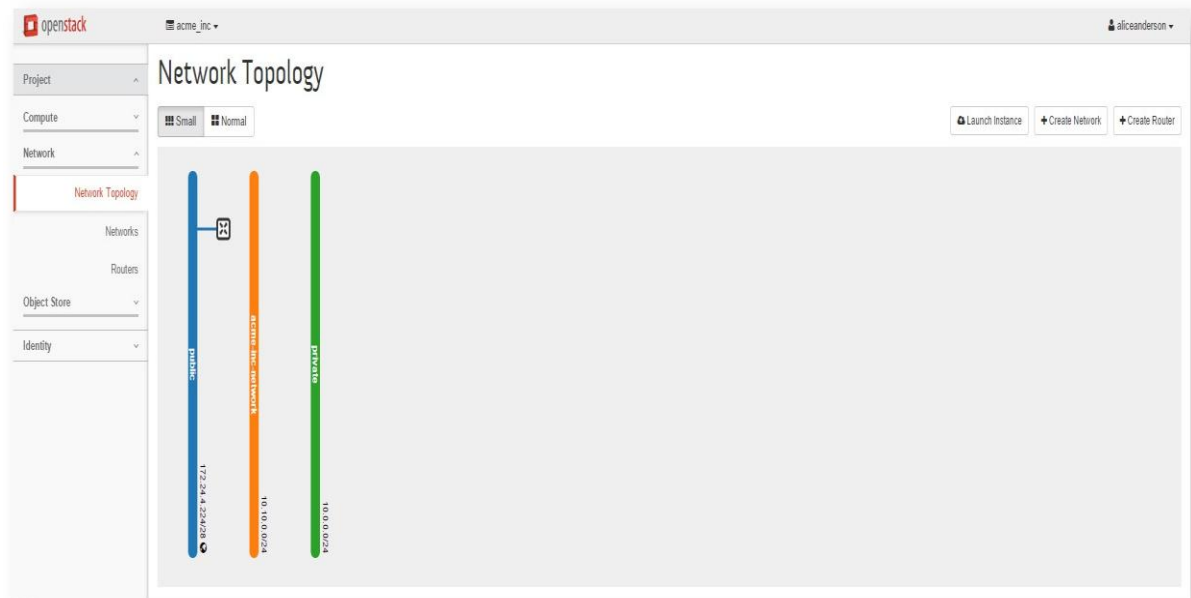
The Network Topology screen you are on should change to reflect the creation of this new network.



6. Let's create a router to connect our new network acme-inc-network to the network public. Click on the **" + Create Router "** button in the upper right corner.
7. Fill out pop up box with the following information:

```
RouterName: acme-inc-router  
AdminState: UP  
ExternalNetwork: public
```

The Network Topology page should now display the new router connected to the network public.



8. All that is left is to connect this router to the acme-inc-network. Start by navigating to **Project > Network > Routers**
9. Click on the router named **acme-inc-router**
10. Now click on the tab labeled **"Interfaces"**
11. Finally click on the button **"+ Add Interface"** in the upper right corner.
12. Fill out the pop up box with the following information:

```
Subnet: acme-inc-network: 10.10.0.0/24
IPAddress: 10.10.0.1
RouterName: acme-inc-router
RouterID:
```


Add Interface

Subnet *
acme-inc-network: 10.10.0.0/24 (acme-inc-netv)

IP Address (optional) ⓘ
10.10.0.1

Router Name *
acme-inc-router

Router ID *
df3b8543-6f68-49b6-816b-576aa269d26c

Description:
You can connect a specified subnet to the router.
The default IP address of the interface created is a gateway of the selected subnet. You can specify another IP address of the interface here. You must select a subnet to which the specified IP address belongs to from the above list.

Cancel Add interface

13. Finally, navigate back to **Project > Network > Network Topology**

14. The Network Topology screen should appear like the following screenshot:



Because the user chestercopperpot is part of a different project (vault_tek), he should not be able to see the newly created network (acme_inc_network). Let's confirm this...

15. Log out of the OpenStack Horizon dashboard, and log back in as chestercopperpot // fa5tpa55w0rd

16. Navigate to **Project > Network > Network Topology**

The user chestercopperpot should only see the public and private networks.

2. Creating a virtual network and a router at the CLI

In this section, we'll create almost the same network for a different tenant (vault_tek), so that you can compare and contrast the differences between working with the OpenStack Horizon dashboard and the CLI commands. SSH to your controller and log in as root (you might use PuTTY for this). Once logged into the controller, issue the following commands:

1. Create a network called vault-tek-network

```
[root@controller]# source keystonerc_chestercopperpot
[root@controller] ~(keystone_chestercopperpot)# neutron net-create vault-tek-network
```

```
[root@controller ~(keystone_chestercopperpot)]# neutron net-create vault-tek-network
Created a new network:
+-----+-----+
| Field | Value |
+-----+-----+
| admin_state_up | True |
| id | ff07f5b0-9d3c-45a9-ba4d-a6c58bc2612c |
| mtu | 0 |
| name | vault-tek-network |
| router:external | False |
| shared | False |
| status | ACTIVE |
| subnets | |
| tenant_id | 0e910ae3c644485fbf33062b9f0ccbea |
+-----+-----+
```

2. Create a vault-tek-network subnet

```
[root@controller] ~(keystone_chestercopperpot)# neutron subnet-create --name
vault-tek-network-subnet-10 --gateway 10.10.0.1 vault-tek-network 10.10.0.0/24
```

NOTE: The command listed above line wraps, so be sure to include the second line as well, or an error stating "too few arguments" will occur.

```
[root@controller ~(keystone_chestercopperpot)]# neutron subnet-create --name vault-tek-network-subnet-10 --gateway 10.10.0.1 vault-tek-network 10.10.0.0/24
Created a new subnet:
+-----+-----+
| Field          | Value                                     |
+-----+-----+
| allocation_pools | {"start": "10.10.0.2", "end": "10.10.0.254"} |
| cidr            | 10.10.0.0/24                             |
| dns_nameservers  |                                             |
| enable_dhcp      | True                                      |
| gateway_ip       | 10.10.0.1                                 |
| host_routes      |                                             |
| id               | 32d1f684-1195-477a-a595-9923f248f556      |
| ip_version       | 4                                          |
| ipv6_address_mode |                                             |
| ipv6_ra_mode     |                                             |
| name             | vault-tek-network-subnet-10               |
| network_id       | ff07f5b0-9d3c-45a9-ba4d-a6c58bc2612c      |
| subnetpool_id    |                                             |
| tenant_id       | 0e910ae3c644485fbf33062b9f0ccbea        |
+-----+-----+
```

3. Create a vault-tek-network router

```
[root@controller] ~(keystone_chestercopperpot)# neutron router-create vault-tek-router
```

4. Connect vault-tek-router to the public network

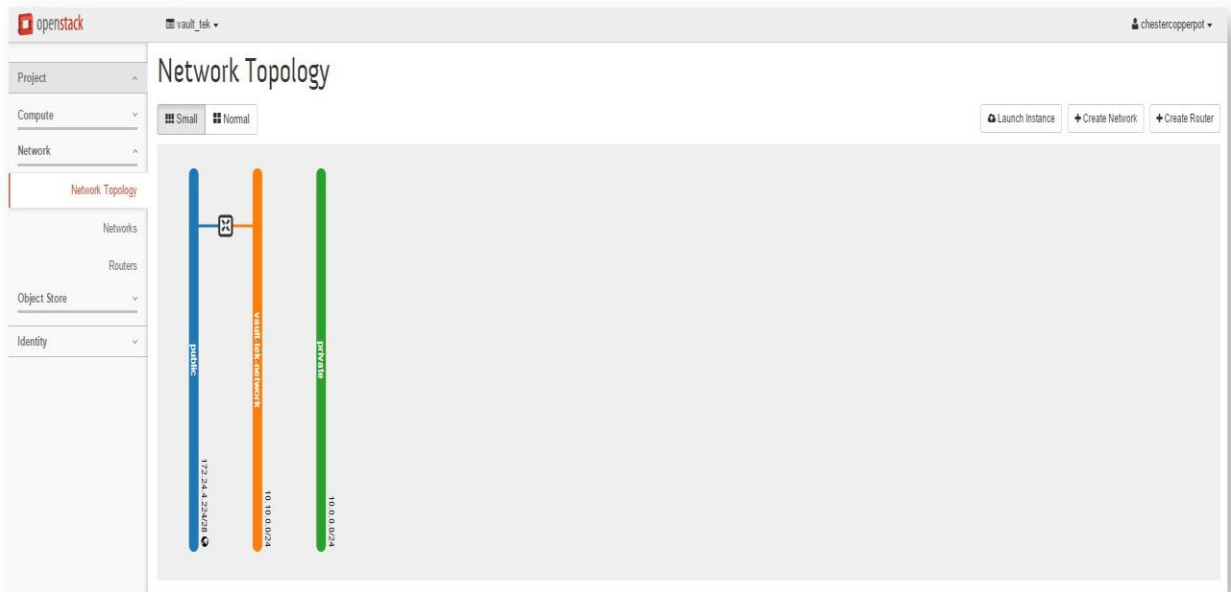
```
[root@controller] ~(keystone_chestercopperpot)# neutron router-gateway-set vault-tek-router public
```

5. Connect the vault-tek-router to vault-tek-network-subnet-10

```
[root@controller] ~(keystone_chestercopperpot)# neutron router-interface-add vault-tek-router vault-tek-network-subnet-10
```

6. Log into the OpenStack Horizon dashboard as chestercopperpot // fa5tpa55w0rd

7. Navigate to **Project > Network > Network Topology**



8. Navigate to **Project > Network > Networks**

Name	Subnets Associated	Shared	Status	Admin State	Actions
vault-tek-network	vault-tek-network-subnet-10 10.0.0.0/24	No	Active	UP	Edit Network
public	public_subnet 172.24.4.224/28	Yes	Active	UP	
private	private_subnet 10.0.0.0/24	Yes	Active	UP	

Note that this network is not a shared network (see the big red arrow on the above screenshot), therefore it cannot be seen by other projects (tenants). Let's confirm this.

- Log out of the OpenStack Horizon dashboard, log back in as a different user (admin, amelirene, or bobbarker) and navigate back to (Project > Network > Networks). You should no longer see vault-tek-network. After you've confirmed that the network is indeed private, log back into the OpenStack Horizon dashboard as chestercopperpot // fa5tpa55w0rd.

10. Navigate to **Project > Network > Routers**

11. Click on the router instance vault-tek-router (the bright red arrow is pointing to it in the above screenshot).

The screenshot shows the OpenStack dashboard with the 'Router Details' page for the 'vault-tek-router'. The left sidebar shows the navigation menu with 'Project', 'Compute', 'Network', 'Network Topology', 'Networks', 'Routers', 'Object Store', and 'Identity'. The 'Routers' section is highlighted. The main content area shows the router's details under the 'Overview' tab. The router's name is 'vault-tek-router' and its ID is '83d1e560-0532-49bd-a72f-7e21dd6aff46', which is highlighted in yellow. Other details include Project ID '92afd0ea2d8c4d2b939675db2473393f', Status 'ACTIVE', and Admin State 'UP'. The 'External Gateway' section shows the Network Name 'public', Network ID '31250e3e-feb0-421a-95f2-41a36f627507', Subnet ID 'ba02f29e-de66-4f21-84c4-777c7e7e863c4', IP Address '172.24.4.235', and SNAT 'Enabled'. A 'Clear Gateway' button is visible in the top right.

12. IMPORTANT: On the screen you are currently on, look for a value called "ID" which is highlighted yellow in the figure above. It is second down on the overview tab. Write down the first 6 or so characters. These will be helpful later.
13. Spend some time checking out the information being presented here. Click: **Network > Routers**, then click on the blue **vault-tek-router**, then you can click on the **Overview** and **Interfaces** tabs. The example below is clicked on the Interfaces tab. Note that the router is connected to a Gateway (public). Finally, click on the interfaces tab (the bright red arrow is pointing to it in the above screenshot). This page displays a list of associated interfaces connected to the router. Note that this router has a fixed IP address (10.10.0.1), which connects it to the 10.10.0.0/24 network we just created.

The screenshot shows the 'Router Details' page for the 'vault-tek-router' with the 'Interfaces' tab selected. The left sidebar is the same as in the previous screenshot. The main content area shows a table of interfaces connected to the router. A red arrow points to the 'Fixed IPs' column, which contains the value '10.10.0.1'. The table has columns for Name, Fixed IPs, Status, Type, Admin State, and Actions. The interface name is '(43ba945-bedf)' and its status is 'Active'. The type is 'Internal Interface' and the admin state is 'UP'. There are buttons for 'Add Interface', 'Delete Interfaces', and 'Delete Interface'.

- You are looking at the interface on the vault-tek-router

3. Launch an instance, and SSH into it (this is WAY COOL!)

Now that we know a bit about security groups and networks, let's launch a new instance, and see if we can SSH into it!

2. Navigate to **Project > Compute > Instances**
3. Click on the "Launch Instance" button
4. Fill out the Details tab as follows:

```
Availability Zone: nova
Instance Name: vt2
Flavor: m1.tiny
Instance Count: 1
Instance Boot Source: image
Image name: cirros (12.6 MB)          <- cirros ships with OpenStack, and
is an OS used to test out VMs. It is as lightweight as a Linux OS can
possibly be
```

5. Now click on the **Access & Security** tab so we can add our security group
 - Check the box **http-ssh**
 - Uncheck the box **default**
6. Now click on the **Networking** tab and click the + sign beside the **vault-tek-network**
7. Great! All that is left is to click the "Launch" button
 - Wait until the machine spawns and comes up as **ACTIVE** and **Running** before going to the next step
8. Before we SSH to the Neutron server, type the following command to check out your new instance:

```
[root@controller] ~(keystone_chestercopperpot)# nova show vt2
```

Property	Value
OS-DCF:diskConfig	AUTO
OS-EXT-AZ:availability_zone	nova
OS-EXT-STS:power_state	1
OS-EXT-STS:task_state	-
OS-EXT-STS:vm_state	active
OS-SRV-USG:launched_at	2015-10-28T20:43:59.000000
OS-SRV-USG:terminated_at	-
accessIPv4	
accessIPv6	
config_drive	
created	2015-10-28T20:42:53Z
flavor	m1.tiny (1)
hostId	1f9d27fe57644512f82655d7339f19ae4c86a026c80f4db3a786ed39
id	53ad58a5-4fae-45f7-a03b-d82a64e3452f

image	cirros (8a724f78-2673-4ea1-b607-373626a15afb)	
key_name	-	
metadata	{}	
name	vt2	
os-extended-volumes:volumes_attached	[]	
progress	0	
security_groups	http-ssh	
status	ACTIVE	
tenant_id	41773a923b924ef7934e8aea532c0680	
updated	2015-10-28T20:43:59Z	
user_id	7198d60229d14888914d6e1ef5d0ef2b	
vault-tek-network network	10.10.0.3	

- Look at the VERY bottom of the output. It should say, "vault-tek-network network" followed by an IP address. It is likely something like 10.0.0.2, 10.0.0.3, 10.10.0.2, or 10.10.0.3. It doesn't matter what it is, just record it. We'll need it to SSH to this VM instance (vt2)

9. Navigate back to the CLI, time to SSH into the machine. In order to do this, we'll need to access that machine's namespace. In order to do this, we'll SSH to the neutron server. Type the following:

```
[root@controller] ~(keystone_chestercopperpot)# ssh root@neutron
[root@neutron ~]#
```

10. Fantastic. We are now in a Neutron node. If you're confused, click back to Lab 00, and check out the picture of the network. Remember that this is a dedicated node just for Neutron. Networking takes resources! Let's take a look at the namespaces the neutron node knows about. Type the following command:

```
[root@neutron ~]# ip netns list
qrouter-b1579da2-d5a4-40ec-b6a5-606413f1738e
qdhcp-cfe3722a-f584-4150-a6ae-1a178677ac78
qrouter-c9a2b225-bab6-4965-9c98-417d9a53ba3c
qdhcp-3288dfae-a262-42c2-92b2-1b6ae652fa5b
qdhcp-20ca30a2-e3fb-4e62-bd78-08dc471e93ed
```

- The above command is going to create some output. Look near the top of the output. We're looking for a match that begins **qrouter-**, but notice that there are many **qrouter-** entries. We want the one that is followed by the same characters you recorded back in step "2.12"
- Once you've found this value, highlight it with your mouse

10. Now we're going to tell neutron to access that router's namespace, and then SSH to the VM we just created. If it works, you'll be prompted to log into the VM instance you just created (vt2)

```
[root@neutron ~]# ip netns exec
<right_click_to_paste_the_highlighted_namespace> ssh cirros@<IP_address_of_vt2>
```

NOTE 1: The name space should include qrouter- portion in front of the ID
NOTE 2: The IP_address_of_vt2 was obtained in step 7 when you issued the command "nova show vt2" on the controller.

```
The authenticity of host '10.10.0.3 (10.10.0.3)' can't be established.  
RSA key fingerprint is 26:6b:23:9c:ab:bb:eb:b5:3a:18:b5:8e:72:ba:b4:4f.  
Are you sure you want to continue connecting (yes/no)? yes  
Warning: Permanently added '10.10.0.3' (RSA) to the list of known hosts.  
cirros@10.10.0.3's password: cubswin:)
```

11. There you go! If it worked, you'll be inside of the little VM you just started! This lab was heavy lifting, so if you don't understand something, ask the instructor.
12. There isn't much to see inside of a CirrOS VM, so once your curiosity is satisfied, type the following to return to root@controller

```
$ exit  
[root@neutron ~]# exit  
[root@controller ~(keystone_chestercopperpot)]#  
[root@controller ~(keystone_chestercopperpot)]# source .bashrc  
[root@neutron ~]#  
This will put you back to root. Done!
```

10 locate the various logs for services supporting OpenStack, thereby allowing admins to monitor their OpenStack deployment.

The second part of this lab is designed as a reference for how to build an rsyslog server for your OpenStack infrastructure. This lab has no dependency on other labs.

1. Where are the logs?

Most OpenStack services use the convention of writing their log files to sub directories of /var/log/

1. Display the nova-* service logs

```
[root@controller ~]# ls /var/log/nova  
nova-api.log      nova-conductor.log  nova-manage.log    nova-scheduler.log  
nova-cert.log     nova-consoleauth.log  nova-novncproxy.log
```

2. Display the glance-* service logs


```
[root@controller ~]# ls /var/log/glance
api.log  api.log-20151004.gz  api.log-20160113  registry.log  registry.log-20160111
```

3. Display the cinder-* service logs

```
[root@controller ~]# ls /var/log/cinder
api.log  api.log-20151004.gz  api.log-20160113  registry.log  registry.log-20160111
```

4. Display the keystone-* service logs

```
[root@controller ~]# ls /var/log/keystone
keystone.log  keystone.log-20151004.gz  keystone.log-20160111
```

5. Display the horizon service logs

```
[root@controller ~]# ls /var/log/horizon
horizon.log
```

6. SSH to compute node #1!

```
[root@controller ~]# ssh root@compute1
```

7. CD to the directory that contains the instances

```
[root@compute1 ~]# cd /var/lib/nova/instances
```

8. List the directory and note the instances

```
[root@compute1 instances]# ll
drwxr-xr-x. 2 nova nova 69 Oct 4 20:20 07334a4c-dac3-474b-83f0-1d8f4db1f093 <-- Instance#1
drwxr-xr-x. 2 nova nova 69 Nov 4 20:26 6a56b82f-547d-440b-9ebc-04813a343a2a <-- Instance#2
drwxr-xr-x. 2 nova nova 53 Nov 4 20:25 _base
-rw-r--r--. 1 nova nova 73 Nov 4 21:15 compute_nodes
drwxr-xr-x. 2 nova nova 91 Nov 4 20:25 locks
```

9. Change directory into Instance#1

```
[root@compute1 instances]# cd iiii where iiii = instance#1 shown above
```

10. List the files in this directory

```
[root@compute1 07334a4c-dac3-474b-83f0-1d8f4db1f093]# ll
-rw-rw----. 1 nova qemu 28356 Oct 3 00:03 console.log <--cat console.log | tail -20
-rw-r--r--. 1 qemu qemu 1703936 Oct 3 00:04 disk <-- BORING to look at, ephemeral storage
-rw-r--r--. 1 nova nova 79 Oct 2 23:24 disk.info <-- cat disk.info
-rw-r--r--. 1 nova nova 2715 Oct 2 23:27 libvirt.xml <--less libvirt.xml
```

11. Inspect the log files shown above. No special analysis required on your part, just check out the logs

Use the **cat** or **less** command to look at these files. Hints of what to use are shown above

2. Introduction to rsyslog (or remote logging) (Reference ONLY)

All systems generate and update log files recording their actions and any problems they encounter. In a distributed or cloud computing environment that contains many systems, and it may be unreasonable to expect admins to collect log files from these many locations. Remote logging is the solution to this problem.

The rsyslog service provides a centralized logging server, and configuration for individual systems to send their log files to the centralized logging server.

1. **WARNING WARNING WARNING** You won't actually do these steps, but it might be useful to see how they would be performed in a production system.
2. Firstly, install the rsyslog server on the system you want to use as the centralized logging server. All other nodes will be configured to send logs to this location, and they too will need rsyslog installed. We'll assume an installation onto a CentOS system (this is what you're using in class)
 - `# yum install rsyslog` <--Read Step 1 above, this is FYI
 - To be clear, you'd install rsyslog on the centralized logging server, as well as every node you wish to have report logging.
3. Allow SELinux to allow rsyslog traffic (centralized server only)
 - `# semanage -a -t syslogd_port_t -p udp 514`
4. Configure `/etc/sysconfig/iptables` to allow rsyslog traffic, then restart the service so the new rules take effect (centralized server only)
 - Note -- IP TABLES ACCEPT rules need to appear before REJECT rules
 - `# vim /etc/sysconfig/iptables`
 - `-A INPUT -m state --state NEW -m udp -p udp --dport 514 -j ACCEPT` <-- place this line inside of `/etc/sysconfig/iptables`
 - `# service iptables restart`
5. Configure `/etc/rsyslog.conf` to define where the logs will be saved to. Place the following snippet in `rsyslog.conf` (centralized server only)
 - `# vim /etc/sysconfig/iptables`

```
6. $template TmplAuth, "/var/log/%HOSTNAME%/%PROGRAMNAME%.log"
7.
8. authpriv.*    ?TmplAuth
9. *.info,mail.none,authpriv.none,cron.none    ?TmplMsg
```

10. Before you exit and save /etc/rsyslog.conf, uncomment the following lines

```
11. $ModLoad imudp
12. $UDPServerRun 514
```

13. The central server is now configured-- time to turn our attention to the individual nodes.

14. On each node (not the centralized server), edit the /etc/rsyslog.conf, and specify the address of your centralized logging server with the following:

- *.* @REPLACE_WITH_IP_ADDRESS_OF_SERVER:PORT_SERVER_LISTENING_ON
- The port used in the above statement would be 514 if you are following this procedure as it is written

15. Start rsyslog, and then confirm that rsyslog will always run on startup (do this on the centralized server AND all nodes)

- # service rsyslog start
- # chkconfig rsyslog on

11 *some exposure to using the Glance (Image) service.*

In this lab, a CirrOS image has already been downloaded and named. We will launch an instance with it.

We keep talking about CirrOS. If it hasn't been explained already, CirrOS is a minimal (CirrOS < 15Mb) Linux distro that was designed for testing cloud instances (such as those on an OpenStack compute node). Take a minute, and check out all of the CirrOS downloads available by navigating to: <https://download.cirros-cloud.net/>.

Be aware that if your deployment is using QEMU or KVM, OpenStack recommends that you use images using the qcow2 format. The most recent 64-bit qcow2 image is available at https://download.cirros-cloud.net/0.3.4/cirros-0.3.4-x86_64-disk.img.

1. Creating a new image with glance at the CLI

1. SSH to your controller and log in as root (you might use PuTTY for this)
2. Once logged into the controller, issue the following command:

```
[root@controller ~]# source keystonerc_admin
[root@controller ~(keystone_admin)]# glance image-list
[root@controller ~(keystone_admin)]# glance help image-create
```

- The location option is important to note. It does not copy the entire image into the Image service, but references an original location where the image can be found. Upon launching an instance of that image, the Image service accesses the image from the location specified.

```
[root@controller ~(keystone_admin)]# glance image-create --name cirros_alpha -
-disk-format qcow2 --container-format bare --is-public true --location
https://download.cirros-cloud.net/0.3.4/cirros-0.3.4-x86_64-disk.img
[root@controller ~(keystone_admin)]# glance image-list
```

3. Launch an instance from the command line with the new instance cirros_alpha:

```
[root@controller ~(keystone_admin)]# neutron net-list
```

- Highlight the id of the private network

```
[root@controller ~(keystone_admin)]# nova boot --flavor custom.shrimpy --image
cirros_alpha --nic net-id=<private_network_id> admin_box
```

4. Log into the Horizon dashboard as admin // amel
5. Navigate to **Project > Compute > Instances**
 - Confirm that the VM instance admin_box is running
 - While you're here, you may launch another VM with Horizon. Call it what ever you'd like, but be sure to launch it with the new image we created (cirros_alpha). On the Instance Boot Source, there are several options, but two are interesting to us, namely **Boot from image** and **Boot from image (create new volume)**. Use the second. Create a new volume will cause nova to order cinder to create a new volume and install the bootbale image there.
6. That's it for this lab! Hang tight, or check out the Additional Learning / References listed below while everyone else is wrapping up

Additional Learning / References

The following are a list of pages we thought might be helpful for our students to know about:

- [Glance OpenStack project](#)
- [Glance bugs](#)

12 create a block storage volume with Cinder and then attach it to a VM instance.

This is the virtual equivalent of plugging a new drive into a computer. Just like a physical drive, volumes can be attached, detached, reattached, and moved between VM instances.

1. Creating a block storage volume with Horizon

1. Log into Horizon as chestercopperpot // fa5tpa55w0rd
2. Navigate to **Project > Compute > Volumes**
3. Click on the button **+Create Volume** (Look in the upper right corner)
4. Fill out the pop-up window as follows:

```
Name: cargo-bay
Description: new volume for vt2
Volume Source: No source, empty volume
Type: iscsi
Size (GB): 1
Availability Zone: Any Availability Zone
```

- Click the button
5. Let's attach this volume to our VM instance (vt2)
6. Under **Actions** click the drop down menu, and then select **Manage Attachments**
7. In the pop-up window, select (vt2) from the drop down menu

8. Click the **Attach Volume** button

The volume will now be attached and mounted by our instance (vt2). It will take 20 to 30 seconds to attach.

- Answer the following questions:
- Is the volume being actively used by the instance (is it in-use)?
- Where is the volume attached on vt2?
- Is this a bootable volume?
- Could you detach this volume from instance vt2, and attach it to a different instance? Why might you want to do this?

2. Create a Cinder block storage volume using the CLI

1. We'll replicate the same steps at the CLI that we just performed in Horizon. If you get the idea, and not particularly strong at the CLI, it is alright to skip this section.
2. Issue the following commands

```
[root@controller ~]# source keystonerc_chestercopperpot
[root@controller ~(keystone_chestercopperpot)]# cinder create --display_name
NASferatu 1
```

- The above command says, "Cinder, I want you to create a volume named NASferatu that is 1 GB in size"
3. Based on the output, answer the following questions:
 - Is this a bootable volume?
 - Is this volume encrypted?
 - What is the status?
 4. Issue the following command to retrieve a list of cinder volumes available to the current users chestercopperpot:

```
[root@controller ~(keystone_chestercopperpot)]# cinder list
```

- Note the ID of the volume for NASferatu

- Has the status changed? What is it now?

5. Issue the following command to retrieve a list of the instances currently in operation

```
[root@controller ~(keystone_chestercopperpot)]# nova list
```

- Note the ID of the instance vt2

6. Issue the following command to attach the new volume (NASferatu) to a VM instance (vt2)

```
[root@controller ~(keystone_chestercopperpot)]# nova volume-attach  
<replace_with_ID_of_instance_vt2> <replace_with_ID_of_volume_NASferatu> auto
```

- The above commands says attached NASferatu to the instance vt2, and auto assign it (alternatively you could specify /dev/vdb, /dev/vdc/, etc.)
- The output should look similar to the following:

Property	Value
device	/dev/vdc
id	d43ff424-684e-4aa4-953d-eb119e0c6a0d
serverId	5eb02b66-0e74-429a-a4d1-6c70fe18d178
volumeId	d43ff424-684e-4aa4-953d-eb119e0c6a0d

7. Check on the status of the volume (NASferatu)

```
[root@controller ~(keystone_chestercopperpot)]# cinder list
```

- What is the status of NASferatu?

8. Issue the following command to detach the volume from vt2

```
[root@controller ~(keystone_chestercopperpot)]# nova volume-detach  
<replace_with_ID_of_instance_vt2> <replace_with_ID_of_volume_NASferatu>
```

9. Check on the status of the volume (NASferatu)

```
[root@controller ~(keystone_admin)]# cinder list
```

- What is the status of NASferatu?

10. Now that the volume is detached, let's remove it

```
[root@controller ~(keystone_admin)]# cinder delete NASferatu
```

11. Source keystone_admin and then confirm that the volume has been deleted

```
[root@controller ~(keystone_chestercopperpot)]# source keystone_admin  
[root@controller ~(keystone_admin)]# cinder list
```

- What is the state of NASferatu?
- If it hasn't yet deleted, continue to issue the command until it is removed.

3. Moving volumes (and data) between instances - Rocket Scientists Only

This section **ONLY** for students that finish early and are looking for a challenge. Try this out...

Here is a great [reference](#)

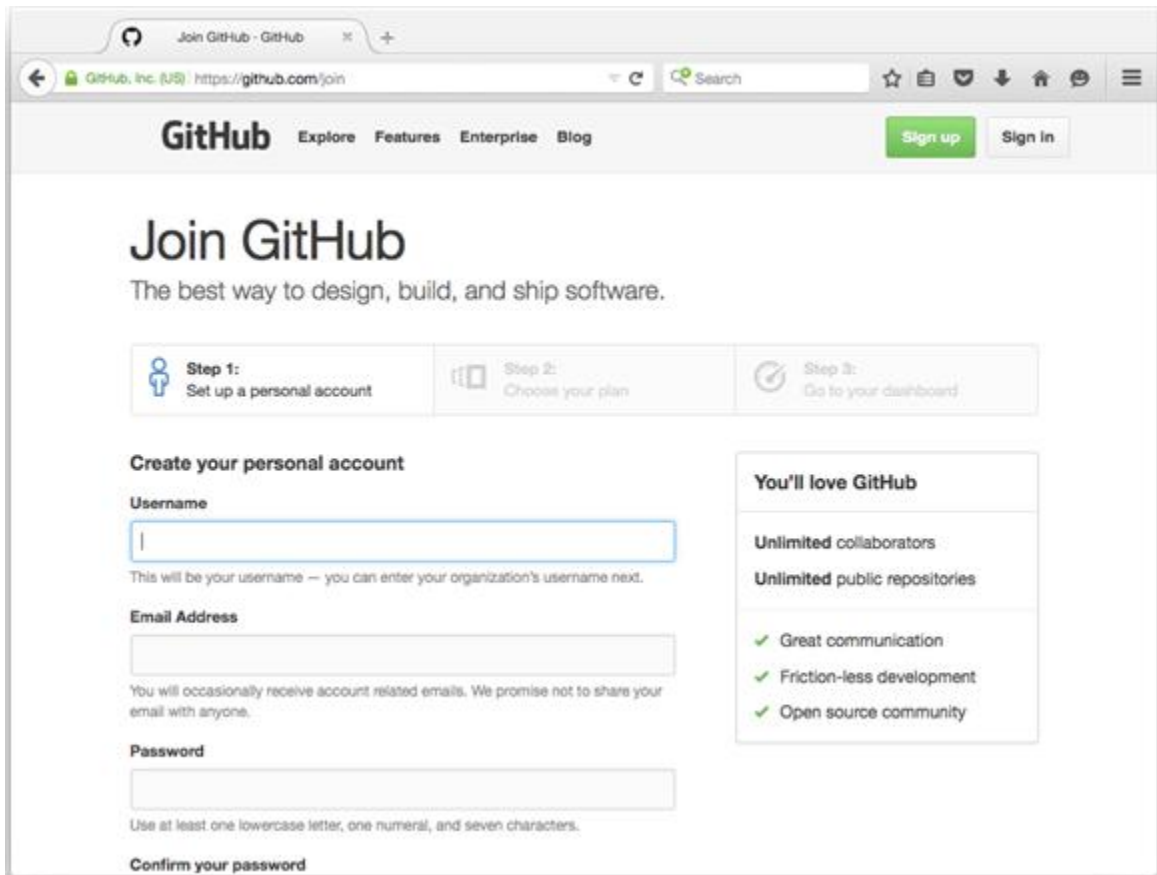
1. Create a volume called cargo-bay
2. SSH into vt2
3. Access the Volume from the running instance, instructions [here] (https://access.redhat.com/documentation/en-US/Red_Hat_Enterprise_Linux_OpenStack_Platform/2/html/Getting_Started_Guide/ch16s03.html)
4. Create a text-file on the volume cargo-bay (Hello World! is fine)
5. Log out of the instance and then disconnect the volume from vt2
6. Start a new instance (vt3... whatever you want). Use the same security-group (http-ssh) and network as vt2
7. Attach the volume cargo-bay to the new instance vt3
8. SSH into vt3
9. Look at the new volume on vt3 and confirm it contains the file (Hello World!) that you created in step 2

13 introduction to git and git pull requests..

Many open source projects rely on git for decentralized collaboration including OpenStack.

1. Setup Github Account

1. In a new tab, [Create a Github account](#) or [Login](#)



The screenshot shows the GitHub 'Join GitHub' page. The header includes the GitHub logo, navigation links (Explore, Features, Enterprise, Blog), and 'Sign up' and 'Sign in' buttons. The main heading is 'Join GitHub' with the tagline 'The best way to design, build, and ship software.' Below this is a three-step process: Step 1: Set up a personal account (active), Step 2: Choose your plan, and Step 3: Go to your dashboard. The 'Create your personal account' section contains four input fields: Username, Email Address, Password, and Confirm your password. A note under the Username field states: 'This will be your username — you can enter your organization's username next.' A note under the Email Address field states: 'You will occasionally receive account related emails. We promise not to share your email with anyone.' A note under the Password field states: 'Use at least one lowercase letter, one numeral, and seven characters.' To the right of the form is a sidebar titled 'You'll love GitHub' listing benefits: 'Unlimited collaborators', 'Unlimited public repositories', 'Great communication', 'Friction-less development', and 'Open source community'.

2. Navigate to the Amel OpenStack Glossary project

<https://github.com/amel/openstack-labs>

3. Star the repository

Staring is like a bookmark on github.com, you can view and search your stared repositories at github.com/stars

2. Fork the project

1. **Fork the lab repository** into your account

3. Edit a file in the repository

1. **Navigate to (click on) a file** you would like to edit. The this example will add a new glossary term to our glossary.
2. The file contents are summarized on this page. Additionally we can perform operations that give us insights about this specific file (i.e. length, size, authors, and change history). **Click the edit file icon** to make a new change.
3. **Make a change**, in this example we add a fictitious new OpenStack Service "Aardvark"
4. After you are finished editing, ****scroll to the bottom and add a descriptive commit message and click "Commit Changes"**

4. Create a Pull Request

1. **Navigate back to the project base page.** Notice that our branch is now ahead of the master branch. Let's create a pull request to get our addition added back into the main repository. Click the "Pull Request" link.
2. On the pull request page, **first scroll down to see the list of changes in this request.** It should only be our added Aardvark service highlighted in green. If we had removed lines they would be highlighted in red.
3. **Create the pull request**
4. **this point your instructor will accept the changes.** Below is a screenshot of the email that is generated to a project maintainer when a pull request is submitted. Refresh your page a few times until your instructor accepts your changes.

14 Ansible.

You will learn the syntax and mechanisms for one-off commands, inventories to reach multiple hosts, and playbooks to accomplish complex and multi-part tasks.

An appropriate ssh environment amiable to ansible has been setup for you inside this lab environment. This is usually a non-trivial amount of work which involves setting up passwordless sudoers and distributing public keys.

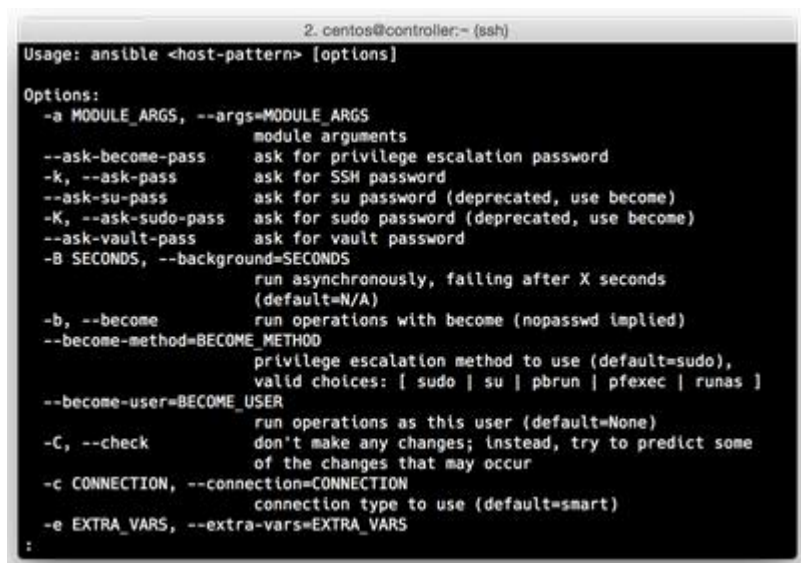
The final portion of this lab will focus on using ansible to fix a specific OpenStack configuration issue.

1. Basic Ansible Commands

1. SSH to your controller via <https://amel.com/sshproxy> (ask us for the log in / password if you forget)
 - Type labs followed by your name to connect to root@controller
2. De-elevate your privileges to the normal user centos

```
[root@controller ~]# su centos
[centos@controller ~]# cd
```
3. Ensure ansible is installed

```
[centos@controller ~]$ sudo yum install ansible
[centos@controller ~]$ ansible -h | less
```



```
2. centos@controller:~ (ssh)
Usage: ansible <host-pattern> [options]

Options:
  -a MODULE_ARGS, --args=MODULE_ARGS      module arguments
  --ask-become-pass                        ask for privilege escalation password
  -k, --ask-pass                          ask for SSH password
  --ask-su-pass                          ask for su password (deprecated, use become)
  -K, --ask-sudo-pass                    ask for sudo password (deprecated, use become)
  --ask-vault-pass                      ask for vault password
  -B SECONDS, --background=SECONDS        run asynchronously, falling after X seconds
                                           (default=N/A)
  -b, --become                          run operations with become (nopasswd implied)
  --become-method=BECOME_METHOD          privilege escalation method to use (default=sudo),
                                           valid choices: [ sudo | su | pbrun | pfexec | runas ]
  --become-user=BECOME_USER              run operations as this user (default=None)
  -C, --check                          don't make any changes; instead, try to predict some
                                           of the changes that may occur
  -c CONNECTION, --connection=CONNECTION connection type to use (default=smart)
  -e EXTRA_VARS, --extra-vars=EXTRA_VARS
```

It's always useful to check out the help file. Take a second and look for some key words which should be familiar from the lecture.

4. Create a simple hosts file with the following content:

```
[centos@controller ~]$ nano hosts
192.168.0.10
```

- Press CTRL + O to Write Out, ENTER, then CTRL + X to exit

5. Use the ping ansible module to test connectivity to the controller

```
[centos@controller ~]$ ansible all -i hosts -m ping
192.168.0.10 | success >> {
|
|   "changed": false,
|
|   "ping": "pong"
|
| }
```

The above text will be GREEN.

Refer back to the help command and read what each of these flags represent.

6. Gather Facts about controller with ansible

```
[centos@controller ~]$ ansible all -i hosts -m setup | less
```

Skim through these facts and look for interesting values. How many processor cores does this node have? Which Linux kernel? Which release of CentOS is it running? All of these variables are available to your playbooks, these can be very useful for eliminating many repetitive information gathering tasks in your environments!

7. Okay, that was cool. Now lets replace what is in our hosts file with the following content. Type the following command:

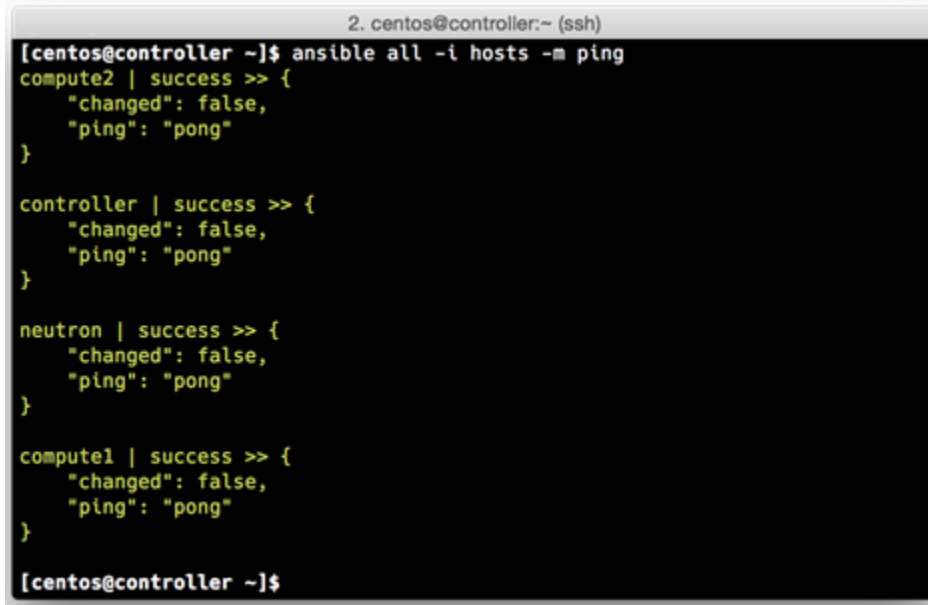
```
[centos@controller ~]$ nano hosts
[openstack]
controller ansible_ssh_host=192.168.0.10
neutron     ansible_ssh_host=192.168.0.11
compute1    ansible_ssh_host=192.168.0.12
compute2    ansible_ssh_host=192.168.0.13
```

- Press CTRL + O to Write Out, ENTER, then CTRL + X to exit

8. Ping all your hosts

```
[centos@controller ~]$ ansible all -i hosts -m ping
```

The **"all"** pattern targets all hosts in the inventory **-i** specifies a certain inventory file (a listing of hosts) **"hosts"** is the traditional inventory filename. **-m** specifies a certain module, in this case **"ping"**



```
2. centos@controller:~ (ssh)
[centos@controller ~]$ ansible all -i hosts -m ping
compute2 | success >> {
  "changed": false,
  "ping": "pong"
}

controller | success >> {
  "changed": false,
  "ping": "pong"
}

neutron | success >> {
  "changed": false,
  "ping": "pong"
}

compute1 | success >> {
  "changed": false,
  "ping": "pong"
}

[centos@controller ~]$
```

9. Try different host-patterns and modules

```
[centos@controller ~]$ ansible openstack -i hosts -m command -a 'hostname'
[centos@controller ~]$ ansible controller -i hosts -m command -a 'whoami'
[centos@controller ~]$ ansible openstack -i hosts -m command -a '/usr/sbin/ip
addr'
```

Read about modules [HERE]

(http://docs.ansible.com/ansible/list_of_all_modules.html) Check out the custom Open Stack modules while you are at this link!

10. Red ink - this command will fail

```
[centos@controller ~]$ ansible openstack -i hosts -m yum -a 'name=htop
state=installed'
```

NOTE: This step may take a moment to complete Read the help in order to find the argument to run this command with `sudo`, turn this red ink to green

11. Gather some information about our compute nodes

```
[centos@controller ~]$ ansible compute1 -i hosts -m setup | grep vcpus
[centos@controller ~]$ ansible compute2 -i hosts -m setup | grep vcpus
```

At this point you now know how to run ansible one-off commands. These can be very useful for making changes and gathering information from your systems quickly and

efficiently from a central location. Next we will use an ansible playbook to accomplish an ordered list of tasks.

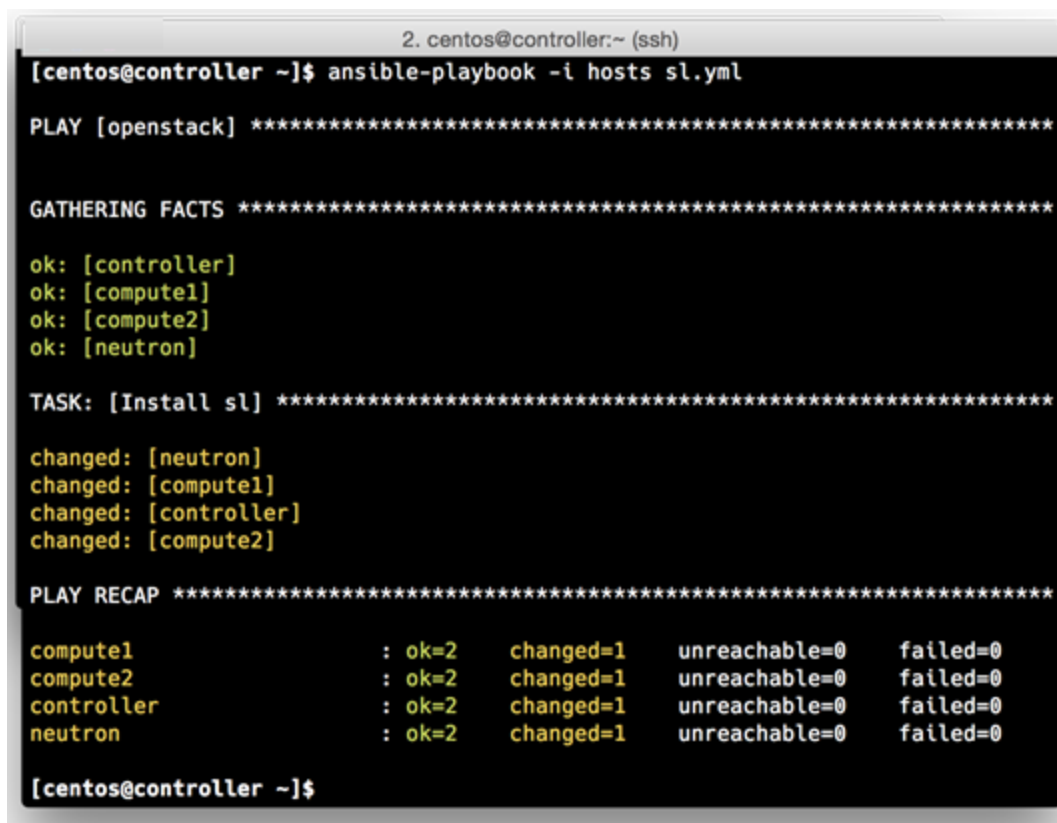
2. Ansible Playbooks

Ansible playbooks are an efficient way to organize a list of ordered tasks. These tasks are grouped into a play which specify which hosts will be configured. In this section we will make a simple playbook that installs a new program to our controller.

1. Download the `sl.yml` playbook, inspect it, and run it

```
[centos@controller ~]$ curl https://ame1.com/labs/files/sl.yml
[centos@controller ~]$ curl https://ame1.com/labs/files/sl.yml > sl.yml
[centos@controller ~]$ less sl.yml
[centos@controller ~]$ ansible-playbook -i hosts sl.yml
```

Note that we did not need to enumerate the host-pattern in this command! My fingers feel less tired already!



```
2. centos@controller:~ (ssh)
[centos@controller ~]$ ansible-playbook -i hosts sl.yml

PLAY [openstack] *****

GATHERING FACTS *****

ok: [controller]
ok: [compute1]
ok: [compute2]
ok: [neutron]

TASK: [Install sl] *****

changed: [neutron]
changed: [compute1]
changed: [controller]
changed: [compute2]

PLAY RECAP *****

compute1      : ok=2    changed=1    unreachable=0    failed=0
compute2      : ok=2    changed=1    unreachable=0    failed=0
controller    : ok=2    changed=1    unreachable=0    failed=0
neutron       : ok=2    changed=1    unreachable=0    failed=0

[centos@controller ~]$
```

2. Run it again and see how the PLAY RECAP is different
`[centos@controller ~]$ ansible-playbook -i hosts sl.yml`
3. Use the new application (don't forget to read the man page!)

```
[centos@controller ~]$ s1
[centos@controller ~]$ man s1
```

3. Fix Nova VNC Service

First we will see what is broken. There is a configuration issue with Nova VNC Proxy service. This service allows a user to view and interact with an instance console from the web browser, a super useful feature! In order to configure this service correctly the controller and the compute nodes need to know the external ip address of the controller in order to correctly direct the browser to the vnc proxy resource. Because your lab environment was stood up *just for you*, your external ip address is not predictable. As a result this service is incorrectly configured and pointing to an incorrect public ip address.

In this section we will read and run a playbook that fixes this issue.

1. Clone the ansible-novnc repository

```
[centos@controller ~]$ git clone https://github.com/bryfry/ansible-novnc.git
[centos@controller ~]$ cd ansible-novnc
```
2. Read the novnc.yml and configuration template file

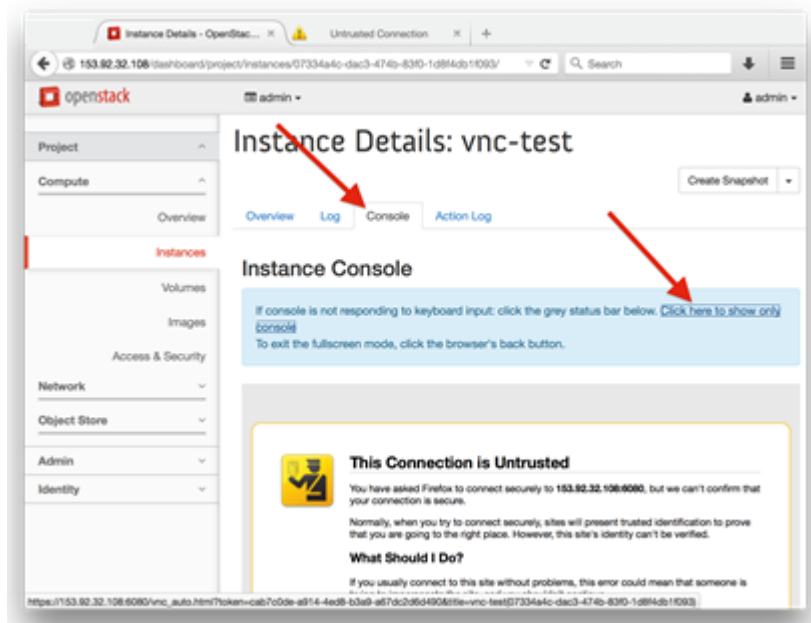
```
[centos@controller ansible-novnc]$ less novnc.yml
[centos@controller ansible-novnc]$ less files/nova-controller.conf.j2
[centos@controller ansible-novnc]$ less files/nova-compute.conf.j2
```
3. We actually care about the vnc related parameters, let's filter on that

```
[centos@controller ansible-novnc]$ grep vnc files/nova-controller.conf.j2
[centos@controller ansible-novnc]$ grep vnc files/nova-compute.conf.j2
```
4. Run the novnc.yml playbook

```
[centos@controller ~]$ ansible-playbook -i hosts novnc.yml
```
5. Match up each PLAY and TASK with the corresponding portion of the playbook below

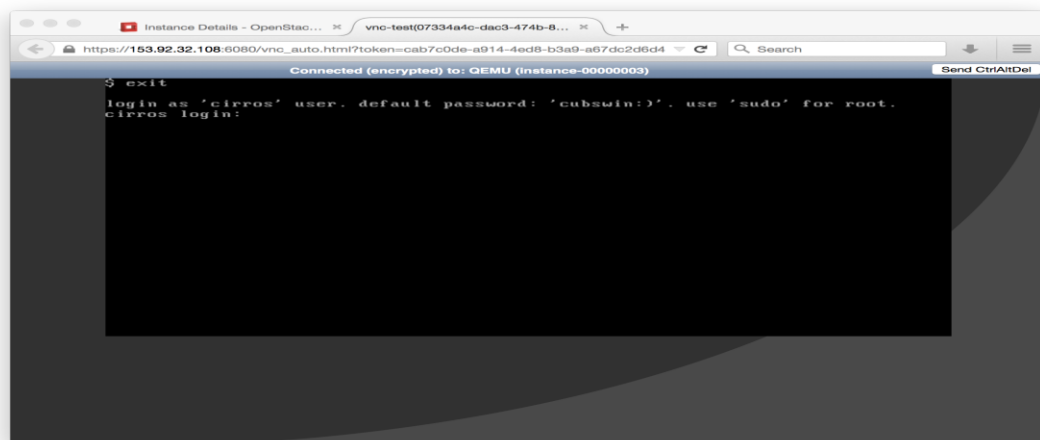
```
2. vim
1 ---
2 - hosts: controller
3   remote_user: root
4   vars:
5   tasks:
6   - name: Record public IP
7     shell: curl http://ipecho.net/plain;
8     register: public_ip
9
10  - name: Deploy Nova Compute Template
11    template: src=files/nova-controller.conf.j2 dest=/etc/nova/nova.conf
12
13  - name: Restart httpd
14    service: name=httpd state=restarted
15
16 - hosts:
17   - compute1
18   - compute2
19   remote_user: root
20   vars:
21     controller_public_ip: "{{ hostvars['controller'].public_ip.stdout }}"
22     compute_ip: "{{ ansible_default_ipv4.address }}"
23   tasks:
24
25   - name: Deploy Nova Compute Template
26     template: src=files/nova-compute.conf.j2 dest=/etc/nova/nova.conf
27
28   - name: Restart Nova Compute Service
29     service: name=openstack-nova-compute state=restarted
30
31 - hosts: controller
32   remote_user: root
33   tasks:
34   - name: Restart Nova Compute and VNC Proxy Service
35     service: name=openstack-nova-novncproxy state=restarted
```

6. Let's go use the vnc service! Open the OpenStack Horizon interface (log in as admin / amel)
7. Launch an instance or navigate to a launched instance
8. Click the console tab
9. Open the console in a separate window

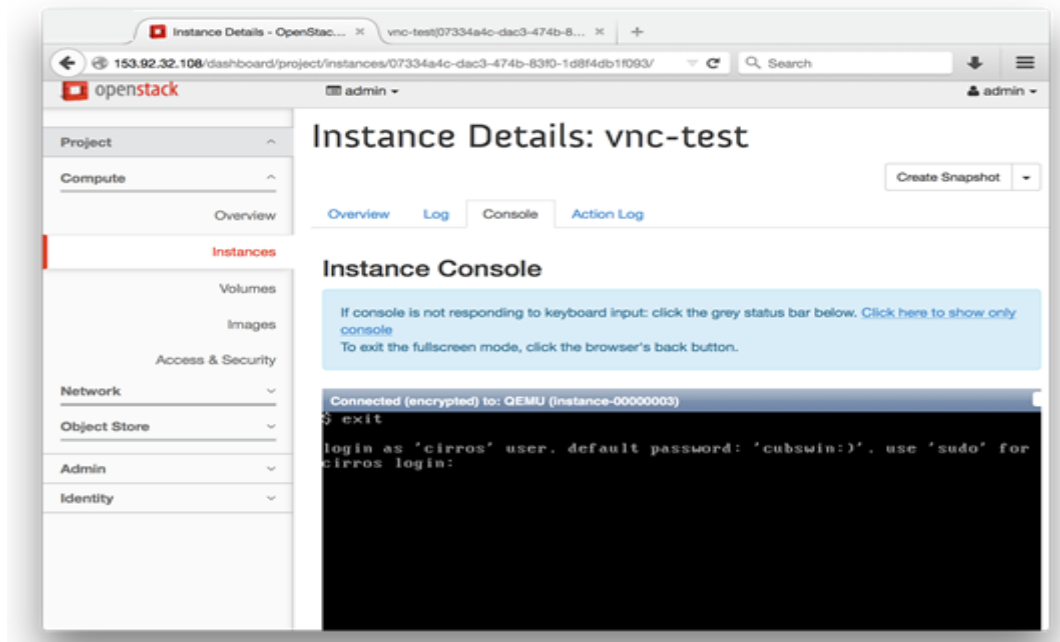


10. Accept the privacy exception, we're using a self signed certificate

11. View and interact with the console



12. Refresh the page the console tab page, it should also work now



This is only a very small subset of the use cases for ansible. Like any tool you can customize and utilize it to fit your environments needs. Especially recommended is to use ansible in combination with a revision control system (like git). This allows for a succinct and manageable orchestration of your environments throughout time. The sky is definitely the limit!

🚧 15 OpenStack skills. If you are successful,

Notice below that you will need access to a fairly powerful machine. Most people want to create their own OpenStack lab inside of a virtual machine that shows off all the subsystems. We think the AIO (All In One) from <https://github.com/openstack/openstack-ansible> which is based on linux containers is a great choice for this environment. If you get this working, it will have far more utility than DevStack because the playbooks you will use are the same ones you would edit to build a complete datacenter.

Here is a good reference: <http://docs.openstack.org/developer/openstack-ansible/developer-docs/quickstart-aio.html?highlight=aio>

Create an Ubuntu Server 14.04 LTS machine using my specifications if you want to exactly match the steps below:

Using ESXI server 5.5, 6.0 may work even better, not tested (Yeah, we need an AWS version, but this is really cheap for me since I have this already)

16 GB RAM (This may not be enough)
200 Gig storage
8 CPUs
Login: ender
Password: endersgame
Hostname: openstack
openstack's IP Address: 10.20.30.40
Using ESXI 5.5 (or 6.0)
Virtual machine named: openstack
Start step 1 with the ubuntu 14.04 VM installed but turned OFF.

BEGIN

1. **VM is turned off, right?** SSH into your ESXI hypervisor

2. `ssh root@xxx` # where xxx is the IP address of your ESXI server, assuming user is root

3. Edit the "openstack" vmx file, where xxxx is the name of the volume to enable nested virtualization.

4. `vi /vmfs/volumes/xxxxxxx/openstack.vmx``

5. Add this line to the bottom of the vmx file

6. `vhv.enable = "TRUE"`

7. Save, log out of ESXI SSH session and reboot the VM.

8. Turn on the openstack VM

9. SSH back into your new openstack VM and continue...

`screen sudo apt-get update sudo apt-get upgrade sudo apt-get install git ```

7. Add static IP address, see next step for the config

8. `sudo vim /etc/network/interfaces`

9. Here is a **sample** config for the above step

`auto eth0 iface eth0 inet static address 192.168.xxx.xxx netmask 255.255.255.0 gateway 192.168.xxx.1 dns-search amel.com dns-nameservers 192.168.yyy.zzz ```

9. sudo reboot

10. Gen keys

```
11. ssh-keygen -t rsa -C "ender@xxxx"
```

12. Containers DEMAND at least 3.13.0-34 kernel so go get it.

```
13. sudo apt-get install linux-image-3.13.0-34-generic
```

14. Reboot to start the new kernel

sudo reboot ``

13. **SNAPSHOT TIME!!** Shut the openstack VM down and take a snapshot. You will return to this step MANY times until you get this working.

14. Go get openstack/openstack-ansible

sudo apt-get install git sudo git clone <https://github.com/stackforge/os-ansible-deployment> /opt/os-ansible-deployment cd /opt/os-ansible-deployment sudo git checkout kilo screen ``

15. **SNAPSHOT TIME again!!** OK to take *ANOTHER* snapshot if you trust the previous step. This will shorten your testing cycle when you return to this point for the 50th time :(. Don't give up!

16. Create your all in one (AIO) system with a single line of code.

```
17. sudo ./scripts/run-aio-build.sh`
```

18. Fix all problems, and please let us know what you learn. We will add your fixes to this lab. We will post more bulletproof updates to this lab as we hone the steps. Return to the snapshot and try again. Don't give up, this journey will be tremendously educational.

 **16 how to take an Ubuntu 14.04 (LTS) machine and stand-up an OpenStack environment so that you may continue your exploration of OpenStack on your own time outside of class.**

You might use an old laptop, AWS, Google Cloud, or even try it out on a VMWare VM on your desktop. The only prerequisite, is that you have a new install of Ubuntu 14.04 on the machine.

This lab doesn't actually install OpenStack, it installs DevStack. As the name implies, DevStack is for developers. It is designed to install on a single or dual-node system (the steps listed are for a single node system). Rather than do this lab in class, the instructor will demo it for you.

0. Instructions for setting up on Oracle Virtual Box

Tested with [Oracle VirtualBox 5.0.6 for Windows](#)

- The hardware set used was 4 gigs RAM, 1 NIC, 2 cores, and a 20 gig drive
- If the NIC is set to, then you may access Horizon via a browser on your host OS

NOTE: Running into a small bug. It does not seem able to access to MySQL backend, and is throwing errors. However, the Openstack CLI seems to work fine... test with the CLI for now, and check back for a Horizon solution soon.

Ubuntu 14.04 Desktop (or Server) is available [here](#)

- If you use Ubuntu 14.04 Desktop, then you can use the browser in Ubuntu to access Horizon. If you use the Server distro, you'll need to configure HTTP access and access Horizon from your host OS
- In all cases, default settings were chosen for installation

1. Prepare Ubuntu 14.04 for DevStack deployment

1. Issue the following commands to the Ubuntu CLI:

```
[ubuntu@PC01 ~]$ sudo apt-get update
[ubuntu@PC01 ~]$ ip a
Record the IP address of your VM. Mine was 10.0.2.15
[ubuntu@PC01 ~]$ sudo apt-get install ntp
[ubuntu@PC01 ~]$ cat /etc/ntp.conf
```

- You will see lots of lines, but the NTP servers being used are the following:

```
server 0.ubuntu.pool.ntp.org
server 1.ubuntu.pool.ntp.org
server 2.ubuntu.pool.ntp.org
server 3.ubuntu.pool.ntp.org
[ubuntu@PC01 ~]$ sudo timedatectl set-timezone UTC
[ubuntu@PC01 ~]$ sudo /etc/init.d/ntp restart
[ubuntu@PC01 ~]$ timedatectl
[ubuntu@PC01 ~]$ sudo apt-get install git -y
```

```

[ubuntu@PC01 ~]$ pwd
[ubuntu@PC01 ~]$ git clone https://git.openstack.org/openstack-dev/devstack
[ubuntu@PC01 ~]$ ls
[ubuntu@PC01 ~]$ cd devstack
[ubuntu@PC01 ~]$ less stack.sh
Press q to quit
[ubuntu@PC01 ~]$ less unstack.sh
Press q to quit
[ubuntu@PC01 ~]$ less clean.sh
Press q to quit
[ubuntu@PC01 ~]$ less samples/local.conf
Press q to quit
[ubuntu@PC01 ~]$ ./run_tests.sh
Confirm that all of the tests pass

```

2. Configure stack.sh via local.conf

Now to configure stack.sh. DevStack includes a sample in `~/devstack/samples/local.conf`. The local.conf has the following parameters that need to be defined:

- Set `FLOATING_RANGE` - A range not used on the local network, i.e. 192.168.1.224/27. This configures IP addresses ending in 225-254 to be used as floating IPs.
- Set `FIXED_RANGE` and `FIXED_NETWORK_SIZE` to configure the internal address space used by the instances.
- Set `FLAT_INTERFACE` to the Ethernet interface that connects the host to your local network. This is the interface that should be configured with the static IP address mentioned above.
- Set the administrative password. This password is used for the admin and demo accounts set up as OpenStack users.
- Set the MySQL administrative password. The default here is a random hex string which is inconvenient if you need to look at the database directly for anything.
- Set the RabbitMQ password.
- Set the service password. This is used by the OpenStack services (Nova, Glance, etc) to authenticate with Keystone.

1. Copy the setup file into `~/devstack`, then explore what it contains. It is okay to leave this file as it. When the installation finishes, it will announce your sign in / password

```

ubuntu@PC01:~/devstack$ cp samples/local.conf ~/devstack/local.conf
ubuntu@PC01:~/devstack$ cat local.conf

```

2. `ubuntu@PC01:~/devstack$ script -a ~/stack.out`
3. `ubuntu@PC01:~/devstack$ ./stack.sh`

Note the current time. The installation of DevStack will take a bit; we've clocked it around 28 minutes. This is a good time to review the stack.sh documentation (<http://docs.openstack.org/developer/devstack/stack.sh.html>), as well as documentation on the DevStack project (<http://docs.openstack.org/developer/devstack/>). Be sure to keep checking the status of the installation. If something fails, it is important to catch it as quickly as possible.

```
ubuntu@PC01:~/devstack$ exit
ubuntu@PC01:~/devstack$ less ~/stack.out
ubuntu@PC01:~/devstack$ ls -l /opt/stack/logs
ubuntu@PC01:~/devstack$ cat ~/devstack/openrc
ubuntu@PC01:~/devstack$ source openrc admin admin
less /opt/stack/logs/stack.sh.log ## this doesn't work
diff ~/stack.out /opt/stack/logs/stack.sh.log
```

3. Take control of DevStack with admin

This section will log you get you logged into Horizon, and give you some direction, but it's up to you to explore and continue your OpenStack education. The installation of DevStack is quick and painless, so don't be afraid that you might 'break' something! Refer to previous labs if you get stuck.

1. Log into Horizon (user: admin // password: nomoresecrete)
 - <http://<IP address of Ubuntu VM>/dashboard>
NOTE: At this time, you can sign into Horizon, but it is throwing errors, however, the CLI seems to work fine. Check back soon for a Horizon fix.

2. To work from the CLI, type the following:

```
ubuntu@PC01:~/devstack$ source openrc admin
```

- This will not change your command prompt like it did in class, however you are now sourced as admin
3. Launch the python-OpenStack wrapper (easy place to start from)... think of this platform as the CLI version of Horizon. You can issue a common set of commands from here

```
ubuntu@PC01:~/devstack$ openstack
(openstack) help
```

- Help will return the common set of commands you can issue to OpenStack openstacks
4. Use Horizon and the CLI to complete the following tasks (to access the CLI, use PuTTY [Windows] or Terminal [Apple])

- Horizon tasks
 - Create the project **timelords**
 - Add the user **thegooddoctor** to the orion project
 - Add the user **amypond** to the orion project
 - Create a private network **gallifrey**
 - Create a router called **timelord** on the gallifrey network
 - Create a security group called **http-ssh** allowing HTTP and SSH access
 - Start an instance named **tardis** in the orion project
 - Create a **1 GB volume named vault** and attach it to tardis

- CLI tasks
 - **SSH** into the instance tardis and confirm you can see your new 1 GB volume
 - Create the project **camelot**
 - Add the user **kingarthur** to the camelot project
 - Add the user **sirrobin** to the camelot project
 - Start an instance in the camelot project

5. Enjoy! Don't be afraid to explore. If you come up with any 'cool' tasks, be sure to let us know!

**** THANKS AGAIN FOR TRAINING WITH AMEL RESEARCH!!!****

Nova Command.

subcommand

absolute-limits Print a list of absolute limits for a user.

add-fixed-ip Add new IP address on a network to server.

add-floating-ip DEPRECATED, use floating-ip-associate instead.

add-secgroup Add a Security Group to a server.

agent-create Create new agent build.

agent-delete Delete existing agent build.

agent-list List all builds.

agent-modify Modify existing agent build.

aggregate-add-host Add the host to the specified aggregate.

aggregate-create Create a new aggregate with the specified details.

aggregate-delete Delete the aggregate.

aggregate-details Show details of the specified aggregate.

aggregate-list Print a list of all aggregates.

aggregate-remove-host Remove the specified host from the specified aggregate.

aggregate-set-metadata Update the metadata associated with the aggregate.

aggregate-update Update the aggregate's name and optionally availability zone.

availability-zone-list List all the availability zones.

backup Backup a server by creating a 'backup' type snapshot.

boot Boot a new server.

clear-password Clear the admin password for a server.

cloudpipe-configure Update the VPN IP/port of a cloudpipe instance.

cloudpipe-create Create a cloudpipe instance for the given project.

cloudpipe-list Print a list of all cloudpipe instances.

console-log Get console log output of a server.

credentials Show user credentials returned from auth.

delete Immediately shut down and delete specified server(s).

diagnostics Retrieve server diagnostics.

dns-create Create a DNS entry for domain, name and IP.

dns-create-private-domain Create the specified DNS domain.

dns-create-public-domain Create the specified DNS domain.

dns-delete Delete the specified DNS entry.

dns-delete-domain Delete the specified DNS domain.

dns-domains Print a list of available dns domains.

dns-list List current DNS entries for domain and IP or domain and name.

endpoints Discover endpoints that get returned from the authenticate services.

evacuate Evacuate server from failed host.

fixed-ip-get Retrieve info on a fixed IP.

fixed-ip-reserve Reserve a fixed IP.

fixed-ip-unreserve Unreserve a fixed IP.

flavor-access-add Add flavor access for the given tenant.

flavor-access-list Print access information about the given flavor.

flavor-access-remove Remove flavor access for the given tenant.

flavor-create Create a new flavor.

flavor-delete Delete a specific flavor.

flavor-key Set or unset extra_spec for a flavor.

flavor-list Print a list of available 'flavors' (sizes of servers).

flavor-show Show details about the given flavor.

floating-ip-associate Associate a floating IP address to a server

floating-ip-bulk-create Bulk create floating IPs by range.

floating-ip-bulk-delete Bulk delete floating IPs by range.

floating-ip-bulk-list List all floating IPs.

floating-ip-create Allocate a floating IP for the current tenant.

floating-ip-delete De-allocate a floating IP.

floating-ip-disassociate Disassociate a floating IP address from a server.
floating-ip-list List floating IPs.
floating-ip-pool-list List all floating IP pools.

get-password Get the admin password for a server.
get-rdp-console Get a rdp console to a server.
get-serial-console Get a serial console to a server.
get-spice-console Get a spice console to a server.
get-vnc-console Get a vnc console to a server.

host-action Perform a power action on a host.
host-describe Describe a specific host.
host-list List all hosts by service.
host-update Update host settings.

hypervisor-list List hypervisors.
hypervisor-servers List servers belonging to specific hypervisors.
hypervisor-show Display the details of the specified hypervisor.
hypervisor-stats Get hypervisor statistics over all compute nodes.
hypervisor-uptime Display the uptime of the specified hypervisor.

image-create Create a new image by taking a snapshot of a running server.
image-delete Delete specified image(s).
image-list Print a list of available images to boot from.
image-meta Set or Delete metadata on an image.
image-show Show details about the given image.

interface-attach Attach a network interface to a server.
interface-detach Detach a network interface from a server.
interface-list List interfaces attached to a server.

keypair-add Create a new key pair for use with servers.
keypair-delete Delete keypair given by its name.
keypair-list Print a list of keypairs for a user.
keypair-show Show details about the given keypair.

list List active servers.
list-secgroup List Security Group(s) of a server.
live-migration Migrate running server to a new machine.
lock Lock a server. A normal (non-admin) user will not be able to execute actions on a locked server.
meta Set or Delete metadata on a server.
migrate Migrate a server. The new host will be selected by the scheduler.

network-associate-host Associate host with network.
network-associate-project Associate project with network.
network-create Create a network.
network-delete Delete network by label or id.
network-disassociate Disassociate host and/or project from the given network.
network-list Print a list of available networks.
network-show Show details about the given network.

pause Pause a server.

quota-class-show List the quotas for a quota class.
quota-class-update Update the quotas for a quota class.
quota-defaults List the default quotas for a tenant.
quota-delete Delete quota for a tenant/user so their quota will Revert back to default.
quota-show List the quotas for a tenant/user.
quota-update Update the quotas for a tenant/user.

rate-limits Print a list of rate limits for a user
reboot Reboot a server.
rebuild Shutdown, re-image, and re-boot a server.
refresh-network Refresh server network information.
remove-fixed-ip Remove an IP address from a server.
remove-floating-ip DEPRECATED, use floating-ip-disassociate instead.
remove-secgroup Remove a Security Group from a server.
rename Rename a server.
rescue Reboots a server into rescue mode, which starts the machine from either the initial image or a specified image, attaching the current boot disk as secondary.
reset-network Reset network of a server.
reset-state Reset the state of a server.
resize Resize a server.
resize-confirm Confirm a previous resize.
resize-revert Revert a previous resize (and return to the previous VM).
resume Resume a server.
root-password Change the admin password for a server.
scrub Delete networks and security groups associated with a project.

secgroup-add-default-rule Add a rule to the set of rules that will be added to the 'default' security group for new tenants.
secgroup-add-group-rule Add a source group rule to a security group.
secgroup-add-rule Add a rule to a security group.
secgroup-create Create a security group.
secgroup-delete Delete a security group.
secgroup-delete-default-rule Delete a rule from the set of rules that will be added to the 'default' security group for new tenants.
secgroup-delete-group-rule Delete a source group rule from a security group.
secgroup-delete-rule Delete a rule from a security group.
secgroup-list List security groups for the current tenant.
secgroup-list-default-rules List rules that will be added to the 'default' security group for new tenants.
secgroup-list-rules List rules for a security group.
secgroup-update Update a security group.

server-group-create Create a new server group with the specified details.
server-group-delete Delete specific server group(s).
server-group-get Get a specific server group.
server-group-list Print a list of all server groups.

service-delete Delete the service.
service-disable Disable the service.
service-enable Enable the service.
service-list Show a list of all running services. Filter by host & binary.

shelve Shelve a server.
shelve-offload Remove a shelved server from the compute node.
show Show details about the given server.
ssh SSH into a server.
start Start the server(s).
stop Stop the server(s).
suspend Suspend a server.
unlock Unlock a server.
unpause Unpause a server.
unrescue Restart the server from normal boot disk again.
unshelve Unshelve a server.
usage Show usage data for a single tenant.
usage-list List usage data for all tenants.
version-list List all API versions.

volume-attach Attach a volume to a server.
volume-create Add a new volume.
volume-delete Remove volume(s).

volume-detach Detach a volume from a server.
volume-list List all the volumes.
volume-show Show details about a volume.
volume-snapshot-create Add a new snapshot.
volume-snapshot-delete Remove a snapshot.
volume-snapshot-list List all the snapshots.
volume-snapshot-show Show details about a snapshot.
volume-type-create Create a new volume type.
volume-type-delete Delete a specific volume type.
volume-type-list Print a list of available 'volume types'.
volume-update Update volume attachment.

x509-create-cert Create x509 cert for a user in tenant.
x509-get-root-cert Fetch the x509 root cert.
bash-completion Prints all of the commands and options to stdout so that the nova.bash_completion script doesn't have to hard code them.
help Display help about this program or one of its subcommands.

baremetal-interface-add Add a network interface to a baremetal node.
baremetal-interface-list List network interfaces associated with a baremetal node.
baremetal-interface-remove Remove a network interface from a baremetal node.
baremetal-node-create Create a baremetal node.
baremetal-node-delete Remove a baremetal node and any associated interfaces.
baremetal-node-list Print list of available baremetal nodes.
baremetal-node-show Show information about a baremetal node.

cell-capacities Get cell capacities for all cells or a given cell.
cell-show Show details of a given cell.
force-delete Force delete a server.
restore Restore a soft-deleted server.
host-evacuate Evacuate all instances from failed host.
host-evacuate-live Live migrate all instances of the specified host to other available hosts.
host-servers-migrate Migrate all instances of the specified host to other available hosts.
instance-action Show an action.
instance-action-list List actions on a server.
list-extensions List all the os-api extensions that are available.
host-meta Set or Delete metadata on all instances of a host.
migration-list Print a list of migrations.
net DEPRECATED, Use tenant-network-show instead.
net-create DEPRECATED, use tenant-network-create instead.
net-delete DEPRECATED, use tenant-network-delete instead.
net-list DEPRECATED, use tenant-network-list instead.
tenant-network-create Create a tenant network.
tenant-network-delete Delete a tenant network.
tenant-network-list List tenant networks.
tenant-network-show Show a tenant network.

Optional arguments:

--version show program's version number and exit
--debug Print debugging output
--os-cache Use the auth token cache. Defaults to False if env[OS_CACHE] is not set.
--timings Print call timing info
--os-auth-token OS_AUTH_TOKEN Defaults to env[OS_AUTH_TOKEN]
--os-tenant-name <auth-tenant-name> Defaults to env[OS_TENANT_NAME].
--os-tenant-id <auth-tenant-id> Defaults to env[OS_TENANT_ID].
--os-region-name <region-name> Defaults to env[OS_REGION_NAME].
--os-auth-system <auth-system> Defaults to env[OS_AUTH_SYSTEM].
--service-type <service-type> Defaults to compute for most actions
--service-name <service-name> Defaults to env[NOVA_SERVICE_NAME]
--volume-service-name <volume-service-name> Defaults to env[NOVA_VOLUME_SERVICE_NAME]

--os-endpoint-type <endpoint-type> Defaults to env[NOVA_ENDPOINT_TYPE], env[OS_ENDPOINT_TYPE] or publicURL.
--os-compute-api-version <compute-api-ver> Accepts 1.1 or 3, defaults to env[OS_COMPUTE_API_VERSION].
--bypass-url <bypass-url> Use this API endpoint instead of the Service Catalog. Defaults to env[NOVACLIENT_BYPASS_URL]
--insecure Explicitly allow client to perform "insecure" TLS (https) requests. The server's certificate will not be verified against any certificate authorities. This option should be used with caution.
--os-cacert <ca-certificate> Specify a CA bundle file to use in verifying a TLS (https) server certificate. Defaults to env[OS_CACERT].
--os-cert <certificate> Defaults to env[OS_CERT].
--os-key <key> Defaults to env[OS_KEY].
--timeout <seconds> Set request timeout (in seconds).
--os-auth-url OS_AUTH_URL Authentication URL
--os-domain-id OS_DOMAIN_ID Domain ID to scope to
--os-domain-name OS_DOMAIN_NAME Domain name to scope to
--os-project-id OS_PROJECT_ID Project ID to scope to
--os-project-name OS_PROJECT_NAME Project name to scope to
--os-project-domain-id OS_PROJECT_DOMAIN_ID Domain ID containing project
--os-project-domain-name OS_PROJECT_DOMAIN_NAME Domain name containing project
--os-trust-id OS_TRUST_ID Trust ID
--os-user-id OS_USER_ID User ID
--os-user-name OS_USERNAME, --os-username OS_USERNAME Username
--os-user-domain-id OS_USER_DOMAIN_ID User's domain id
--os-user-domain-name OS_USER_DOMAIN_NAME User's domain name
--os-password OS_PASSWORD User's password .

Keystone Command groups work.

Run all the commands (as admin) indicated with "<--"

Service Catalog

catalog <-- keystone catalog

Amazon

ec2-credentials-create
ec2-credentials-delete
ec2-credentials-get
ec2-credentials-list

Endpoints

endpoint-create
endpoint-delete
endpoint-get
endpoint-list <-- keystone endpoint-list

Password Reset

password-update

Roles

role-create
role-delete
role-get
role-list <-- keystone role-list

Services

service-create
service-delete
service-get

service-list <-- keystone service-list

Tenants

tenant-create

tenant-delete

tenant-get

tenant-list <-- keystone tenant-list tenant-update

Get a token

token-get

Users

user-create

user-delete

user-get <-- keystone user-get amelirene

user-list

user-password-update

user-role-add

user-role-list <-- keystone user-role-list

user-role-remove

user-update

Other Stuff

discover <-- keystone discover

bootstrap

bash-completion

help .

Openstack Manual:

AARP - American Association of Retired Person.

Aardvark - The latest addition to the OpenStack service catalog, this service provides formally verified computations on unstructured data in linear time (as a Service) through the advanced use of string-theory based, quantum wormhole colliders. Specialized hard ware is required. It is a very useful feature, but not yet in the training course. What a burger!

Absolute Limit [6] Ceiling (limits) for guest VMs. Controlled by defining total limits on RAM, number of vCPUs, and disk size.

Active Passive Configuration [19] In a high-availability setup with an active/passive configuration, systems are set up to bring additional resources online to replace those that have failed.

Address Pool [7] A list of the fixed and/or floating IP addresses that are allocated/assigned to a project (tenant) to be used or assigned to the VM instances in a project.

amypond - The girl who waited.

API [1] Application Programming Interface. A collection of specifications used to access a service, application, or program. Includes service calls, required parameters for each call, and the expected return values.

API Endpoint [1] The daemon, worker, or service that a client communicates with to access an API. API endpoints can provide any number of services such as authentication, accounting information, performance metrics, compute (nova) VM instance commands, population density of European & African Swallows anything!

API Key [5] A term sometimes used to describe an API token.

API Token [5] Passed on API requests and used by OpenStack services to verify that this particular user is authorized to access and run the requested operation.

API Version [5] The API version can be found as part of the URL. Atest [5] This is a test.

Aurelius' - A new and power OpenStack service, this service interrogates all other services to ensure their logic is stoic, sound and virtuous.

Austin [1] The first release of OpenStack was called Austin. Unsurprisingly, the first design summit took place in this capital city of Texas. OpenStack versions are short lived, and the release dates are aggressive (about 2 new releases a year). Generally, a customer will fork an OpenStack release version, and stick with it.

Authentication Token [5] The identity service (Keystone) first confirms the identity of a user, process, or client; authorized because they've provided a private key, token, password, fingerprint, or similar security mechanism. The authentication token is a string of text returned to the user that is then included with a subsequent API request.

Back End [1] Any processes that are obfuscated from the user. Think daemons waiting in the background waiting for requests, or carrying out tasks in response to messages in a queue; an example might be the creation and deletion of volumes on storage arrays, or passing tokens with API calls.

Bare [17] A repository without a clear working directory, therefore, cannot be used for development work just maintain commits. The central repository is typically a bare repository.

Bare Metal Service [1] OpenStack project that provisions bare metal, as opposed to virtual, machines. The code name for the project is ironic.

Block Device [11] A device that moves data in the form of blocks. These device nodes interface the devices, such as hard disks, CD-ROM drives, flash drives, and other addressable regions of memory.

Blockhead Device [11] A device that moves data in the form of blocks between your ears. Also known as mental floss.

Brno A cool city in Czech Republic

BUI [2] Browser User Interface

Bursting [18] Utilizing a secondary environment to dynamically build instances on-demand when the primary environment is resource constrained.

Cache Pruner [10] Image cache has a configurable maximum size. If glance-cache-pruner is running, and the image_cache_max_size is exceeded, it will start running and try to remove images in a stalled or invalid state due to image cache writing failing to complete, or not being written properly. It is recommended that cron is used to fire off glance-cache-pruner at a regular interval.

Ceilometer [14] The project name for the Telemetry service, which is an integrated project that provides metering and measuring facilities for OpenStack.

Ceph [12] The OpenStack storage system of choice because it is massively scalable, and therefore well suited for big data. Ceph uses a distributed storage system that consists of an object and block store, as well as a POSIX-compatible distributed file system.

ceph mds [12] ceph-mds The metadata server daemon for the Ceph distributed file system. One or more instances of ceph-mds collectively manage the file system namespace, coordinating access to the shared OSD cluster. Each ceph-mds daemon instance should have a unique name. The name is used to identify daemon instances in the ceph.conf.

Chance Scheduler [6] A random scheduling method used by Compute (nova) to choose an available host from the pool of hosts.

Chef [18] OS configuration management tool supporting OpenStack deployments.

Cinder [13] An OpenStack service that provides block storage for VM instances.

Cirros [6] Packaged with Openstack, and designed to open test images.

Clone [17] Developers will typically clone a central repository so that they can develop in parallel.

Commit [17] Takes all of the files in the staging area, and stores that snapshot permanently to your Git directory. All commits can be referenced by a UUID, and include metadata such as the author, date, contact info, notes, etc. A commit is synonymous with version.

Community Project [17] Projects not officially endorsed by the OpenStack Foundation. If successful, it may become an incubated project and then to a core project, or may be merged with the main code trunk.

Compute [6] The term compute is synonymous with Nova. An OpenStack core project that provides compute services (manages VM instances).

Compute Controller [6] Chooses which hosts to start a VM instance.

Compute Host [6] A set of physical hardware that is dedicated to running compute nodes.

Compute Node [6] Runs the nova-compute daemon, which is responsible for managing VM instances.

Container [11] Organizes and stores objects in Object Storage. Similar to the concept of a Linux directory but cannot be nested. Alternative term for an Image service container format.

Container Format [10] Wrapper used by the Image service (glance) that contains a VM image and its associated metadata, such as machine state, OS disk size, and so on.

Core Project [1] Any official OpenStack Foundation projects; Compute (nova), Object Storage (swift), Image service (glance), Identity (keystone), Dashboard (horizon), Networking (neutron), and Block Storage (cinder), the Telemetry module (ceilometer), Orchestration module (heat), Database service (trove), Bare Metal service (ironic), Data processing service (sahara).

Credentials [5] Passed by a user during the authentication process; password, secret key, digital certificate, and fingerprint

Crowbar [18] An open source Dell project that aims to provide all necessary services to quickly deploy clouds.

Customer [5] Synonymous with tenant or project.

Customization Module [2] A custom made Python module loaded into Horizon to change the dashboard layout. Inclusive of visual-only, and functionality changes enhancements.

Customization Module [2] User created modules loaded into horizon to skin the dashboard differently.

DaaS [1] Desktop as a Service Another example of a possible cloud service offering. Customers have a choice of desktop environments that users access to receive a desktop experience from any location. Could be an everyday use, or testing.

Daemon [1] Processes that run in the background and wait for requests. Commonly examples include glance-api, keystone-server, neutron-server, nova-api, nova-scheduler, and so on.

Dashboard [2] Another name for Horizon, the Web Graphical User Interface (or Browser User Interface) that can be used to control OpenStack. Advanced OpenStack users will not spend much time within Horizon.

Database Service [16] An integrated project that provide scalable and reliable Cloud Database-as-a-Service functionality for both relational and non-relational database engines. The project name of Database service is trove.

Default Tenant [5] If a new user gets created and no tenant is specified at the time of creation, they will be assigned a 'default tenant' (remember, tenants are projects).

Deprecated [1] This mechanism has lost favor and may be removed from future version. An example is using the option within Compute that allows an admin to create and manage users through the nova-manage command as opposed to using the Identity service (Keystone).

DevStack [1] Great for testing and (relatively) quick to install. Community project that uses shell scripts to quickly build complete OpenStack development environments. Runs well in Debian (Ubuntu) systems

Distributed Virtual Router [7] DVR Highly-available multi-host routing using OpenStack Networking (neutron).

Domain [7] Domain can refer to an Identity API v3 entity where it would reference a collection of projects, groups and users that defines administrative boundaries for managing OpenStack Identity entities. When talking about the internet, it is the name space that is to the left of the TLD (.com, .gov., .edu, and so on) that must be unique.

Ephemeral Volume [12] A VM instance that does not save changes made to its volumes, therefore, it will return to the original state when the user relinquishes control (i.e. terminated).

ESXi [4] One of the hypervisors supported by OpenStack

Evacuate [3] The process of migrating one or all virtual machine (VM) instances from one host to another, compatible with both shared storage live migration and block migration.

External Network [7] Usually the network that a VM instance will use to access the internet.

Fan Out Exchange [15] Within RabbitMQ and Compute, it is the messaging interface that is used by the scheduler service to receive capability messages from the compute, volume, and network nodes.

Fibre Channel [13] FC A network technology commonly running at 2, 4, 8, and 16 Gbps primarily used to connect computers to data storage.

Developed during the Jurassic period of computing, millions of years ago (1988)

Fill First Scheduler [6] A Compute (nova) scheduling method to fill up a host with VM instances, as opposed to distributing those VM instances across many hosts.

Fixed IP Address [7] Also known as a static IP address. To be associated with the same instance every time that instance boots. Used for managing that instance, and not typically accessible from the public-web.

Flat Manager [7] The Compute component that gives IP address to authorized nodes and assumes DHCP, DNS, and routing configuration are provided elsewhere (such as by the FlatDHCP manager).

Flat Mode Injection [7] A compute networking method where the OS network configuration information is injected into the image before the VM instance is started.

FlatDHCP Manager [7] The compute component that gives dnsmasq (DHCP, DNS, BOOTP, TFTP) and routing services.

Flavor [6] Synonymous with VM instance type. Definable parameters include: Flavor ID, Name, VCPUs, RAM, Root Disk GB, Ephemeral Disk GB, and Swap. Simply put, it is the 'size' of the VM instance that may be launched. Default flavors are: m1.tiny, m1.small, m1.medium, m1.large, m1.xlarge.

Floating IP Address [7] IP address that a project can associate with a VM instance so that it has the same public IP address each time it boots. Create a pool of floating IP addresses, then assign them to instances (usually when they are launched) to maintain a consistent IP address for maintaining DNS assignments.

Front End [1] The point where a user interacts with a service; can be an API endpoint, the horizon dashboard, or a command-line tool.

FWaaS [1] Firewall as a Service Offering customers perimeter firewall functionality via cloud infrastructure. Marketed by Managed Security Service Providers (MSSP) as Security-as-a-Service, and Network based firewalls. Advantages include offering predictable expense (subscription based), can grow or shrink firewall bandwidth capacity as needed, and no additional devices needed on customer's site.

Gateway [7] An IP address assigned to an interface, typically on a router, that can pass network traffic from one network to another.

Git [17] Free version control software created by Linus Torvalds in 2005 for Linux kernel development. Primarily used for software development, including OpenStack. Distributed model with complete history and version tracking without relying on client-server systems.

Git Directory [17] The .git directory is where Git stores the metadata and object database for your project. This is the most important part of Git, and it is what is copied when you clone a repository from another computer. Typically exists on the central repository.

GitHub [17] A website that enhances the functionality of Git by exposing Git users to a social community. Allows users to visualize projects and branches, host repositories, share ideas, and contact information.

Glance [10] One of the OpenStack Foundation core projects that provides the Image Service.

Golden Image [10] A method of operating system installation where a finalized disk image is created and then used by all nodes without modification.

Guest OS [4] The operating system on a VM instance running under the control of a hypervisor.

Heat [18] One of the OpenStack Foundation core projects that orchestrates multiple cloud applications for OpenStack.

High Availability [19] A high availability system design approach and associated service implementation ensures that a prearranged level of operational performance will be met during a contractual measurement period. High availability systems seek to minimize system downtime and data loss.

Horizon [2] One of the OpenStack Foundation core projects that provides a Dashboard via a web GUI interface.

Host [3] Synonymous with node. Refers to a physical computer, not a VM instance.

HOT [16] Heat Orchestration Template Heat input in the format native to OpenStack.

HTTP [3] Hypertext Transfer Protocol. HTTP is an application protocol for distributed, collaborative, hypermedia information systems. It is the foundation of data communication for the World Wide Web. Hypertext is structured text that uses logical links (hyperlinks) between nodes containing text. HTTP is the protocol to exchange or transfer hypertext.

HTTPS [3] Hypertext Transfer Protocol Secure is a communications protocol for secure communication over a computer network, with especially wide deployment on the Internet. Technically, it is not a protocol in and of itself; rather, it is the result of simply layering the Hypertext Transfer Protocol (HTTP) on top of the SSL/TLS protocol, thus adding the security capabilities of SSL/TLS to standard HTTP communications.

Hyper V [4] One of the hypervisors supported by OpenStack.

Hypervisor [4] Software that controls VM instance access to the underlying hardware.

IaaS [1] Infrastructure as a Service is the outsourcing of physical components of a data center; storage, servers, hardware, and/or networking. Typically, a customer would pay to access these services.

Identity [5] Identity is provided by the Keystone project. Provides a centralized common authentication system that maps users to the OpenStack services they can access. Also registers endpoints for OpenStack services.

Image [10] Specifies the OS that you use to create or rebuild a server. OpenStack provides pre-built images. You can also create custom images, or snapshots, from servers that you have launched. Custom images can be used for data backups or as "gold" images for additional servers.

Immutable Infrastructure [1] Stability, efficiency, and fidelity to your applications through automation and the use of successful patterns from programming. No rigorous or standardized definition of immutable infrastructure exists yet, but the basic idea is that you create and operate your infrastructure using the programming concept of immutability: once you instantiate something, you never change it. Instead, you replace it with another instance to make changes or ensure proper behavior.

Incubated Project [1] The OpenStack Foundation may elevate a community project to this status before it is finally promoted to a core project.

Infiniband [13] IB A high bit rate communications link used in half of today's super computers. Can be much faster than 10Gbps Ethernet, with 2015 speeds expected to reach 300 Gps. Normally implemented with fiber, but copper links up to 30 meters are possible with IB.

Infrastructure [1] The basic physical and organizational structures and facilities used to house computer systems and associated components, such as telecommunications and storage systems.

Injection [3] Files may be put into a virtual machine image before the instance is started.

Instance [3] A running VM, or a VM in a known state such as suspended, that can be used like a hardware server. Instance, VM, VM instance are all interchangeable terms.

Instance Tunnels Network [7] A network used for instance traffic tunnels between compute nodes and the network node.

Integrity Check [11] A period comparison between the stored file's computed MD5 value and its MD5 value stored in the metadata. When the two match, the file passes this integrity check.

Interface [3] Physical or virtualized devices that interconnect devices or medium.

IroniC [15] OpenStack project that provisions bare metal, as opposed to VM instances.

JSON [5] JavaScript Object Notation One of the supported response formats in OpenStack.

Kernel Based VM [4] KVM An OpenStack-supported hypervisor. KVM is a full virtualization solution for Linux on x86 hardware containing virtualization extensions (Intel VT or AMD-V), ARM, IBM Power, and IBM zSeries. It consists of a loadable kernel module, that provides the core virtualization infrastructure and a processor specific module.

Kelli [1] One crazy old lady studying Openstack.

Keystone [5] One of the OpenStack Foundation core projects that provides the Identity Service.

Large Object [11] An object within Object Storage that is larger than 5 GB.

Linux Bridge [7] Software that allows multiple VM instances to share a single Physical NIC within Compute.

Linux Containers [4] An OpenStack supported hypervisor.

Linux Supported [1] OpenStack is supported on many Linux platforms, common deployments include: CentOS, SUSE Linux Enterprise Server (SLES), openSUSE, Debian, Ubuntu, Fedora and Red Hat Enterprise Linux.

Live Migration [3] Moving a VM instance from one host to another with a minimum service interruption.

Management Network [7] A network segment used for administration, not accessible to the public Internet.

Memory Overcommit [4] The ability to start new VM instances based on the actual memory usage of a host, as opposed to basing the decision on the amount of RAM each running instance thinks it has available. Also known as RAM overcommit.

Monitor [12] Mon A Ceph component that communicates with external clients, checks data state and consistency, and performs quorum functions.

Multi-host [7] High-availability mode for legacy (nova) networking. Each compute node handles NAT and DHCP and acts as a gateway for all of the VMs on it. A networking failure on one compute node doesn't affect VMs on other compute nodes.

Nebula [6] A NASA open source release that serves at the basis for Compute.

Network Controller [7] Compute daemon that orchestrates the network configuration of nodes, including IP addresses, VLANs, and bridging. Also manages routing for both public and private networks.

Network Manager [7] The Compute component that manages various network components, such as firewall rules, IP address allocation, and so on.

Network Namespace [4] Linux kernel feature that provides independent virtual networking instances on a single host with separate routing tables and interfaces. Similar to virtual routing and forwarding (VRF) services on physical network equipment.

Network Node [7] Any compute node that runs the network worker daemon.

Networking [7] A core OpenStack project that provides a network connectivity abstraction layer to OpenStack Compute. The project name of Networking is neutron.

Node [4] A VM instance that runs on a host.

Non-bare [17] A repository with a working directory. Cloned repositories for development work are non-bare.

Nova [6] A core OpenStack project that provides compute services.

NTP [16] Network Time Protocol A method of keeping a clock for a host or node correct through communications with a trusted, accurate time source.

Object [11] A BLOB of data held by Object Storage; includes any format.

Open vSwitch [7] Open vSwitch is a production quality, multilayer virtual switch licensed under the open source Apache 2.0 license. It is designed to enable massive network automation through programmatic extension, while still supporting standard management interfaces and protocols (for example NetFlow, sFlow, SPAN, RSPAN, CLI, LACP, 802.1ag).

OpenStack [1] A cloud operating system that controls large pools of compute, storage, and networking resources throughout a data center, all managed through a dashboard that gives administrators control while empowering their users to provision resources through a web interface. OpenStack is an open source project licensed under the Apache License 2.0.

Orchestration [16] An integrated project that orchestrates multiple cloud applications for OpenStack. The project name of Orchestration is heat.

**** Terry ****

OSD [12] Object Storage Device Is a Ceph storage daemon.

PaaS [1] Platform-as-a-Service Provides to the consumer the ability to deploy applications through a programming language or tools supported by the cloud platform provider. Think an Eclipse/Java programming platform provided with no downloads required.

Pause [3] A state a VM instance may be placed in where no changes occur (no changes in memory, network communications stop, etc); the VM is frozen but not shut down. Also known as a suspended mode.

Private Image [10] An image that is only available to a particular tenant.

Project [5] Projects represent the base unit of ownership in OpenStack, in that all resources in OpenStack should be owned by a specific project. In OpenStack Identity, a project must be owned by a specific domain. A group of users is referred to as a project, or tenant. The terms are interchangeable.

Pull [17] A locally issued request for a remote repository to return a commit on the remote repository. You pull in work from others to your own project.

Puppet [3] An OpenStack supported tool for configuration-management of operating systems.

Push [17] Local commit is pushed to a targeted library. This can be forceful, and disrupt a current working branch, therefore, a push should only ever target the central repository (which has no working directory).

Python [1] A popular programming language used in applications from Google's credit card sized RaspberryPi project to OpenStack.

QCOW2 [10] QEMU Copy On Write 2 One of the VM image disk formats supported by Image service.

Quick EMULATOR [4] QEMU is a generic and open source machine emulator and virtualizer. It is one of the hypervisors supported by OpenStack, generally used for development purposes.

RADOS [12] Reliable, Autonomic Distributed Object Store Collection of components that provides object storage within Ceph. Similar to OpenStack Object Storage.

RADOS Block Device (RBD) [12] Ceph component that enables a Linux block device to be striped over multiple distributed data stores.

RAM Overcommit [3] The ability to start new VM instances based on the actual memory usage of a host, as opposed to basing the decision on the amount of RAM each running instance thinks it has available. Also known as memory overcommit.

Raw [4] One of the VM image disk formats supported by Image service; an unstructured disk image.

Raw Device Mapping [13] RDM An option in the VMware server virtualization environment that enables a storage logical unit number (LUN) to be directly connected to a virtual machine (VM) from the storage area network (SAN).

Rebalance [11] Distributing Object Storage partitions across all drives in the ring; used during initial ring creation and after ring reconfiguration.

Reference Architecture [1] Recommended architecture for an OpenStack cloud per the OpenStack Foundation.

Replica [11] Data redundancy and fault tolerance by creating copies of Object Storage objects, accounts, and containers so that they are not lost when the underlying storage fails.

Replica Count [11] total count of replicas of the data in an Object Storage ring.

Replication [11] The process of copying data to a separate physical device for fault tolerance and performance.

Repository [17] The purpose of Git is to manage a project, or a set of files, as they change over time. Git stores this information in a data structure called a repository.

Rescue Image [3] A special type of VM image that is booted when an instance is placed into rescue mode. Allows an administrator to mount the file systems for an instance to correct the problem.

RESTful [3] A kind of web service API that uses REST, or Representational State Transfer. REST is the style of architecture for hypermedia systems that is used for the World Wide Web.

Rotted [11] Slang used by storage administrators to describe a file whose MD5 has no longer matches the MD5 hash in the metadata, indicating that file integrity was lost, most likely due to a disk error or failure.

Scheduler Manager [6] A Compute component that determines where VM instances should start. Uses modular design to support a variety of scheduler types.

Server [3] Two meanings might apply. The first, a computer that provides explicit services to the client software running on that system; LAMP, SIP, Minecraft, etc. The second is a VM instance (running within Compute).

Service Catalog [5] Also sometimes just referred to as the 'catalog', or the 'endpoint registry'. Contains a list of API endpoints that are available to a user (i.e. such as Compute, Image Service, Identity, Object Storage, Block Storage, Networking, etc.) after authentication with the Identity service.

Shared Storage [11] Block storage that is simultaneously accessible by multiple clients, for example, NFS.

Snapshot [3] A point-in-time copy of an OpenStack storage volume or image. Use storage volume snapshots to back up volumes. Use image snapshots to back up data, or as "gold" images for additional servers.

Source [1] Just like . A bash shell built-in command that executes the content of the file passed as argument, in the current shell. It has a synonym in '.'

Space [11] A new word that makes you look cool by expressing any area with finite boundaries as a 'space'. Therefore, you would not say a dog's lifespan is 12 years, you would say his lifespan is 12 twelve 365 day-spaces where you attempt to inject the word 'space' when a far more efficient expression is available but due to low vocabulary-space, the wordspace will continue to be dominated excessive use of the word 'space'.

Staging Area [17] You modify files in your working directory, then stage the files. Staged files are snapshots, but not permanently committed. Staged files are permanently snapshotted when they are committed.

Storage Node [11] An Object Storage node that provides container services, account services, and object services; controls the account databases, container databases, and object storage.

Swift [11] An OpenStack core project that provides object storage services.

TAP Traffic Access Point

Telemetry [14] An integrated project that provides metering and measuring facilities for OpenStack. The project name of Telemetry is ceilometer.

Tenant [5] It refers to a group of users; used to isolate access to Compute resources. Also known as a project, these terms are interchangeable. The initial implementation of OpenStack Compute had its own authentication system and used the term project. When authentication moved into the OpenStack Identity (keystone) project, it used the term tenant to refer to a group of users. Because of this legacy, some of the OpenStack tools refer to projects and some refer to tenants.

Thunder [27] A Cloud-based electrical discharge, may be intra-cloud, inter-cloud, or cloud-to-ground.

Token [5] An alpha-numeric string of text used to access OpenStack APIs and resources.

Trove [16] OpenStack project that provides database services to applications.

UML [4] User Mode Linux An OpenStack-supported hypervisor.

User [5] In OpenStack Identity, entities represent individual API consumers and are owned by a specific domain. In OpenStack Compute, a user can be associated with roles, projects, or both.

vCPU [4] Virtual Central Processing Unit Subdivides physical CPUs. Instances can then use those divisions.

Virtual Machine [4] VM An operating system instance that runs on top of a hypervisor. Multiple VMs can run at the same time on the same physical host. Also called a 'virtual server', or 'VM instance'.

Virtual Networking [7] A generic term for virtualization of network functions such as switching, routing, load balancing, and security using a combination of VMs and overlays on physical network infrastructure.

VirtualBox [4] An OpenStack-supported hypervisor.

VLAN Network [7] The Network Controller provides virtual networks to enable compute servers to interact with each other and with the public network. All machines must have a public and private network interface. A VLAN network is a private network interface, which is controlled by the `vlan_interface` option with VLAN managers.

Volume [13] Disk-based data storage generally represented as an iSCSI target with a file system that supports extended attributes; can be persistent or ephemeral.

***VOLTE" - an acronym for Voice over LTE, which is based on the IP Multimedia Subsystem (IMS) network, with specific profiles for control and media planes of voice service on LTE defined by GSMA in PRD IR.92.*

vSphere [4] An OpenStack-supported hypervisor.

Working Directory [17] A single version of the project. These files are pulled out of the compressed database in the Git directory and placed on disk for you to use or modify.

XaaS [1] Anything as a service.

Ingress [99] Analogous to incoming traffic or data

Egress [100] Analogous to outgoing traffic or data

Rewrite Resume. [101] In case of DR check your passport and rewrite your resume without the last DR.

References

